

DBMS PROJECT REPORT

Open-Ended Assignment

SHRUTU

Insider Threat Monitoring System

A real-time web application using Flask, MySQL 8 and Socket.IO

Course / Subject: Database Management Systems (DBMS) Project

Submitted by: Shreya Marde

Roll Number: [enter roll number]

Institution / Department: [enter institution / department]

Guide / Instructor: [enter guide / instructor]

Date of submission: 29 September 2026

Live application: <https://shrutu-insider-threat.onrender.com>

Source code: github.com/shreyamarde/insider-threat-monitor (private)

Abstract

Insider threats — employees or other trusted users who misuse their legitimate access — are among the hardest security problems for an organisation to detect, because the actions look like normal work. This project, **SHRUTU (Insider Threat Monitoring System)**, is a working web application that records every meaningful action of every employee in a relational database, scores each action with an explainable rule-based risk engine, and pushes real-time alerts to a security manager's dashboard the moment suspicious behaviour is detected, without the manager refreshing the page.

The system is built as a three-tier web application: an HTML5/CSS3/JavaScript front end (with Chart.js charts and a Socket.IO client), a Python 3 / Flask 3 application tier (Flask-SQLAlchemy, Flask-Login, Flask-WTF and Flask-SocketIO), and a **MySQL 8** database of nine normalised tables (users, departments, records, activity_logs, alerts, risk_scores, risk_events, login_attempts and risk_rules) connected by thirteen foreign keys. Activity insertion, risk-score update, explanation records and alert creation happen inside a single ACID transaction. The application provides role-based access for employees and managers, employee account management, thirteen configurable detection rules (failed logins, off-hours access, confidential and restricted record access, mass data export, privilege escalation and more), alert acknowledgement and resolution with a complete audit trail, charts and search/filter reports. It is covered by 52 automated tests (all passing) and is deployed publicly on Render with a managed MySQL database on Aiven. The expected outcome — verified end to end — is that a suspicious employee action is stored, explained and shown as an alert on the manager's screen within about one second.

Keywords: *insider threat, DBMS, MySQL, relational schema, normalisation, transactions, Flask, Socket.IO, risk scoring, role-based access control, audit trail*

Table of Contents

Abstract	1
Table of Contents	2
1. Introduction	4
1.1 Background and motivation	4
1.2 Problem statement	4
1.3 Objectives	5
1.4 Scope	5
1.5 Limitations	5
2. System Analysis	7
2.1 Existing system and its drawbacks	7
2.2 Proposed system and improvements	7
2.3 Feasibility study	8
2.4 Requirements specification	9
2.5 Use-case model	10
3. System Design	12
3.1 Architecture	12
3.2 Data Flow Diagrams	13
3.3 Entity–Relationship design	17
3.4 Schema design	19
3.5 Normalisation	24
3.6 Risk engine design	25
3.7 Real-time notification design	26
3.8 Security design	28
4. Implementation	29
4.1 Technologies used	29
4.2 Project structure	29
4.3 Key modules — coding snapshots	30
4.4 Database creation	34
4.5 Sample queries executed from the front end	39
4.6 Deployment	44
5. Testing	45
5.1 Unit testing	45
5.2 Integration testing	46
5.3 System testing (end-to-end)	47
5.4 Security and validation checks	47

5.5 Automated test run.....	48
6. Results and Snapshots.....	49
6.1 Web application screenshots.....	49
6.2 Sample queries and outputs from the DBMS	66
7. Conclusion	71
7.1 Summary of achievements.....	71
7.2 Benefits	71
7.3 Limitations	71
8. Future Enhancements.....	72
9. Bibliography / References.....	73
10. Appendix.....	74
Appendix A — Complete database creation script (schema.sql)	74
Appendix B — Source code of key modules	77
Appendix C — User manual.....	95
Appendix D — REST API endpoints	96

1. Introduction

1.1 Background and motivation

Most security tools look outwards — firewalls, antivirus and intrusion detection systems try to keep attackers out. An **insider**, however, already has a valid account, a password and permission to use the company's applications. When an insider copies payroll data before leaving, browses restricted board documents, or probes pages meant only for administrators, conventional perimeter defences see nothing unusual. Industry surveys consistently report that a large share of data breaches involve insiders, whether malicious, careless or using stolen credentials.

The only reliable way to detect such behaviour is to **record what users actually do inside applications and analyse it**: who accessed which record, when, from where, how often, and whether that is normal for them. This is fundamentally a database problem — the activity must be captured reliably, stored in a well-designed schema, queried efficiently over time windows, and linked to users, alerts and explanations. That makes insider-threat monitoring an ideal open-ended DBMS project: it exercises schema design, keys and constraints, normalisation, joins and aggregates, transactions and concurrency, and shows how a database sits at the centre of a real application.

The project started from a small prototype (a Flask login page and a static dashboard template). It has been redesigned and rebuilt into a complete system named **SHRUTU**, with a proper MySQL schema, an employee portal, a manager console, a risk engine and real-time notifications.

1.2 Problem statement

Organisations need a system that automatically answers the question **"Is any employee misusing their access right now — and why do we think so?"** Specifically, the problems to solve are:

- Employee actions inside business applications are not recorded centrally, so misuse is discovered late or never.
- Where logs exist, they are raw text that a manager must read manually; there is no scoring, prioritisation or alerting.
- Black-box risk scores are not trusted — a manager must be able to see exactly which actions increased an employee's risk and by how much.
- Managers find out about incidents only when they refresh a report; there is no real-time notification.

- The monitoring system itself must be accountable: managers' own actions (creating accounts, resetting passwords, closing alerts) must be audited.
- Access control must be enforced on the server, so that an employee cannot reach manager pages or other departments' data by typing a URL.

1.3 Objectives

- Design a normalised relational database (MySQL 8) for users, business records, activity logs, login attempts, alerts, risk scores, risk explanations and configurable rules, with primary keys, foreign keys, unique constraints and indexes.
- Build a secure web application with two roles — **employee** and **manager** — and server-side role-based access control.
- Log every meaningful action automatically through one central function, capturing time, IP address, user agent, status and resource.
- Implement an explainable, configurable rule-based risk engine with scores capped at 100 and four levels (LOW, MEDIUM, HIGH, CRITICAL).
- Generate alerts for suspicious behaviour and deliver them to managers in real time using WebSockets (Socket.IO).
- Provide manager tools: employee management, live activity feed, alert acknowledgement/resolution, risk overview, audit logs, charts, search and filters.
- Keep the database consistent by performing activity, score, explanation and alert writes inside a single transaction.
- Verify the system with automated unit, integration and system tests, and deploy it publicly.

1.4 Scope

The system covers the complete monitoring life-cycle for one organisation: account management, authentication, day-to-day record work by employees, automatic activity logging, risk evaluation, alerting, real-time notification, investigation and audit. The business data used to generate realistic activity is a set of company records (reports, invoices, contracts, personnel files, technical documents) classified as PUBLIC, INTERNAL, CONFIDENTIAL or RESTRICTED and owned by six departments. The application runs in any modern browser, including on mobile, and is deployed on the internet.

1.5 Limitations

- Monitoring is limited to actions performed inside this application; e-mail, USB devices, file servers and other systems are not monitored.

- The risk engine is rule-based. It is explainable and predictable, but it does not learn each employee's behaviour automatically (a machine-learning model is listed as future work).
- "New location" detection compares IP addresses only; it does not use geo-location.
- The free hosting tier sleeps after 15 minutes without visitors, so the first request afterwards takes about a minute.
- Socket.IO rooms are kept in the memory of one server process, so the deployment runs a single worker; scaling to several servers would need a message queue such as Redis.

2. System Analysis

2.1 Existing system and its drawbacks

Two "existing systems" were studied: the typical manual practice in small and medium organisations, and the project's own initial prototype.

2.1.1 Manual / conventional practice

In many organisations application access is either not logged at all or written to plain server log files. Reviewing them is a manual, after-the-fact activity, usually done only after an incident has already been reported. Commercial User and Entity Behaviour Analytics (UEBA) and SIEM products solve this, but they are expensive, complex to deploy and far beyond the needs and budget of a small organisation or an educational setting.

2.1.2 Initial prototype

The project began as a Flask prototype with a login page for administrators and a dashboard template. Analysis of that code showed:

Table 2.1: Drawbacks of the initial prototype

Area	Drawback found in the prototype
Functionality	Only admin login worked. The dashboard expected employees, alerts and device counts but the route never queried the database, so every number was always 0.
Monitoring	No activity logging, no risk calculation, no alerts — the core purpose was not implemented.
Roles	No employee interface; the employee page referenced a route (/submit_activity) that did not exist.
Security	Flask secret key and MySQL root password hard-coded in source code; debug mode on; no CSRF protection; role stored but never checked.
Deployment	Static files placed in the wrong folder (CSS would not load); empty requirements.txt; no database schema script; no version control.
Real time	None — data could only be seen by refreshing a page.

2.2 Proposed system and improvements

SHRUTU replaces manual log review with an automated pipeline: **every action** → **database** → **rule engine** → **alert** → **real-time notification**. The main improvements are summarised below.

Table 2.2: Comparison of existing and proposed system

Aspect	Existing situation	Proposed system (SHRUTU)
Activity capture	Not recorded / raw text logs	Every action stored in activity_logs with time, IP, user agent, resource, status and risk

Aspect	Existing situation	Proposed system (SHRUTU)
Detection	Manual reading of logs	13 configurable rules evaluated automatically on every action
Explainability	None	risk_events table stores the points, rule and reason for every score change
Alerting	After-the-fact reports	Alerts inserted in the same transaction and pushed to managers over WebSocket within ~1 s
Access control	Role not enforced	Server-side RBAC decorators; unauthorised attempts are blocked (403) and themselves raise alerts
Accountability	No audit of administrators	Every manager action (create, disable, reset, acknowledge, resolve, rule change) is audited
Data integrity	No schema script	Normalised MySQL schema with 13 foreign keys, unique constraints, indexes and transactions
Cost	Commercial UEBA/SIEM is expensive	Built entirely with free, open-source tools; hosted on free tiers

2.3 Feasibility study

2.3.1 Technical feasibility

All required technologies are mature, free and well documented: Python 3 and Flask for the web tier, SQLAlchemy as the ORM, MySQL 8 (InnoDB) for transactional storage with foreign keys, Flask-SocketIO for WebSockets and Chart.js for charts. The system runs on an ordinary laptop (tested on Windows 11 with Python 3.14 and MySQL 8.0.41) and on a Linux cloud server (Render, Python 3.14, gunicorn). The development team needed only standard DBMS and web-programming knowledge. The project is therefore technically feasible, which was confirmed by the working implementation and 52 passing automated tests.

2.3.2 Economic feasibility

Table 2.3: Cost estimate

Item	Technology / service	Cost
Programming language & framework	Python 3, Flask 3 and extensions	Free (open source)
Database server	MySQL 8 Community Edition (local); Aiven for MySQL free plan (cloud)	Free
Front-end libraries	Chart.js, Socket.IO client, Google Font Inter	Free
Hosting	Render free web service (HTTPS included)	Free
Source control	Git + private GitHub repository	Free
Development tools	VS Code, MySQL Workbench, web browser	Free
Total		₹ 0

Because the whole stack is open source and the hosting uses free tiers, the system has no licence or infrastructure cost, making it economically feasible for small organisations and for academic use. Upgrading to always-on paid tiers would cost roughly USD 5–7 per month.

2.3.3 Operational feasibility

The interface is a familiar web dashboard with a dark/light theme and works on desktops and phones. Employees need no training: monitoring is automatic and invisible to them apart from a notice that the system is monitored. Managers see prioritised, colour-coded alerts with plain-English reasons (for example "Accessed 17 records within 5 minutes") and act with one click (Acknowledge / Resolve). Detection thresholds can be tuned from a Settings screen without changing code. The system is therefore operationally feasible.

2.4 Requirements specification

2.4.1 Functional requirements

Table 2.4: *Functional requirements*

ID	Requirement
FR-1	Users log in with employee ID and password; managers and employees are redirected to their own dashboards.
FR-2	Accounts created or reset by a manager must change their temporary password at first login.
FR-3	Employees can search, view, add, edit and export records they are permitted to access.
FR-4	Every meaningful action (login, logout, failed login, search, view, add, update, export, unauthorised access, password change, all manager actions) is logged automatically.
FR-5	Each activity is evaluated by the risk engine; risk points and reasons are stored; the employee's score (0–100) and level are updated.
FR-6	Alerts are created for suspicious behaviour and pushed to all connected managers in real time.
FR-7	Managers can create employees, edit details, enable/disable accounts and reset passwords.
FR-8	Managers can view the live activity feed, alerts, risk overview, charts, audit logs and login attempts, with search and filters.
FR-9	Managers can acknowledge and resolve alerts; alerts are never deleted.
FR-10	Managers can tune detection rules (points, threshold, time window, severity, enabled).
FR-11	The system shows which employees are online (heartbeat-based presence).

2.4.2 Non-functional requirements

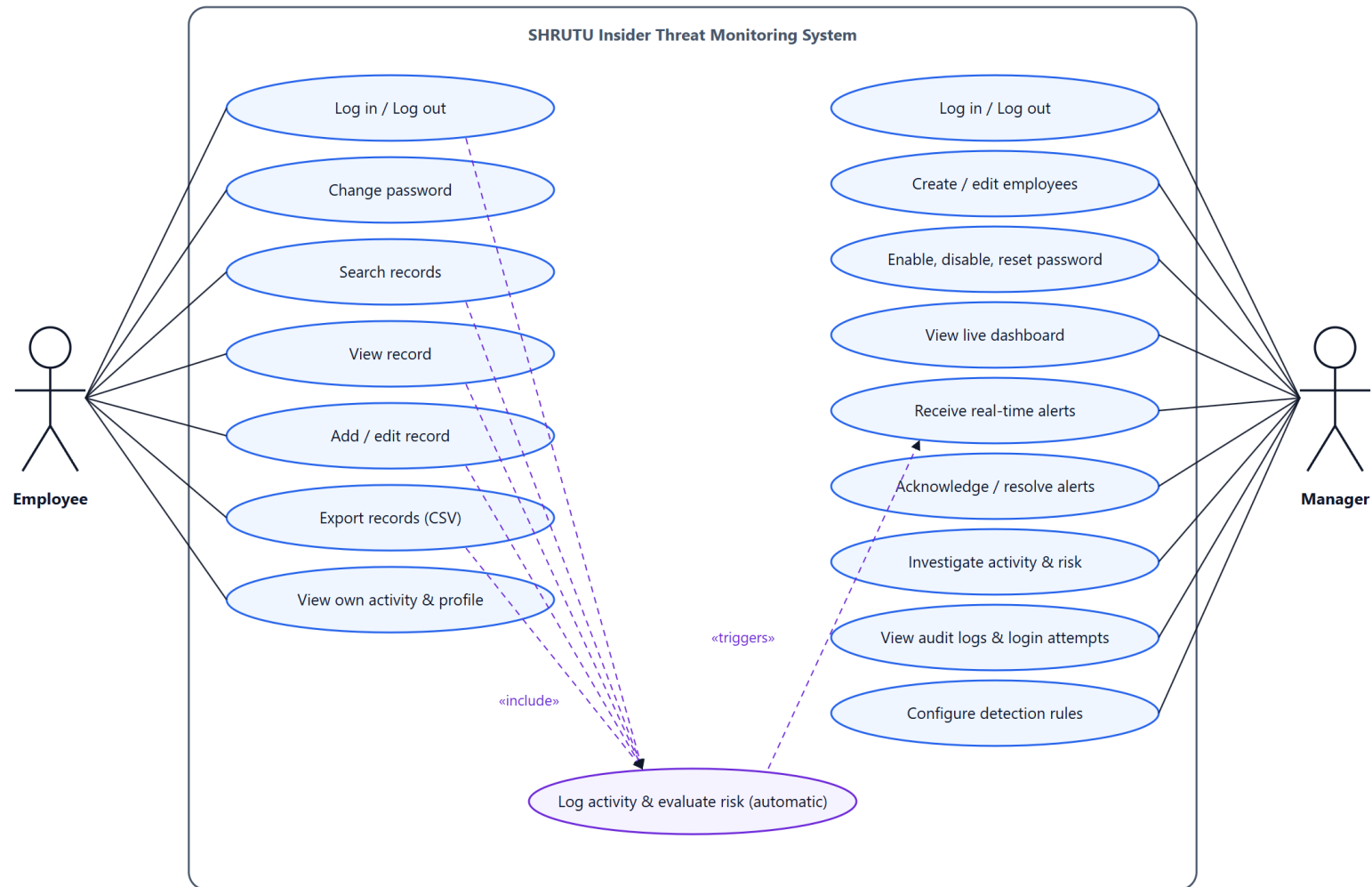
Table 2.5: *Non-functional requirements*

Category	Requirement
Security	Hashed passwords (bcrypt), CSRF protection, server-side RBAC, account lockout, session timeout, security headers, no secrets in source code.
Integrity	Referential integrity through foreign keys; multi-table writes in one transaction; audit data never deleted.

Category	Requirement
Performance	Indexes on timestamps, user and employee IDs; alert delivered to managers within about one second.
Usability	Responsive UI (desktop and mobile), dark/light theme, colour + text labels for severity (not colour alone).
Explainability	Every risk point must have a stored human-readable reason.
Maintainability	Modular code (models / routes / services), configuration in one file, 52 automated tests.

2.5 Use-case model

Figure 2.1 shows the two actors and their use cases. Every employee use case «includes» automatic activity logging and risk evaluation, which in turn «triggers» the manager's real-time alerts.

Figure: Use-case diagram**Figure 2.1: Use-case diagram**

3. System Design

3.1 Architecture

SHRUTU follows a classic **three-tier architecture** (Figure 3.1). The presentation tier is the browser: Jinja2-rendered HTML pages, JavaScript that calls the REST API with `fetch()`, Chart.js charts, and a Socket.IO client for live events. The application tier is a Flask 3 server organised into blueprints (`auth`, `employee`, `manager`, `api`), a security layer, a services layer containing the business logic, and SQLAlchemy ORM models. The database tier is MySQL 8, reached through the PyMySQL driver over TLS in production.

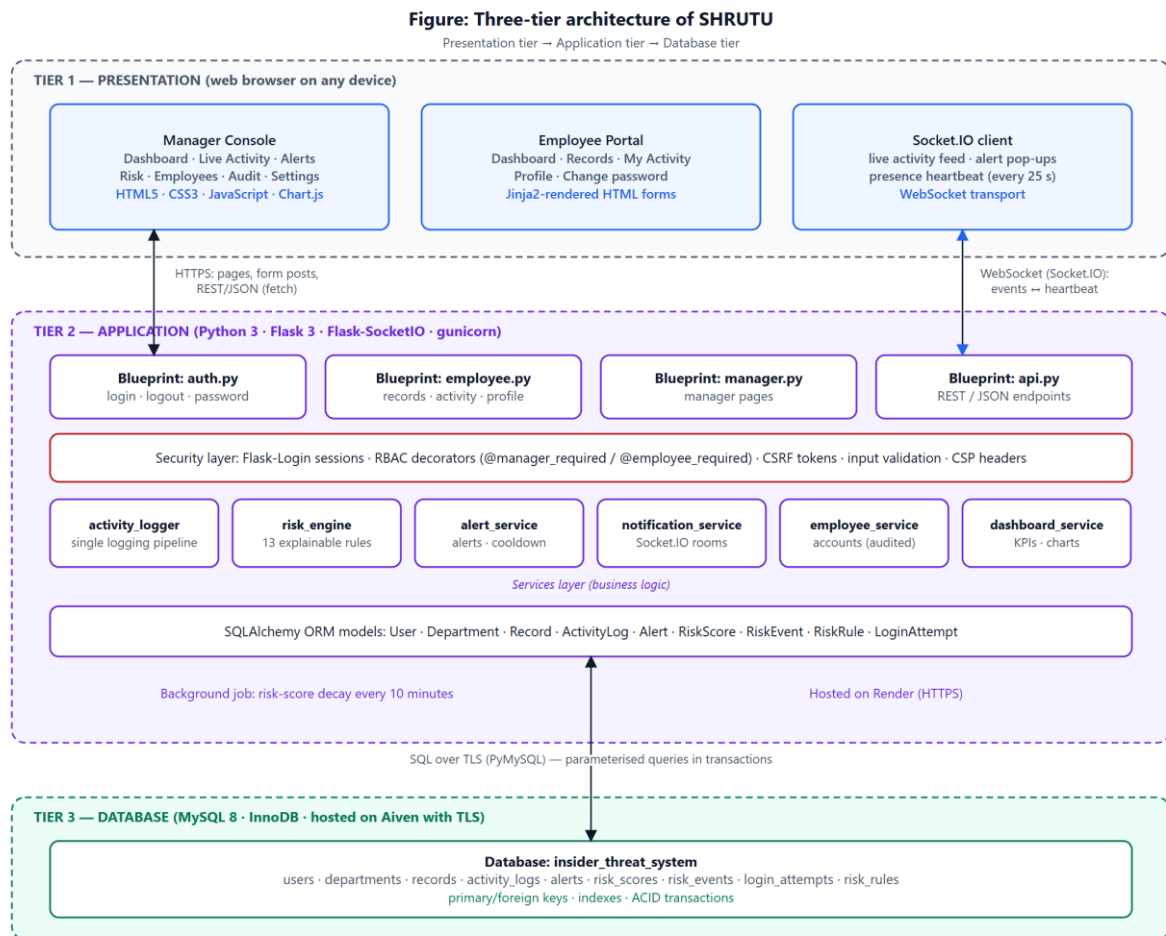


Figure 3.1: *Three-tier architecture of SHRUTU*

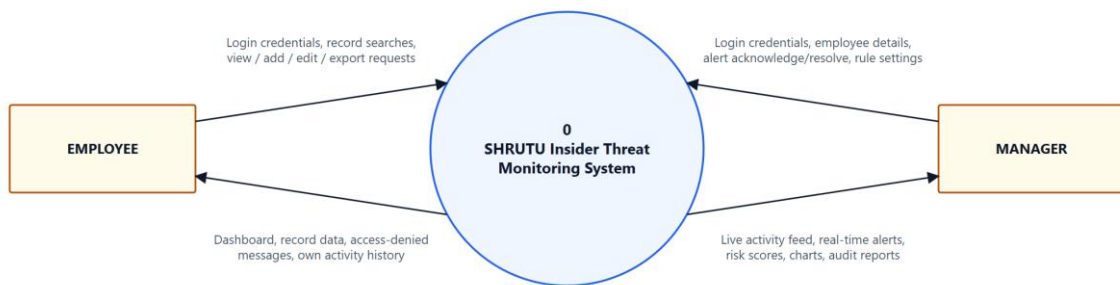
The most important design decision is the **single monitoring pipeline**: every route that performs a meaningful action calls one function, `log_activity()`, which inserts the activity, evaluates the rules, updates the score, writes the explanations and creates alerts inside one database transaction, and only after COMMIT emits Socket.IO events. This guarantees that every notification a manager sees corresponds to data that is already safely stored.

3.2 Data Flow Diagrams

3.2.1 Level 0 — Context diagram

At the highest level the system exchanges data with two external entities, Employee and Manager (Figure 3.2).

Figure: Data Flow Diagram — Level 0 (Context diagram)



External entities are shown as rectangles; the whole system is a single process.

Figure 3.2: DFD Level 0 (context diagram)

3.2.2 Level 1

Level 1 (Figure 3.3) decomposes the system into eight processes and seven data stores. Processes 1.0, 2.0 and 7.0 handle user-facing work; 3.0 to 6.0 form the monitoring pipeline; 8.0 serves the manager's monitoring, reporting and alert handling.

Figure: Data Flow Diagram — Level 1

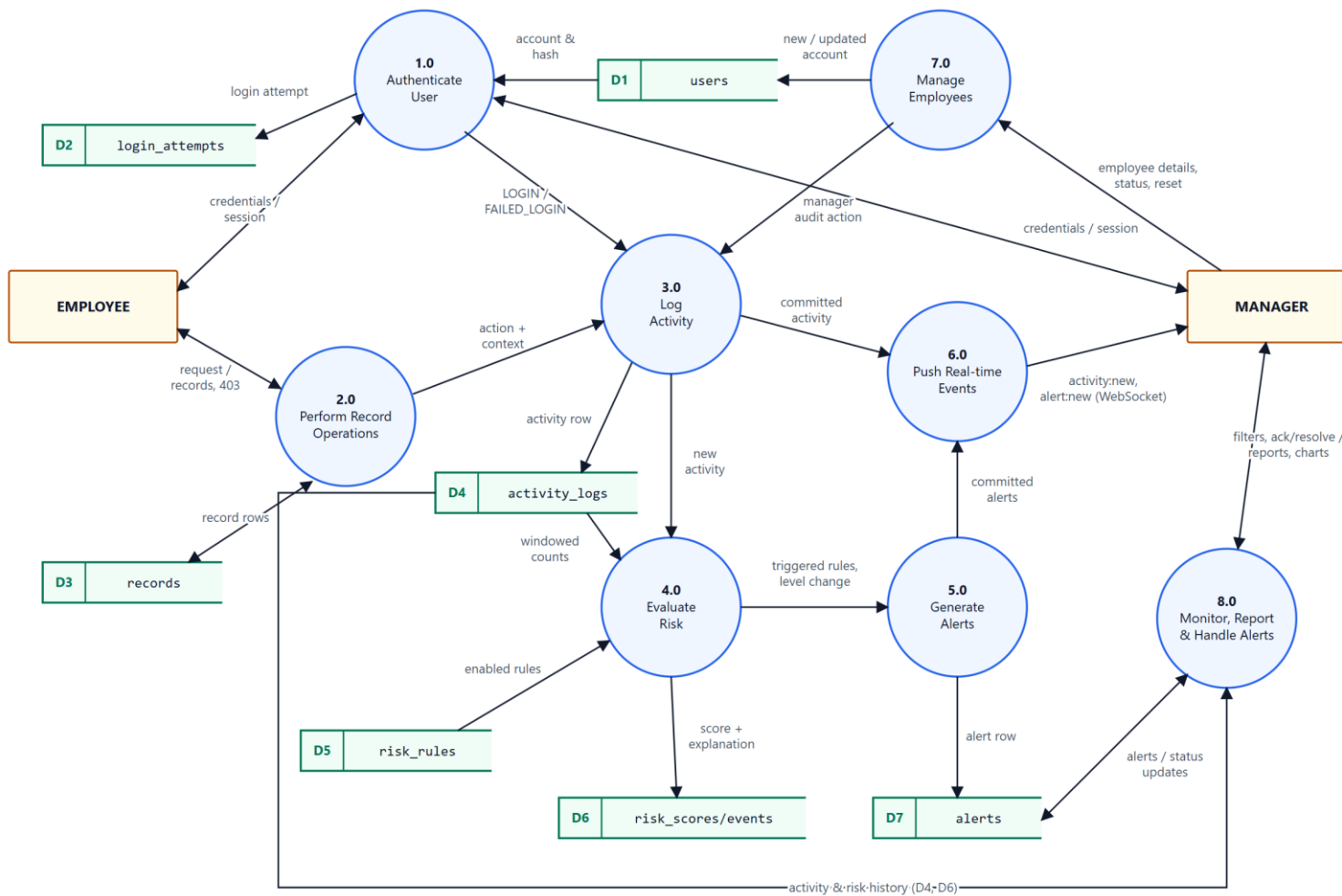


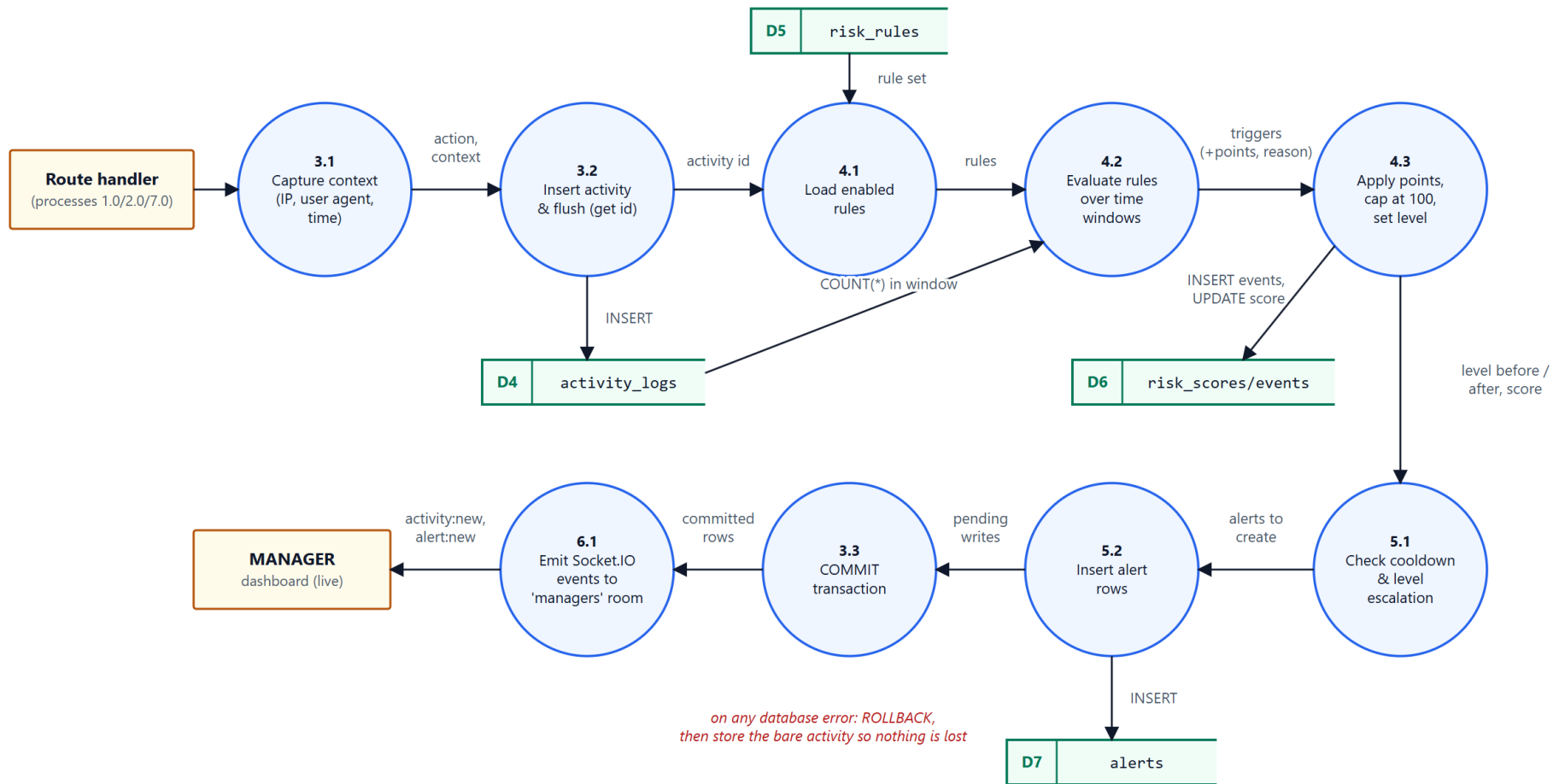
Figure 3.3: DFD Level 1

Table 3.1: *Level 1 processes*

Process	Description	Data stores
1.0 Authenticate User	Verifies employee ID and password hash, applies lockout, records the attempt and starts the session.	D1 users, D2 login_attempts
2.0 Perform Record Operations	Search, view, add, edit and export records after a permission check.	D3 records
3.0 Log Activity	Stores every action with context (IP, user agent, time, status).	D4 activity_logs
4.0 Evaluate Risk	Loads enabled rules, counts recent activity in time windows, applies points and levels.	D4, D5 risk_rules, D6 risk_scores/risk_events
5.0 Generate Alerts	Creates alerts for rules with a severity, respecting cooldown, plus level-escalation alerts.	D7 alerts
6.0 Push Real-time Events	Emits activity and alert events to the managers' Socket.IO room after commit.	—
7.0 Manage Employees	Creates and updates accounts, enables/disables, resets passwords (all audited).	D1 users
8.0 Monitor, Report & Handle Alerts	Dashboards, charts, filters, investigation, acknowledge/resolve.	D4, D6, D7

3.2.3 Level 2 — the monitoring pipeline

Figure 3.4 expands processes 3.0–6.0. The route handler passes the action to 3.1, which captures the request context; 3.2 inserts and flushes the activity to obtain its primary key; 4.1–4.3 evaluate the rules and update the score; 5.1–5.2 create alerts; 3.3 commits; and 6.1 notifies managers. A database error at any step rolls back the whole transaction and the bare activity is stored separately so that no evidence is lost.

Figure: Data Flow Diagram — Level 2 (processes 3.0–6.0: the monitoring pipeline)

All writes from 3.2 to 5.2 belong to ONE database transaction; notifications are sent only after COMMIT.

Figure 3.4: DFD Level 2 (monitoring pipeline)

3.3 Entity–Relationship design

The conceptual model contains nine entities (Figure 3.5). USER is the central entity: every user belongs to one DEPARTMENT, has exactly one RISK_SCORE, makes LOGIN_ATTEMPTs, performs ACTIVITY_LOGs, creates RECORDs and is the subject of ALERTs, while managers HANDLE (acknowledge/resolve) alerts. An ACTIVITY_LOG TRIGGERS alerts and CAUSES RISK_EVENTS, and each RISK_EVENT APPLIES one RISK_RULE. USER also has a recursive CREATED_BY relationship (the manager who created an account).

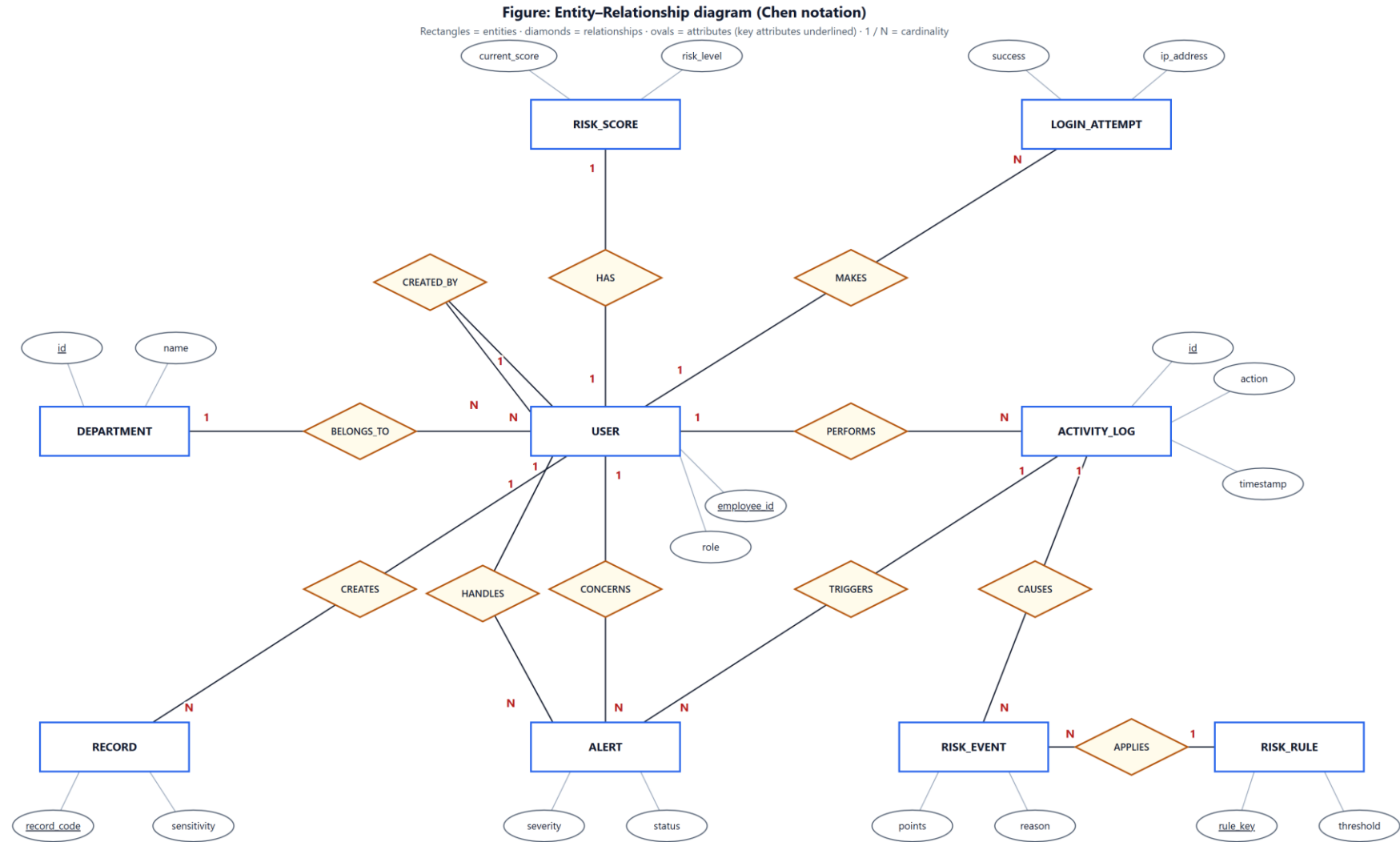


Figure 3.5: Entity–Relationship diagram (Chen notation)

Table 3.2: Relationships and their implementation

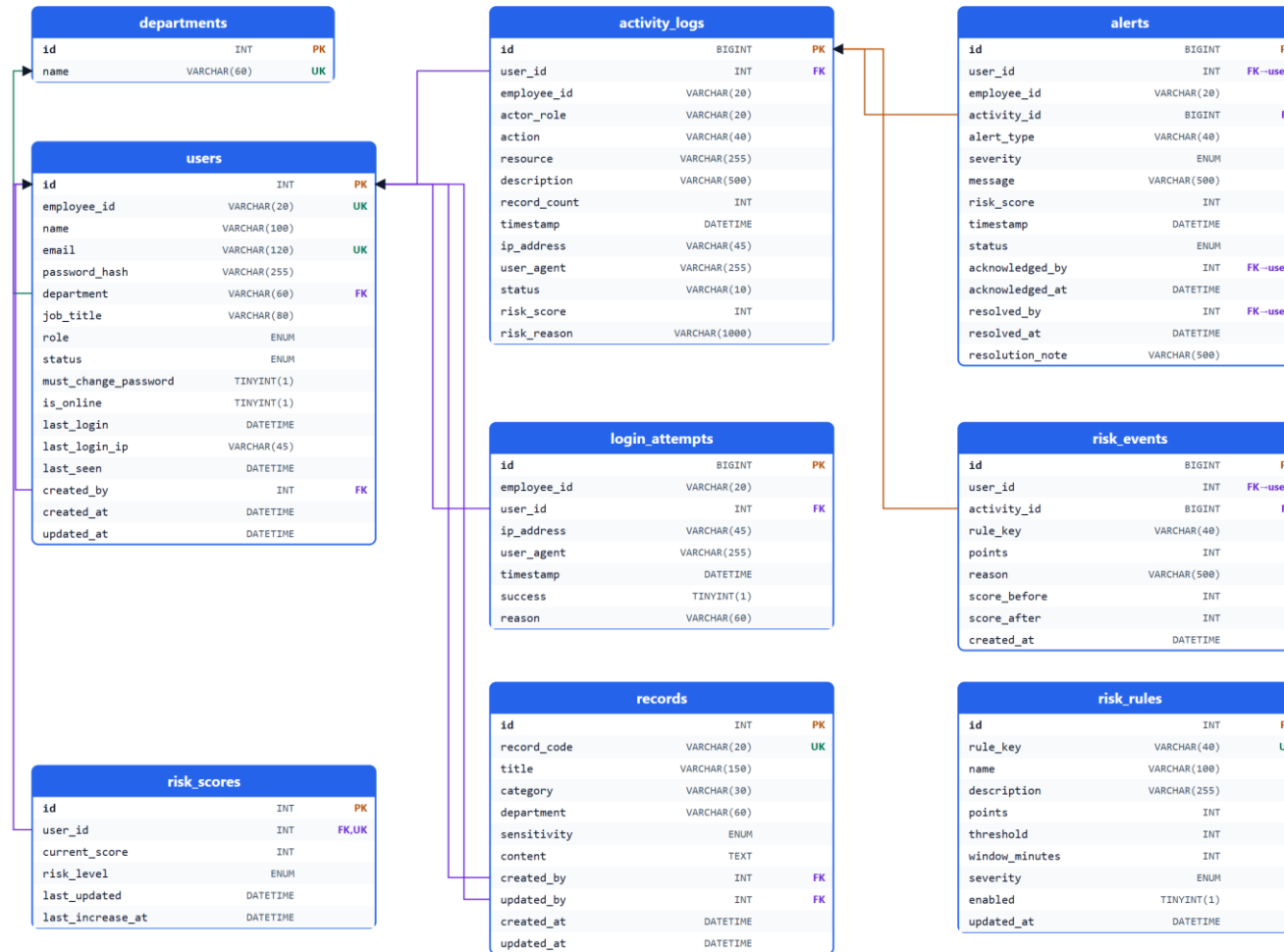
Relationship	Entities	Cardinality	Implemented by
BELONGS_TO	DEPARTMENT – USER	1 : N	users.department → departments.name
HAS	USER – RISK_SCORE	1 : 1	risk_scores.user_id (FK + UNIQUE)
MAKES	USER – LOGIN_ATTEMPT	1 : N	login_attempts.user_id (nullable for unknown IDs)
PERFORMS	USER – ACTIVITY_LOG	1 : N	activity_logs.user_id
CREATES	USER – RECORD	1 : N	records.created_by, records.updated_by
CONCERNS	USER – ALERT	1 : N	alerts.user_id
HANDLES	USER (manager) – ALERT	1 : N	alerts.acknowledged_by, alerts.resolved_by
TRIGGERS	ACTIVITY_LOG – ALERT	1 : N	alerts.activity_id
CAUSES	ACTIVITY_LOG – RISK_EVENT	1 : N	risk_events.activity_id
APPLIES	RISK_RULE – RISK_EVENT	1 : N	risk_events.rule_key (logical link)
CREATED_BY	USER – USER (recursive)	1 : N	users.created_by → users.id

3.4 Schema design

The logical design maps each entity to a table. Surrogate integer primary keys (INT or BIGINT AUTO_INCREMENT for fast-growing log tables) are used everywhere; natural identifiers such as `employee_id`, `email`, `record_code` and `rule_key` are enforced with UNIQUE constraints. Enumerated domains use MySQL ENUM columns. Figure 3.6 shows the physical schema, and Tables 3.3–3.11 describe each table.

Figure: Relational schema diagram (physical design, MySQL 8)

PK = primary key · FK = foreign key · UK = unique · arrows point from the foreign key to the referenced key



Columns marked FK--users (alerts.user_id, acknowledged_by, resolved_by, risk_events.user_id) also reference users.id.
 risk_events.rule_key → risk_rules.rule_key is a logical (application-level) link.

Figure 3.6: Relational schema diagram (physical design)

Table 3.3: *Table users — Employees and managers*

Column	Data type	Constraint	Description
id	INT AUTO_INCREMENT	PK	Surrogate key
employee_id	VARCHAR(20)	UNIQUE, NOT NULL	Login ID, e.g. EMP1001
name	VARCHAR(100)	NOT NULL	Full name
email	VARCHAR(120)	UNIQUE, NOT NULL	E-mail address
password_hash	VARCHAR(255)	NOT NULL	Werkzeug script hash (never plain text)
department	VARCHAR(60)	FK → departments.name	Department
job_title	VARCHAR(80)	NULL	Designation
role	ENUM('employee','manager')	NOT NULL	Access role
status	ENUM('active','disabled')	NOT NULL	Disabled accounts cannot log in
must_change_password	TINYINT(1)	NOT NULL	Temporary password flag
is_online / last_seen	TINYINT(1) / DATETIME		Presence tracking
last_login / last_login_ip	DATETIME / VARCHAR(45)		Last successful login
created_by	INT	FK → users.id	Manager who created the account
created_at / updated_at	DATETIME	NOT NULL	Timestamps

Table 3.4: *Table departments — Lookup of departments*

Column	Data type	Constraint	Description
id	INT	PK	
name	VARCHAR(60)	UNIQUE	Finance, Human Resources, Engineering, IT & Security, Sales, Legal

Table 3.5: *Table records — Business records (the monitored resource)*

Column	Data type	Constraint	Description
id	INT	PK	
record_code	VARCHAR(20)	UNIQUE	e.g. FIN-0004
title / category	VARCHAR	NOT NULL	
department	VARCHAR(60)	INDEX	Owning department or 'General'
sensitivity	ENUM(PUBLIC, INTERNAL, CONFIDENTIAL, RESTRICTED)	NOT NULL	Classification used by the access policy
content	TEXT	NOT NULL	
created_by / updated_by	INT	FK → users.id	

Column	Data type	Constraint	Description
created_at / updated_at	DATETIME		

Table 3.6: Table activity_logs — Every action of every user

Column	Data type	Constraint	Description
id	BIGINT	PK	
user_id	INT	FK → users.id (NULL for unknown IDs)	
employee_id	VARCHAR(20)	INDEX	Kept even for failed logins of unknown IDs
actor_role	VARCHAR(20)		employee / manager
action	VARCHAR(40)	INDEX	LOGIN, SEARCH, VIEW_RECORD, EXPORT_DATA, UNAUTHORIZED_ACCESS, ...
resource / description	VARCHAR(255) / VARCHAR(500)		What was accessed
record_count	INT		Records touched (for mass-access rule)
timestamp	DATETIME	INDEX	
ip_address / user_agent	VARCHAR		Request context
status	VARCHAR(10)		SUCCESS / FAILED / DENIED
risk_score / risk_reason	INT / VARCHAR(1000)		Points added by this action and why

Table 3.7: Table alerts — Suspicious events (never deleted)

Column	Data type	Constraint	Description
id	BIGINT	PK	
user_id	INT	FK → users.id	
employee_id	VARCHAR(20)	INDEX	
activity_id	BIGINT	FK → activity_logs.id	Triggering activity
alert_type	VARCHAR(40)		Rule key
severity	ENUM(LOW, MEDIUM, HIGH, CRITICAL)		
message	VARCHAR(500)		Human-readable reason
risk_score	INT		Score at alert time
status	ENUM(NEW, ACKNOWLEDGED, RESOLVED)	INDEX	
acknowledged_by / resolved_by	INT	FK → users.id	Handling manager

Column	Data type	Constraint	Description
acknowledged_at / resolved_at	DATETIME		
resolution_note	VARCHAR(500)		

Table 3.8: *Table risk_scores — Current score per user*

Column	Data type	Constraint	Description
id	INT	PK	
user_id	INT	FK → users.id, UNIQUE	One row per user (1:1)
current_score	INT		0–100
risk_level	ENUM(LOW, MEDIUM, HIGH, CRITICAL)		
last_updated / last_increase_at	DATETIME		Used for decay

Table 3.9: *Table risk_events — Explanation of every score change*

Column	Data type	Constraint	Description
id	BIGINT	PK	
user_id	INT	FK → users.id	
activity_id	BIGINT	FK → activity_logs.id	
rule_key	VARCHAR(40)		Rule that fired
points	INT		
reason	VARCHAR(500)		e.g. "Accessed 17 records within 5 minutes"
score_before / score_after	INT		
created_at	DATETIME	INDEX	

Table 3.10: *Table login_attempts — Authentication attempts*

Column	Data type	Constraint	Description
id	BIGINT	PK	
employee_id	VARCHAR(20)	INDEX	
user_id	INT	FK → users.id (nullable)	
ip_address / user_agent	VARCHAR	INDEX (ip, time)	
timestamp	DATETIME		
success	TINYINT(1)		

Column	Data type	Constraint	Description
reason	VARCHAR(60)		INVALID_PASSWORD, LOCKED_OUT, ACCOUNT_DISABLED, ...

Table 3.11: *Table risk_rules — Configurable detection rules*

Column	Data type	Constraint	Description
id	INT	PK	
rule_key	VARCHAR(40)	UNIQUE	
name / description	VARCHAR		
points	INT		Points added
threshold / window_minutes	INT		e.g. 15 records in 5 minutes
severity	ENUM(NONE, LOW, MEDIUM, HIGH, CRITICAL)		Alert severity
enabled	TINYINT(1)		

3.5 Normalisation

The schema was derived by normalising a single, flat "monitoring event" relation that a naive design might use:

Unnormalised relation

```
EVENT(event_id, employee_id, employee_name, email, department, role, action, record_code,
record_title,
      record_department, sensitivity, time, ip, alert_type, alert_severity, alert_status,
handled_by_name,
      current_risk_score, risk_level, rule_name, rule_points)
```

First Normal Form (1NF)

A single event could raise several alerts and several rule points, giving repeating groups (alert_type₁, alert_type₂, ...). These were moved into separate relations with one value per cell, and every relation received a primary key: ALERT(alert_id, event_id, alert_type, severity, status, ...) and RISK_EVENT(risk_event_id, event_id, rule_key, points, reason, ...).

Second Normal Form (2NF)

With surrogate single-column keys there are no partial dependencies on part of a composite key. The candidate composite key (employee_id, time) of the activity relation was replaced by the surrogate id, so every non-key attribute depends on the whole key.

Third Normal Form (3NF)

Transitive dependencies were removed: `employee_id` → `name`, `email`, `department`, `role` moved to `USERS`; `record_code` → `title`, `record_department`, `sensitivity` moved to `RECORDS`; `rule_key` → `rule_name`, `points`, `threshold` moved to `RISK_RULES`; `employee_id` → `current_risk_score`, `risk_level` moved to `RISK_SCORES` (1:1); and `departments` became a lookup table referenced by foreign key. Each table now describes one entity and every non-key attribute depends only on its key.

Deliberate, documented exceptions

- `activity_logs.employee_id`, `alerts.employee_id` and `login_attempts.employee_id` repeat the typed employee ID next to the `user_id` foreign key. This is intentional: failed logins with an **unknown** ID have no user row, and audit evidence must remain readable even if an account is later renamed.
- `alerts.risk_score` and `risk_events.score_before/score_after` are **historical snapshots** (the value at that moment), not derived data, so they do not violate 3NF.
- `activity_logs.risk_reason` is a denormalised summary of the related `risk_events` rows kept for fast display in the live feed; `risk_events` remains the authoritative explanation.

3.6 Risk engine design

Rules are stored as data in `risk_rules` (Table 3.12). When an activity arrives, each enabled rule runs a small query — usually a `COUNT` or `SUM` over the employee's recent activity within the rule's time window — and, if it fires, returns points, a reason and an alert severity. Scores are capped at 100 and mapped to levels: 0–29 LOW, 30–59 MEDIUM, 60–79 HIGH, 80–100 CRITICAL. Aggregate rules fire once per window, the same alert type is not repeated within a 10-minute cooldown, and scores decay by 2 points per hour of normal behaviour.

Table 3.12: *Detection rules (default configuration)*

Rule	Points	Fires when	Alert
FAILED_LOGIN	+10	any failed login	—
REPEATED_FAILED_LOGIN	+20	≥ 3 failed logins for one ID in 10 min	HIGH
NEW_LOCATION	+10	login from an IP never used by this employee	MEDIUM
OFF_HOURS	+15	activity outside 09:00–19:00 Mon–Fri (once per hour)	MEDIUM
SENSITIVE_ACCESS	+15	viewing / editing a CONFIDENTIAL record	MEDIUM
UNAUTHORIZED_ACCESS	+30	opening a record or page outside permissions	HIGH
PRIVILEGE_ESCALATION	+10	employee tries a manager-only page or API	HIGH
REPEATED_UNAUTHORIZED	+25	≥ 3 unauthorised attempts in 15 min	CRITICAL
MASS_RECORD_ACCESS	+20	≥ 15 records viewed/exported in 5 min	HIGH
REPEATED_SEARCH	+10	≥ 10 searches in 5 min	LOW

Rule	Points	Fires when	Alert
RAPID_ACTIONS	+15	≥ 20 actions in 1 min	MEDIUM
UNUSUAL_FREQUENCY	+15	$> 3 \times$ the employee's normal hourly rate (7-day baseline)	MEDIUM
SUSPICIOUS_BURST	+25	≥ 3 risk-scored events (not failed logins) in 10 min	HIGH

3.7 Real-time notification design

Figure 3.7 traces a suspicious action from the employee's browser to the manager's screen. Socket.IO connections are authenticated with the same session cookie as HTTP requests. The server places every manager socket in a private room called `managers`; employees are never subscribed to it, so they cannot receive monitoring data even by modifying client code.

Figure: Sequence diagram — suspicious action to real-time manager alert

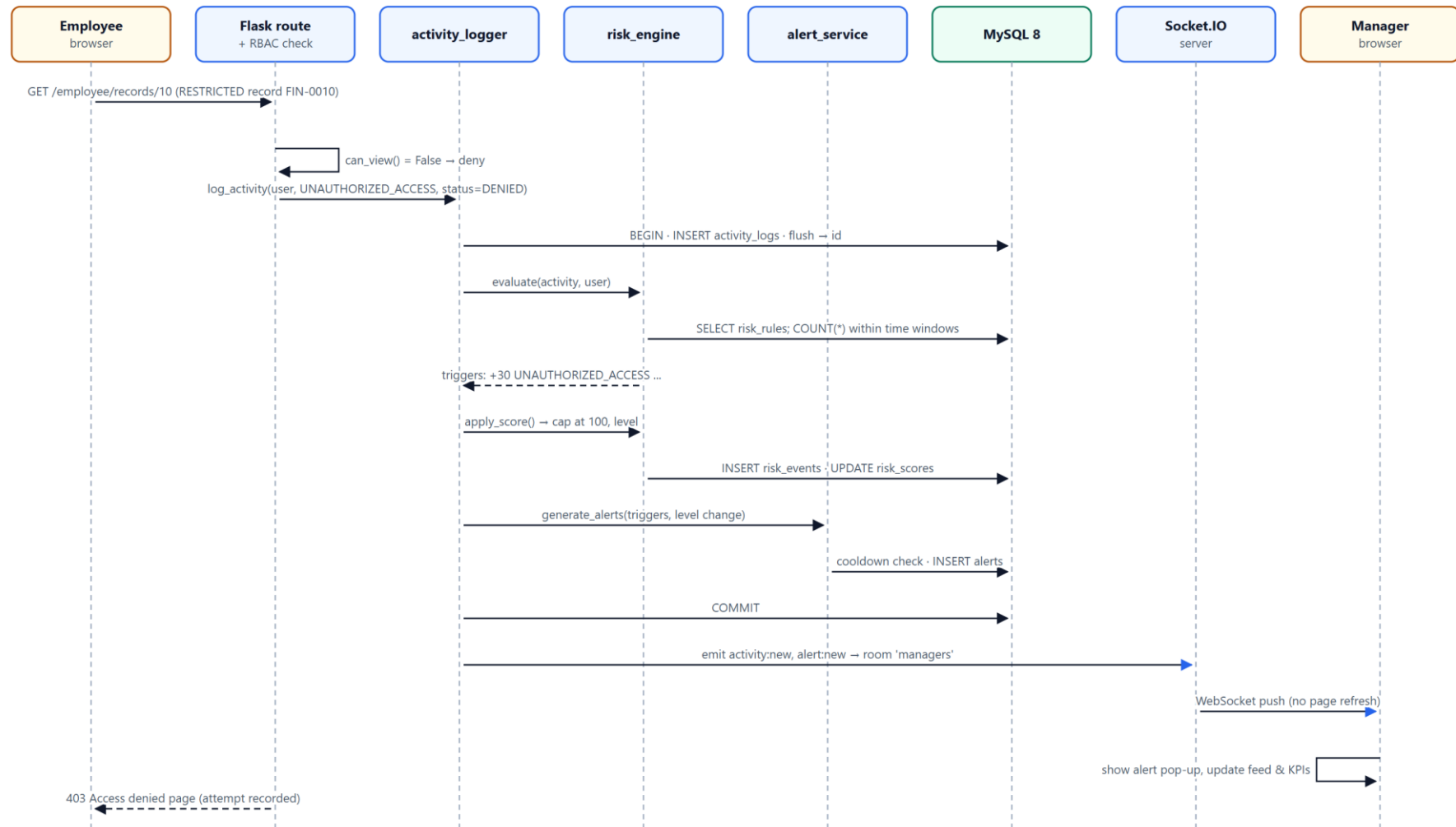


Figure 3.7: Sequence diagram: suspicious action to real-time alert

3.8 Security design

- **Authentication:** passwords hashed with scrypt; generic error messages; logout after 5 failures per ID or 20 per IP in 15 minutes; session cleared on login (prevents fixation).
- **Authorisation:** @login_required, @manager_required, @employee_required decorators on every route and API; record-level policy by department and classification.
- **Session security:** HttpOnly, SameSite=Lax (and Secure over HTTPS) cookies; 30-minute idle timeout; disabled accounts lose their session immediately.
- **Input handling:** server-side validation of every field; SQLAlchemy parameter binding prevents SQL injection; Jinja2 auto-escaping prevents XSS.
- **Browser protections:** CSRF tokens on forms and API writes; Content-Security-Policy without inline scripts; X-Frame-Options DENY.
- **Secrets:** database credentials and secret key only in environment variables (.env, git-ignored); database connections over TLS in production.

4. Implementation

4.1 Technologies used

Table 4.1: *Technologies used*

Layer	Technology	Purpose
Front end	HTML5, CSS3 (custom design system, dark/light), JavaScript (ES2020)	User interface
Front end	Jinja2 templates	Server-side rendering of pages and forms
Front end	Chart.js 4, Socket.IO client 4	Charts; real-time events
Back end	Python 3.14, Flask 3.1	Web framework and routing
Back end	Flask-SQLAlchemy 3.1 / SQLAlchemy 2.1	ORM, transactions, query building
Back end	Flask-Login, Flask-WTF, Werkzeug	Sessions, CSRF protection, password hashing
Back end	Flask-SocketIO 5 + simple-websocket	WebSocket server (real-time alerts)
Database	MySQL 8.0 (local) / MySQL 8.4 on Aiven (cloud), InnoDB	Relational storage with foreign keys and ACID transactions
Database driver	PyMySQL	Python ↔ MySQL connection (TLS)
Testing	pytest 9	Unit, integration and real-time tests
Deployment	gunicorn, Render (web), Aiven (MySQL), Git/GitHub	Public hosting and source control
Tools	VS Code, MySQL Workbench / CLI, Microsoft Edge DevTools	Development and debugging

4.2 Project structure

```

insider-threat-monitor/
├── app.py                application factory, error handling, security headers
├── config.py             configuration (environment variables + monitoring policy)
├── extensions.py         db, login_manager, csrf, socketio objects
├── init_db.py            creates the database from schema.sql and loads seed.sql
├── seed_extra.py         adds 10 more demo employees with realistic history
├── wsgi.py               production entry point (gunicorn)
├── database/             schema.sql · seed.sql
├── models/               user.py · activity.py · alert.py · risk.py · record.py
├── routes/               auth.py · employee.py · manager.py · api.py
├── services/             activity_logger.py · risk_engine.py · alert_service.py · notification_service.py
                        employee_service.py · dashboard_service.py · record_access.py
├── utils/                security.py · validators.py · helpers.py
├── templates/            login.html · layout.html · employee/*.html · manager/*.html · errors/error.html
├── static/               css/app.css · css/login.css · js/*.js (common, realtime, charts, page scripts)
├── tests/                52 automated tests (pytest)

```

Table 4.2: *Size of the implementation*

Component	Size
Python back end (models, routes, services, utils, app)	≈ 3,000 lines in 29 files
Jinja2 templates	21 files, ≈ 1,050 lines

Component	Size
JavaScript / CSS	≈ 1,120 / ≈ 620 lines
SQL (schema + seed)	≈ 530 lines
Automated tests	52 tests, ≈ 710 lines
HTTP endpoints	47 routes (25 REST API endpoints)

4.3 Key modules — coding snapshots

4.3.1 Central activity logger (one transaction)

`log_activity()` is the heart of the system. It inserts the activity, runs the risk engine, stores the explanation, updates the score and creates alerts in one transaction, and emits real-time events only after COMMIT. If anything fails the transaction is rolled back and the bare activity is stored so that evidence is never lost.

Listing 4.1 — `services/activity_logger.py`

```
def log_activity(user, action, resource=None, description=None, risk_score=0, *,
                status="SUCCESS", employee_id=None, record_count=0, context=None):
    """
    Record `action` performed by `user` (None for unknown login IDs) and run it
    through the risk pipeline. `risk_score` is an optional base score added on
    top of whatever the rules decide. Returns the stored ActivityLog.
    """
    emp_id = user.employee_id if user is not None else (employee_id or "UNKNOWN")
    activity = ActivityLog(
        user_id=user.id if user is not None else None,
        employee_id=emp_id[:20],
        actor_role=user.role if user is not None else None,
        action=action,
        resource=(resource or "")[:255] or None,
        description=(description or "")[:500] or None,
        record_count=record_count or 0,
        timestamp=now(),
        ip_address=client_ip(),
        user_agent=client_user_agent(),
        status=status,
    )
    alerts = []
    try:
        db.session.add(activity)
        db.session.flush() # gives the activity an id so rules/alerts can reference it

        triggers = risk_engine.evaluate(activity, user, context)
        if risk_score:
            triggers.insert(0, risk_engine.Trigger("BASE_SCORE", int(risk_score),
                                                  f"Base score for {action}", "NONE"))
        cap = current_app.config["RISK_SCORE_CAP"]
        activity.risk_score = min(cap, sum(t.points for t in triggers))
        activity.risk_reason = "; ".join(f"{t.points} {t.reason}" for t in triggers)[:1000] or None

        level_before = level_after = score = None
        if user is not None and activity.risk_score > 0:
            level_before, level_after, score = risk_engine.apply_score(user, activity, triggers)

        alerts = alert_service.generate_alerts(user, activity, triggers, level_before, level_after,
score)
        db.session.commit()
    except SQLAlchemyError:
        db.session.rollback()
```

```

log.exception("Risk pipeline failed for %s %s – storing the bare activity", emp_id, action)
alerts = []
activity = _store_bare(user, emp_id, action, resource, description, status, record_count)
if activity is None:
    return None

notification_service.emit_activity(activity)
for alert in alerts:
    notification_service.emit_alert(alert)
return activity

```

4.3.2 Risk engine — example rules and scoring

Listing 4.2 — services/risk_engine.py (mass-access rule and score update)

```

def _mass_record_access(a, user, ctx, rule):
    if a.action not in RECORD_ACCESS_ACTIONS:
        return None
    since = a.timestamp - timedelta(minutes=rule.window_minutes)
    total = db.session.query(db.func.coalesce(db.func.sum(ActivityLog.record_count), 0)).filter(
        ActivityLog.employee_id == a.employee_id, ActivityLog.timestamp >= since,
        ActivityLog.action.in_(RECORD_ACCESS_ACTIONS), ActivityLog.status == "SUCCESS").scalar()
    if total < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"Accessed {int(total)} records within {rule.window_minutes} minutes",
        rule.severity)

def apply_score(user, activity, triggers):
    """Add trigger points to the user's score and record why. Returns (level_before, level_after,
    score)."""
    cap = current_app.config["RISK_SCORE_CAP"]
    rs = get_or_create_score(user)
    level_before = rs.risk_level
    score = rs.current_score
    for t in triggers:
        if t.points <= 0:
            continue
        after = min(cap, score + t.points)
        db.session.add(RiskEvent(user_id=user.id, activity_id=activity.id, rule_key=t.rule_key,
            points=t.points, reason=t.reason, score_before=score,
            score_after=after,
            created_at=activity.timestamp))
        score = after
    if score != rs.current_score:
        rs.current_score = score
        rs.risk_level = risk_level_for(score)
        rs.last_updated = activity.timestamp
        rs.last_increase_at = activity.timestamp
    return level_before, rs.risk_level, score

```

4.3.3 Role-based access control

Authorisation is enforced on the server. When an employee requests a manager URL, the decorator logs an UNAUTHORIZED_ACCESS activity (which the risk engine scores and alerts on) and returns 403.

Listing 4.3 — utils/security.py

```

def manager_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not current_user.is_authenticated:
            return login_manager.unauthorized()
        if not current_user.is_manager:

```

```

        # an employee probing manager URLs is itself a monitored event
        from services.activity_logger import log_activity
        log_activity(current_user, "UNAUTHORIZED_ACCESS", resource=request.path,
                     description=f"Blocked {request.method} to manager-only resource",
                     status="DENIED")
        return forbidden()
    return view(*args, **kwargs)
return wrapped

```

4.3.4 Login with logout

Listing 4.4 — routes/auth.py (login)

```

@auth_bp.route("/login", methods=["POST"])
def login():
    employee_id = normalize_employee_id(request.form.get("employee_id"))
    password = request.form.get("password") or ""
    if not employee_id or not password:
        return render_template("login.html", error="Enter your employee ID and password.",
                               employee_id=employee_id), 400

    ip = client_ip()
    user = User.query.filter_by(employee_id=employee_id).first()

    if _is_locked_out(employee_id, ip):
        minutes = current_app.config["LOGIN_LOCKOUT_MINUTES"]
        return _fail(employee_id, user, "LOCKED_OUT", "Login blocked: too many failed attempts",
                     f"Too many failed attempts. Try again in {minutes} minutes.", 429)

    if user is None:
        return _fail(employee_id, None, "UNKNOWN_EMPLOYEE_ID", "Login with unknown employee ID",
                     GENERIC_LOGIN_ERROR)
    if not user.check_password(password):
        return _fail(employee_id, user, "INVALID_PASSWORD", "Incorrect password", GENERIC_LOGIN_ERROR)
    if user.status != "active":
        return _fail(employee_id, user, "ACCOUNT_DISABLED", "Login attempt on a disabled account",
                     "This account is disabled. Please contact your manager.", 403)

    # ---- success
    session.clear() # prevent session fixation
    login_user(user)
    session.permanent = True
    user.last_login = now()
    user.last_login_ip = ip
    user.last_seen = now()
    user.is_online = True
    _record_attempt(employee_id, user, True, "SUCCESS")
    action = "MANAGER_LOGIN" if user.is_manager else "LOGIN"
    log_activity(user, action, resource="/login", description="Signed in")
    emit_presence(user)

    if user.must_change_password:
        flash("Please set a new password before continuing.", "info")
        return redirect(url_for("auth.change_password"))
    target = request.args.get("next")
    return redirect(target if is_safe_next(target) else home_url_for(user))

```

4.3.5 Record access policy

Listing 4.5 — services/record_access.py

```

def viewable_filter(user):
    return and_(
        Record.sensitivity != "RESTRICTED",
        or_(Record.sensitivity == "PUBLIC", Record.department.in_([user.department,
                                                                    SHARED_DEPARTMENT])),
    )

```

```

def can_view(user, record):
    if record.sensitivity == "RESTRICTED":
        return False
    return record.sensitivity == "PUBLIC" or record.department in (user.department, SHARED_DEPARTMENT)

def can_edit(user, record):
    return record.department == user.department and record.sensitivity != "RESTRICTED"

def denial_reason(user, record):
    if record.sensitivity == "RESTRICTED":
        return f"{record.record_code} is a RESTRICTED record"
    return f"{record.record_code} belongs to the {record.department} department"

```

4.3.6 Real-time notifications (server and browser)

Listing 4.6 — services/notification_service.py

```

def emit_activity(activity):
    _emit("activity:new", activity.to_feed_dict(), MANAGERS_ROOM)

def emit_alert(alert):
    _emit("alert:new", alert.to_dict(), MANAGERS_ROOM)

def emit_alert_update(alert):
    _emit("alert:updated", alert.to_dict(), MANAGERS_ROOM)

def emit_presence(user):
    _emit("presence", {
        "user_id": user.id,
        "employee_id": user.employee_id,
        "name": user.name,
        "online": user.online,
        "last_seen": iso(user.last_seen),
        "last_login": iso(user.last_login),
    }, MANAGERS_ROOM)

@socketio.on("connect")
def on_connect(auth=None):
    if not current_user.is_authenticated:
        return False # reject anonymous sockets
    join_room(f"user:{current_user.id}")
    if current_user.is_manager:
        join_room(MANAGERS_ROOM)
    try:
        touch_presence(current_user)
    except Exception:
        db.session.rollback()
        log.exception("Presence update failed on connect")
    return True

```

Listing 4.7 — static/js/realtime.js (manager browser)

```

socket.on("alert:new", (a) => {
    newAlerts.unshift(a);
    renderCount((Number(countEl?.textContent) || 0) + 1);
    renderList();
    queueToast(a);

```

```

        emit("itm:alert", a);
    });
    socket.on("alert:updated", (a) => { loadNewAlerts(); emit("itm:alert-changed", a); });
    socket.on("activity:new", (a) => emit("itm:activity", a));
    socket.on("presence", (p) => emit("itm:presence", p));

```

4.4 Database creation

The database is created by `database/schema.sql` (complete listing in Appendix A) and filled with demo data by `database/seed.sql` and `seed_extra.py`. It can be run from the MySQL command line or with the helper script, which reads the credentials from `.env`:

Database creation commands and output

```

> mysql -u root -p < database/schema.sql
> mysql -u root -p insider_threat_system < database/seed.sql
-- or --
> python init_db.py
Creating schema in `insider_threat_system` ...
Loading demo data ...
Added 10 demo employees (EMP1007-EMP1016).
users          17 rows      records          40 rows
activity_logs  ~130 rows  alerts           ~20 rows
risk_scores    17 rows    risk_events      ~25 rows
login_attempts ~35 rows   risk_rules       13 rows

```

Listing 4.8 — DDL of activity_logs and alerts (from schema.sql)

```

CREATE TABLE activity_logs (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT          NULL,
  employee_id VARCHAR(20)  NOT NULL,
  actor_role  VARCHAR(20)  NULL,
  action      VARCHAR(40)  NOT NULL,
  resource    VARCHAR(255) NULL,
  description  VARCHAR(500) NULL,
  record_count INT        NOT NULL DEFAULT 0,
  timestamp   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  ip_address  VARCHAR(45)  NULL,
  user_agent  VARCHAR(255) NULL,
  status      VARCHAR(10)  NOT NULL DEFAULT 'SUCCESS',
  risk_score  INT          NOT NULL DEFAULT 0,
  risk_reason VARCHAR(1000) NULL,
  INDEX ix_activity_user_time (user_id, timestamp),
  INDEX ix_activity_employee_time (employee_id, timestamp),
  INDEX ix_activity_action (action),
  INDEX ix_activity_timestamp (timestamp),
  CONSTRAINT fk_activity_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-- -----
-- Alerts: never deleted, kept as an audit trail
-- -----

CREATE TABLE alerts (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT          NULL,
  employee_id VARCHAR(20)  NOT NULL,
  activity_id  BIGINT      NULL,
  alert_type  VARCHAR(40)  NOT NULL,
  severity    ENUM('LOW', 'MEDIUM', 'HIGH', 'CRITICAL') NOT NULL,
  message     VARCHAR(500) NOT NULL,
  risk_score  INT          NOT NULL DEFAULT 0,
  timestamp   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  status      ENUM('NEW', 'ACKNOWLEDGED', 'RESOLVED') NOT NULL DEFAULT 'NEW',
  acknowledged_by INT      NULL,
  acknowledged_at DATETIME NULL,
  resolved_by  INT          NULL,

```

```

        resolved_at      DATETIME      NULL,
        resolution_note  VARCHAR(500) NULL,
        INDEX ix_alerts_user (user_id),
        INDEX ix_alerts_employee (employee_id),
        INDEX ix_alerts_status (status),
        INDEX ix_alerts_timestamp (timestamp),
        CONSTRAINT fk_alerts_user      FOREIGN KEY (user_id)      REFERENCES users(id)      ON DELETE
SET NULL,
        CONSTRAINT fk_alerts_activity FOREIGN KEY (activity_id)  REFERENCES activity_logs(id) ON DELETE
SET NULL,
        CONSTRAINT fk_alerts_ack_by    FOREIGN KEY (acknowledged_by) REFERENCES users(id)      ON DELETE
SET NULL,
        CONSTRAINT fk_alerts_res_by    FOREIGN KEY (resolved_by)  REFERENCES users(id)      ON DELETE
SET NULL
    ) ENGINE=InnoDB;

-----
-- Current risk score per user
-----
CREATE TABLE risk_scores (
    id          INT AUTO_INCREMENT PRIMARY KEY,
    user_id     INT      NOT NULL UNIQUE,
    current_score INT      NOT NULL DEFAULT 0,
    risk_level  ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'LOW',
    last_updated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_increase_at DATETIME NULL,
    CONSTRAINT fk_risk_scores_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-----
-- Risk events: WHY a score changed (explainability)
-----
CREATE TABLE risk_events (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id     INT      NOT NULL,
    activity_id  BIGINT      NULL,
    rule_key    VARCHAR(40) NOT NULL,
    points      INT      NOT NULL,
    reason      VARCHAR(500) NOT NULL,
    score_before INT      NOT NULL,
    score_after  INT      NOT NULL,
    created_at  DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX ix_risk_events_user_time (user_id, created_at),
    INDEX ix_risk_events_created (created_at),
    CONSTRAINT fk_risk_events_user      FOREIGN KEY (user_id)      REFERENCES users(id)      ON
DELETE CASCADE,
    CONSTRAINT fk_risk_events_activity FOREIGN KEY (activity_id) REFERENCES activity_logs(id) ON
DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Login attempts (successful and failed)
-----
CREATE TABLE login_attempts (
    id          BIGINT AUTO_INCREMENT PRIMARY KEY,
    employee_id VARCHAR(20) NOT NULL,
    user_id     INT      NULL,
    ip_address  VARCHAR(45) NULL,
    user_agent  VARCHAR(255) NULL,
    timestamp   DATETIME  NOT NULL DEFAULT CURRENT_TIMESTAMP,
    success     TINYINT(1) NOT NULL,
    reason      VARCHAR(60) NULL,
    INDEX ix_login_attempts_employee_time (employee_id, timestamp),
    INDEX ix_login_attempts_ip_time (ip_address, timestamp),
    INDEX ix_login_attempts_timestamp (timestamp),
    CONSTRAINT fk_login_attempts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Configurable detection rules (editable in Manager > Settings)
-----
CREATE TABLE risk_rules (

```

```

    id                INT AUTO_INCREMENT PRIMARY KEY,
    rule_key          VARCHAR(40) NOT NULL UNIQUE,
    name              VARCHAR(100) NOT NULL,
    description        VARCHAR(255) NULL,
    points            INT NOT NULL DEFAULT 0,
    threshold          INT NOT NULL DEFAULT 1,
    window_minutes     INT NOT NULL DEFAULT 0,
    severity           ENUM('NONE','LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'NONE',
    enabled            TINYINT(1) NOT NULL DEFAULT 1,
    updated_at         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-- =====
-- Reference data required by the application
-- =====
INSERT INTO departments (name) VALUES
    ('Finance'), ('Human Resources'), ('Engineering'),
    ('IT & Security'), ('Sales'), ('Legal');

INSERT INTO risk_rules (rule_key, name, description, points, threshold, window_minutes, severity)
VALUES
    ('FAILED_LOGIN', 'Failed login', 'Each failed login attempt.',
    10, 1, 0, 'NONE'),
    ('REPEATED_FAILED_LOGIN', 'Repeated failed logins', 'Threshold failed logins for one
    employee ID within the window.', 20, 3, 10, 'HIGH'),
    ('NEW_LOCATION', 'Login from new location', 'Successful login from an IP address
    this employee has never used before.', 10, 1, 0, 'MEDIUM'),
    ('OFF_HOURS', 'Outside working hours', 'Activity outside configured working
    hours/days (counted once per window).', 15, 1, 60, 'MEDIUM'),
    ('SENSITIVE_ACCESS', 'Sensitive record access', 'Viewing or editing a CONFIDENTIAL
    record.', 15, 1, 0, 'MEDIUM'),
    ('UNAUTHORIZED_ACCESS', 'Unauthorized access attempt', 'Attempt to open a record or page
    outside the employee's permissions.', 30, 1, 0, 'HIGH'),
    ('PRIVILEGE_ESCALATION', 'Privilege escalation attempt', 'Employee attempted to use a manager-
    only page or API.', 10, 1, 0, 'HIGH'),
    ('REPEATED_UNAUTHORIZED', 'Repeated unauthorized attempts', 'Threshold unauthorized attempts within
    the window.', 25, 3, 15, 'CRITICAL'),
    ('MASS_RECORD_ACCESS', 'Large number of records accessed', 'Threshold or more records
    viewed/exported within the window.', 20, 15, 5, 'HIGH'),
    ('REPEATED_SEARCH', 'Repeated searches', 'Threshold searches within the window.',
    10, 10, 5, 'LOW'),
    ('RAPID_ACTIONS', 'Multiple rapid actions', 'Threshold actions of any kind within
    the window.', 15, 20, 1, 'MEDIUM'),
    ('UNUSUAL_FREQUENCY', 'Unusual access frequency', 'Actions in the window exceed threshold
    x the employee's normal hourly rate (7-day baseline).', 15, 3, 60, 'MEDIUM'),
    ('SUSPICIOUS_BURST', 'Multiple suspicious events', 'Threshold risk-scored events (excluding
    failed logins) within the window.', 25, 3, 10, 'HIGH');

CREATE TABLE alerts (
    id                BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id           INT NULL,
    employee_id       VARCHAR(20) NOT NULL,
    activity_id        BIGINT NULL,
    alert_type        VARCHAR(40) NOT NULL,
    severity           ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    message           VARCHAR(500) NOT NULL,
    risk_score         INT NOT NULL DEFAULT 0,
    timestamp         DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status            ENUM('NEW','ACKNOWLEDGED','RESOLVED') NOT NULL DEFAULT 'NEW',
    acknowledged_by   INT NULL,
    acknowledged_at   DATETIME NULL,
    resolved_by       INT NULL,
    resolved_at       DATETIME NULL,
    resolution_note    VARCHAR(500) NULL,
    INDEX ix_alerts_user (user_id),
    INDEX ix_alerts_employee (employee_id),
    INDEX ix_alerts_status (status),
    INDEX ix_alerts_timestamp (timestamp),
    CONSTRAINT fk_alerts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE
SET NULL,
    CONSTRAINT fk_alerts_activity FOREIGN KEY (activity_id) REFERENCES activity_logs(id) ON DELETE
SET NULL,
    CONSTRAINT fk_alerts_ack_by FOREIGN KEY (acknowledged_by) REFERENCES users(id) ON DELETE
SET NULL,

```

```

        CONSTRAINT fk_alerts_res_by FOREIGN KEY (resolved_by) REFERENCES users(id) ON DELETE
SET NULL
) ENGINE=InnoDB;

-----
-- Current risk score per user
-----
CREATE TABLE risk_scores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    current_score INT NOT NULL DEFAULT 0,
    risk_level ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'LOW',
    last_updated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_increase_at DATETIME NULL,
    CONSTRAINT fk_risk_scores_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-----
-- Risk events: WHY a score changed (explainability)
-----
CREATE TABLE risk_events (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL,
    activity_id BIGINT NULL,
    rule_key VARCHAR(40) NOT NULL,
    points INT NOT NULL,
    reason VARCHAR(500) NOT NULL,
    score_before INT NOT NULL,
    score_after INT NOT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    INDEX ix_risk_events_user_time (user_id, created_at),
    INDEX ix_risk_events_created (created_at),
    CONSTRAINT fk_risk_events_user FOREIGN KEY (user_id) REFERENCES users(id) ON
DELETE CASCADE,
    CONSTRAINT fk_risk_events_activity FOREIGN KEY (activity_id) REFERENCES activity_logs(id) ON
DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Login attempts (successful and failed)
-----
CREATE TABLE login_attempts (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    employee_id VARCHAR(20) NOT NULL,
    user_id INT NULL,
    ip_address VARCHAR(45) NULL,
    user_agent VARCHAR(255) NULL,
    timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    success TINYINT(1) NOT NULL,
    reason VARCHAR(60) NULL,
    INDEX ix_login_attempts_employee_time (employee_id, timestamp),
    INDEX ix_login_attempts_ip_time (ip_address, timestamp),
    INDEX ix_login_attempts_timestamp (timestamp),
    CONSTRAINT fk_login_attempts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Configurable detection rules (editable in Manager > Settings)
-----
CREATE TABLE risk_rules (
    id INT AUTO_INCREMENT PRIMARY KEY,
    rule_key VARCHAR(40) NOT NULL UNIQUE,
    name VARCHAR(100) NOT NULL,
    description VARCHAR(255) NULL,
    points INT NOT NULL DEFAULT 0,
    threshold INT NOT NULL DEFAULT 1,
    window_minutes INT NOT NULL DEFAULT 0,
    severity ENUM('NONE','LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'NONE',
    enabled TINYINT(1) NOT NULL DEFAULT 1,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

```

```
-- =====
-- Reference data required by the application
-- =====
INSERT INTO departments (name) VALUES
  ('Finance'), ('Human Resources'), ('Engineering'),
  ('IT & Security'), ('Sales'), ('Legal');

INSERT INTO risk_rules (rule_key, name, description, points, threshold, window_minutes, severity)
VALUES
  ('FAILED_LOGIN', 'Failed login', 'Each failed login attempt.',
10, 1, 0, 'NONE'),
  ('REPEATED_FAILED_LOGIN', 'Repeated failed logins', 'Threshold failed logins for one
employee ID within the window.', 20, 3, 10, 'HIGH'),
  ('NEW_LOCATION', 'Login from new location', 'Successful login from an IP address
this employee has never used before.', 10, 1, 0, 'MEDIUM'),
  ('OFF_HOURS', 'Outside working hours', 'Activity outside configured working
hours/days (counted once per window).', 15, 1, 60, 'MEDIUM'),
  ('SENSITIVE_ACCESS', 'Sensitive record access', 'Viewing or editing a CONFIDENTIAL
record.', 15, 1, 0, 'MEDIUM'),
  ('UNAUTHORIZED_ACCESS', 'Unauthorized access attempt', 'Attempt to open a record or page
outside the employee's permissions.', 30, 1, 0, 'HIGH'),
  ('PRIVILEGE_ESCALATION', 'Privilege escalation attempt', 'Employee attempted to use a manager-
only page or API.', 10, 1, 0, 'HIGH'),
  ('REPEATED_UNAUTHORIZED', 'Repeated unauthorized attempts', 'Threshold unauthorized attempts within
the window.', 25, 3, 15, 'CRITICAL'),
  ('MASS_RECORD_ACCESS', 'Large number of records accessed', 'Threshold or more records
viewed/exported within the window.', 20, 15, 5, 'HIGH'),
  ('REPEATED_SEARCH', 'Repeated searches', 'Threshold searches within the window.',
10, 10, 5, 'LOW'),
  ('RAPID_ACTIONS', 'Multiple rapid actions', 'Threshold actions of any kind within
the window.', 15, 20, 1, 'MEDIUM'),
  ('UNUSUAL_FREQUENCY', 'Unusual access frequency', 'Actions in the window exceed threshold
x the employee's normal hourly rate (7-day baseline).', 15, 3, 60, 'MEDIUM'),
  ('SUSPICIOUS_BURST', 'Multiple suspicious events', 'Threshold risk-scored events (excluding
failed logins) within the window.', 25, 3, 10, 'HIGH');
```

Figure 4.1 shows the tables created in MySQL and Figure 4.2 the foreign-key relationships read back from the data dictionary.

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> SHOW TABLES;
+-----+
| Tables_in_itm_report |
+-----+
| activity_logs         |
| alerts               |
| departments          |
| login_attempts       |
| records              |
| risk_events          |
| risk_rules           |
| risk_scores          |
| users                |
+-----+
9 rows in set
```

Figure 4.1: *SHOW TABLES* — the nine tables of the database

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> SELECT TABLE_NAME, COLUMN_NAME, REFERENCED_TABLE_NAME AS ref_table, REFERENCED_COLUMN_NAME AS ref_column
-> FROM information_schema.KEY_COLUMN_USAGE
-> WHERE TABLE_SCHEMA = DATABASE() AND REFERENCED_TABLE_NAME IS NOT NULL
-> ORDER BY TABLE_NAME, COLUMN_NAME;
```

TABLE_NAME	COLUMN_NAME	ref_table	ref_column
activity_logs	user_id	users	id
alerts	acknowledged_by	users	id
alerts	activity_id	activity_logs	id
alerts	resolved_by	users	id
alerts	user_id	users	id
login_attempts	user_id	users	id
records	created_by	users	id
records	updated_by	users	id
risk_events	activity_id	activity_logs	id
risk_events	user_id	users	id
risk_scores	user_id	users	id
users	created_by	users	id
users	department	departments	name

13 rows in set

Figure 4.2: Foreign keys verified from information_schema

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> DESCRIBE users;
```

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
employee_id	varchar(20)	NO	UNI	NULL	
name	varchar(100)	NO		NULL	
email	varchar(120)	NO	UNI	NULL	
password_hash	varchar(255)	NO		NULL	
department	varchar(60)	NO	MUL	NULL	
job_title	varchar(80)	YES		NULL	
role	enum('employee','manager')	NO	MUL	employee	
status	enum('active','disabled')	NO		active	
must_change_password	tinyint(1)	NO		1	
is_online	tinyint(1)	NO		0	
last_login	datetime	YES		NULL	
last_login_ip	varchar(45)	YES		NULL	
last_seen	datetime	YES		NULL	
created_by	int	YES	MUL	NULL	
created_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED
updated_at	datetime	NO		CURRENT_TIMESTAMP	DEFAULT_GENERATED on update CURRENT_TIMESTAMP

17 rows in set

Figure 4.3: DESCRIBE users

4.5 Sample queries executed from the front end

The application never builds SQL by string concatenation; SQLAlchemy generates parameterised statements. To show how front-end actions become database operations, the SQL engine log was captured while performing two actions in the web interface.

(a) Manager creates an employee (Employees → New employee → Create). The backend inserts the user, creates the risk-score row and writes the audit activity in one transaction (Figure 4.5).

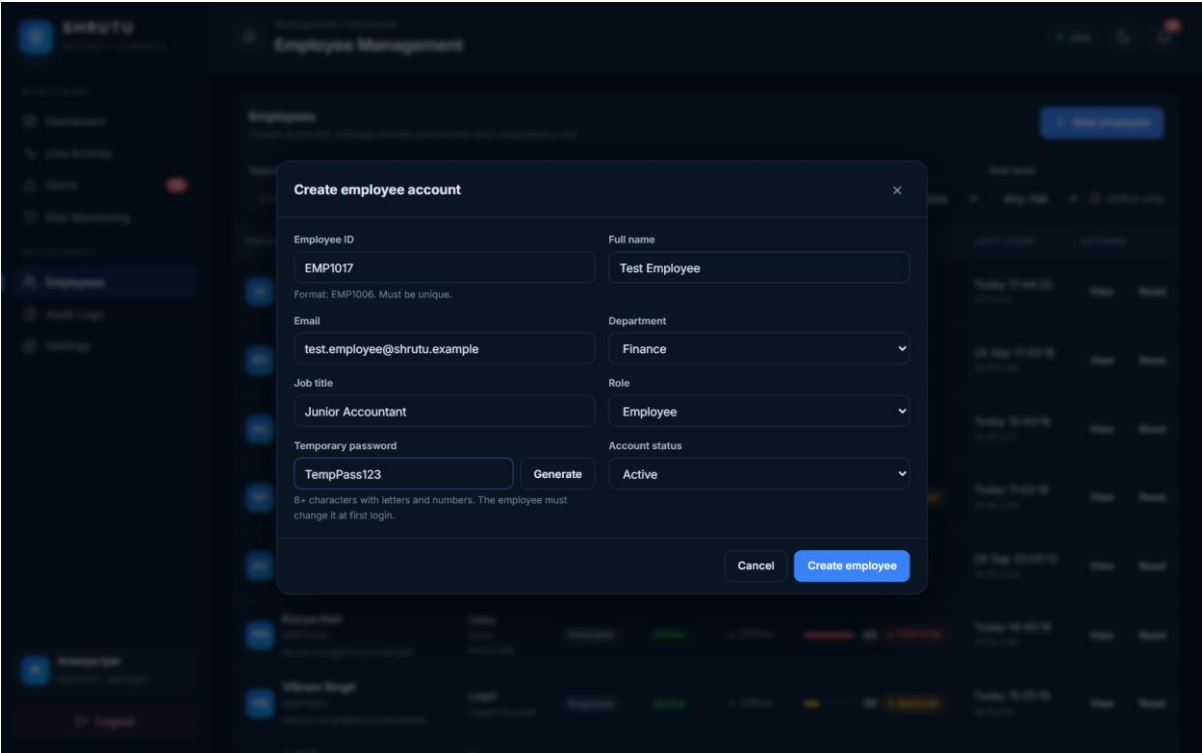


Figure 4.4: Front end: manager's "Create employee account" form

SQL issued by the Flask backend (SQLAlchemy engine log) — Front-end action: Manager clicks “Create employee” (POST /api/manager/employees)

```
BEGIN;
INSERT INTO users (employee_id, name, email, password_hash, department, job_title, `role`, status,
    must_change_password, is_online, last_login, last_login_ip, last_seen, created_by, created_at, updated_at) VALUES
    (:employee_id, :name, :email, :password_hash, :department, :job_title, :role, :status, :must_change_password,
    :is_online, :last_login, :last_login_ip, :last_seen, :created_by, :created_at, :updated_at);
-- values: {'employee_id': 'EMP1018', 'name': 'Rahul Verma', 'email': 'rahul.verma@shrutu.example', 'password_hash':
'bcrypt:32768:8:1$3pW5msrBB0IkhLLp$1959b00491fc8d0e19b54eaf599864f8b8817908efc71cbbfbdf319
INSERT INTO risk_scores (user_id, current_score, risk_level, last_updated, last_increase_at) VALUES (:user_id,
    :current_score, :risk_level, :last_updated, :last_increase_at);
-- values: {'user_id': 19, 'current_score': 0, 'risk_level': 'LOW', 'last_updated': datetime.datetime(2026, 9, 29, 17,
47, 8), 'last_increase_at': None}
INSERT INTO activity_logs (user_id, employee_id, actor_role, action, resource, description, record_count, timestamp,
    ip_address, user_agent, status, risk_score, risk_reason) VALUES (:user_id, :employee_id, :actor_role, :action,
    :resource, :description, :record_count, :timestamp, :ip_address, :user_agent, :status, :risk_score, :risk_reason);
-- values: {'user_id': 1, 'employee_id': 'MGR1001', 'actor_role': 'manager', 'action': 'CREATE_EMPLOYEE', 'resource':
'employee:EMP1018', 'description': 'Created employee account EMP1018 (Rahul Verma, Fin
UPDATE activity_logs SET risk_reason=:risk_reason WHERE activity_logs.id = :activity_logs_id;
-- values: {'risk_reason': None, 'activity_logs_id': 163}
COMMIT;
```

Figure 4.5: SQL generated for the create-employee action

(b) **Employee opens a CONFIDENTIAL record.** One click produces six writes inside one transaction: the activity row, its risk explanation, the score update from 55 to 70 (MEDIUM → HIGH), and two alerts (Figure 4.7).

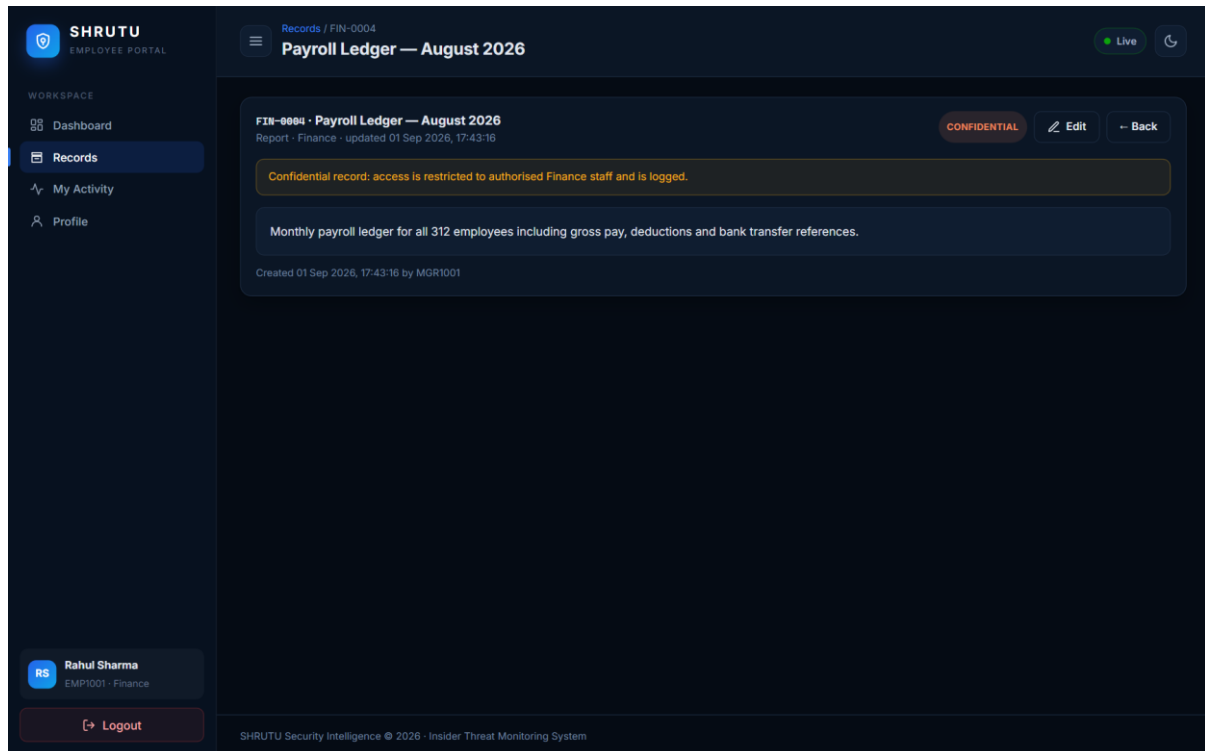


Figure 4.6: Front end: employee viewing a *CONFIDENTIAL* record

SQL issued by the Flask backend (SQLAlchemy engine log) — Front-end action: Employee EMP1002 opens CONFIDENTIAL record HR-0003 (GET /employee/records/<id>)

```
BEGIN;
INSERT INTO activity_logs (user_id, employee_id, actor_role, action, resource, description, record_count, timestamp,
    ip_address, user_agent, status, risk_score, risk_reason) VALUES (:user_id, :employee_id, :actor_role, :action,
    :resource, :description, :record_count, :timestamp, :ip_address, :user_agent, :status, :risk_score, :risk_reason);
-- values: {'user_id': 4, 'employee_id': 'EMP1002', 'actor_role': 'employee', 'action': 'VIEW_RECORD', 'resource':
    'record:HR-0003', 'description': "Viewed CONFIDENTIAL record 'Salary Bands by Grade'", 'r
UPDATE activity_logs SET risk_score=:risk_score, risk_reason=:risk_reason WHERE activity_logs.id = :activity_logs_id;
-- values: {'risk_score': 15, 'risk_reason': '+15 Viewed CONFIDENTIAL record record:HR-0003', 'activity_logs_id': 164}
INSERT INTO risk_events (user_id, activity_id, rule_key, points, reason, score_before, score_after, created_at) VALUES
    (:user_id, :activity_id, :rule_key, :points, :reason, :score_before, :score_after, :created_at);
-- values: {'user_id': 4, 'activity_id': 164, 'rule_key': 'SENSITIVE_ACCESS', 'points': 15, 'reason': 'Viewed
    CONFIDENTIAL record record:HR-0003', 'score_before': 55, 'score_after': 70, 'created_at': date
UPDATE risk_scores SET current_score=:current_score, risk_level=:risk_level WHERE risk_scores.id = :risk_scores_id;
-- values: {'current_score': 70, 'risk_level': 'HIGH', 'risk_scores_id': 4}
INSERT INTO alerts (user_id, employee_id, activity_id, alert_type, severity, message, risk_score, timestamp, status,
    acknowledged_by, acknowledged_at, resolved_by, resolved_at, resolution_note) VALUES (:user_id, :employee_id,
    :activity_id, :alert_type, :severity, :message, :risk_score, :timestamp, :status, :acknowledged_by,
    :acknowledged_at, :resolved_by, :resolved_at, :resolution_note);
-- values: {'user_id': 4, 'employee_id': 'EMP1002', 'activity_id': 164, 'alert_type': 'SENSITIVE_ACCESS', 'severity':
    'MEDIUM', 'message': 'Viewed CONFIDENTIAL record record:HR-0003', 'risk_score': 70, 't
INSERT INTO alerts (user_id, employee_id, activity_id, alert_type, severity, message, risk_score, timestamp, status,
    acknowledged_by, acknowledged_at, resolved_by, resolved_at, resolution_note) VALUES (:user_id, :employee_id,
    :activity_id, :alert_type, :severity, :message, :risk_score, :timestamp, :status, :acknowledged_by,
    :acknowledged_at, :resolved_by, :resolved_at, :resolution_note);
-- values: {'user_id': 4, 'employee_id': 'EMP1002', 'activity_id': 164, 'alert_type': 'RISK_LEVEL_ESCALATION',
    'severity': 'HIGH', 'message': 'Risk level rose from MEDIUM to HIGH (score 70). Latest: Viewe
COMMIT;
```

Figure 4.7: *SQL generated when a confidential record is opened*

Other important queries issued by the application include the windowed counts used by the risk engine and the dashboard aggregates, for example:

Listing 4.9 — representative queries used by the application

```
-- MASS_RECORD_ACCESS rule: records accessed by one employee in the last 5 minutes
SELECT COALESCE(SUM(record_count), 0) FROM activity_logs
WHERE employee_id = :emp AND timestamp >= NOW() - INTERVAL 5 MINUTE
AND action IN ('VIEW_RECORD', 'EXPORT_DATA') AND status = 'SUCCESS';

-- Dashboard KPI: employees at HIGH or CRITICAL risk
SELECT COUNT(*) FROM risk_scores r JOIN users u ON u.id = r.user_id
WHERE u.role = 'employee' AND r.risk_level IN ('HIGH', 'CRITICAL');

-- Alert cooldown check before inserting a new alert
SELECT id FROM alerts
WHERE employee_id = :emp AND alert_type = :type AND status <> 'RESOLVED'
AND timestamp >= NOW() - INTERVAL 10 MINUTE LIMIT 1;
```

4.6 Deployment

The application is deployed publicly. The source code is in a private GitHub repository; Render builds it from a `render.yaml` blueprint and runs `gunicorn --worker-class gthread -w 1 --threads 100 wsgi:app` behind HTTPS. The database is a free Aiven for MySQL 8.4 service in Bangalore, reached over TLS with certificate verification. On the first start the application detects the empty database and creates the schema and demo data automatically. Every git push redeploys the site.

Table 4.3: *Deployment summary*

Item	Value
Live URL	https://shrutu-insider-threat.onrender.com
Web hosting	Render free web service, Singapore region, Python 3.14
Database	Aiven for MySQL 8.4 (free plan), DigitalOcean Bangalore, TLS required
Secrets	Environment variables in Render (never committed)

5. Testing

Testing was carried out at three levels. Unit and integration tests are automated with **pytest** (52 tests) and run against an in-memory database so they are fast and repeatable; the Socket.IO tests use Flask-SocketIO's test client to verify that events really reach manager sockets. System testing was performed manually end to end on the real MySQL database in two separate browser sessions (manager and employee). All tests passed (Figure 5.1).

5.1 Unit testing

Table 5.1: Unit test cases

ID	Test case	Input	Expected output	Actual output	Result
U1	Risk level thresholds	Scores 0, 29, 30, 59, 60, 79, 80, 100	LOW, LOW, MEDIUM, MEDIUM, HIGH, HIGH, CRITICAL, CRITICAL	As expected	Pass
U2	Normal activity adds no risk	Employee searches and opens an INTERNAL own-department record	Score 0, level LOW, no alert	Score 0, no alert	Pass
U3	Sensitive access	Employee opens a CONFIDENTIAL record twice	Score 15 → 30 (MEDIUM); two risk_events; one alert (cooldown)	As expected	Pass
U4	Unauthorised access	Open RESTRICTED record, then other department's record	Score 60 (HIGH) + RISK_LEVEL_ESCALATION alert	Score 60, escalation alert HIGH	Pass
U5	Critical & cap	Four unauthorised attempts	CRITICAL, REPEATED_UNAUTHORIZED alert, score never > 100	Score 100, CRITICAL	Pass
U6	Repeated failed logins	3 wrong passwords for known ID and unknown ID EMP4040	HIGH alert for both; known user score 50 (MEDIUM)	As expected	Pass
U7	Mass export	Export CSV of ≥ 15 records	MASS_RECORD_ACCESS alert, reason "Accessed N records..."	As expected	Pass
U8	New location	Login from 203.0.113.9 after using 127.0.0.1	NEW_LOCATION alert	As expected	Pass
U9	Off-hours rule	Two searches on Sunday 23:15	One OFF_HOURS event (once per window)	One event, reason mentions 23:15 Sunday	Pass
U10	Score decay	Score 70, quiet for 5 hours	Score 60	60	Pass
U11	Configurable rules	Set SENSITIVE_ACCESS points 40, severity HIGH; points 500	Rule applied (score 40, HIGH alert); 500 rejected (400)	As expected	Pass
U12	Extra seed consistency	Run seed_extra twice	10 employees added once; every score equals sum of its risk events	As expected	Pass

5.2 Integration testing

Table 5.2: *Integration test cases*

ID	Test case	Input	Expected output	Actual output	Result
I1	Manager login	MGR1001 / valid password	Redirect to /manager/dashboard; MANAGER_LOGIN logged; login_attempts success row	As expected	Pass
I2	Employee login & presence	EMP1001 / valid password	Redirect to employee dashboard; user online; LOGIN logged	As expected	Pass
I3	Invalid password	EMP1001 / wrong password	HTTP 401, generic message, FAILED_LOGIN (+10) and attempt stored	As expected	Pass
I4	Create employee	POST valid employee JSON	201; row in users with script hash, must_change_password = 1, risk_scores row, CREATE_EMPLOYEE audit; new user can log in	As expected	Pass
I5	Duplicate data	Same employee ID; same e-mail	409 with field employee_id / email	409 + field	Pass
I6	Disable employee	PUT status=disabled	Account disabled, DISABLE_EMPLOYEE audited, existing session invalidated	As expected	Pass
I7	Reset password	POST new temporary password	Old password rejected; new one works and forces change	As expected	Pass
I8	Acknowledge / resolve alert	PUT acknowledge, then resolve with note	Status and acknowledged_by/at, resolved_by/at, note stored; audit rows; second acknowledge 409; DELETE 405	As expected	Pass
I9	Activity filters	action, suspicious, severity, alert_status, employee, role, risk_level	Only matching rows returned	As expected	Pass
I10	Dashboard numbers	GET /api/manager/dashboard	KPIs and chart totals equal database counts	Equal	Pass
I11	Real-time delivery	Employee opens RESTRICTED record while manager socket connected	Manager receives activity:new and alert:new with stored IDs	Received	Pass
I12	Socket authorisation	Anonymous socket; employee socket	Anonymous rejected; employee receives no manager events	As expected	Pass

5.3 System testing (end-to-end)

Table 5.3: *System test cases*

ID	Test case	Input	Expected output	Actual output	Result
S1	Manager creates employee from UI	EMP1006/EMP1017, Finance, temporary password	Row in MySQL users, audit entry, appears in list	Created; verified with SELECT	Pass
S2	First login of new employee	New ID + temporary password (second browser)	Forced password change; LOGIN appears in manager's live feed without refresh	Appeared within ~1 s	Pass
S3	Normal work	Search, view, search, view	Four feed rows, zero risk points, no alert	As expected	Pass
S4	Suspicious behaviour	Open FIN-0010 (RESTRICTED), SAL-0002, /manager/employees	Three 403 pages; score to CRITICAL; grouped alert pop-ups on manager screen without refresh	As expected (Figure 6.15)	Pass
S5	Investigation	Click alert → View activity	Full details incl. IP, user agent, risk reason	As expected	Pass
S6	Acknowledge	Click Acknowledge	alerts.status = ACKNOWLEDGED, acknowledged_by = MGR1001, counter decreases in all manager tabs	Verified in MySQL	Pass
S7	Two laptops / public deployment	Manager and employee on different devices via live URL	Same real-time behaviour over HTTPS/WSS	As expected	Pass
S8	Responsive UI	390 px mobile viewport; light theme	No horizontal scroll; readable layout	As expected (Figures 6.29–6.30)	Pass

5.4 Security and validation checks

Table 5.4: *Security and validation test cases*

ID	Test case	Input	Expected output	Actual output	Result
V1	Brute-force logout	5 wrong passwords then correct password	HTTP 429 "Too many attempts"; LOCKED_OUT recorded	As expected	Pass
V2	Disabled account	Correct password of disabled user	403 "account disabled"; ACCOUNT_DISABLED attempt	As expected	Pass
V3	Privilege escalation	Employee requests /manager/dashboard and /api/manager/*	403; UNAUTHORIZED_ACCESS + PRIVILEGE_ESCALATION alerts	As expected	Pass
V4	Direct URL to other department record	Employee opens /employee/records/<Sales id>	403 and alert; not shown in search	As expected	Pass
V5	Input validation	ID "12", e-mail "nope", department "Space", password "short", name "<script>"	400 with the offending field; nothing inserted	As expected	Pass

ID	Test case	Input	Expected output	Actual output	Result
V6	Employee data isolation	GET /api/employee/activity	Only own rows; no risk_score / risk_reason fields	As expected	Pass
V7	Logout method	GET /logout	405 (POST with CSRF token only)	405	Pass
V8	Expired session (API)	API call without session	401 JSON	401	Pass
V9	Security headers	GET /	CSP default-src 'self', X-Frame-Options DENY	Present	Pass
V10	Password storage	Inspect users.password_hash	bcrypt hash, never plain text; never returned by API	As expected	Pass
V11	SQL injection	Search text: ' OR 1=1 --	Treated as literal text (parameter binding)	No extra rows	Pass

5.5 Automated test run

```

Terminal — python -m pytest -v

===== test session starts =====
platform win32 -- Python 3.14.7, pytest-9.1.1, pluggy-1.6.0 -- insider-threat-monitor\.venv\Scripts\python.exe
rootdir: insider-threat-monitor
collecting ... collected 52 items

tests/test_alerts_and_api.py::test_acknowledge_and_resolve_alert PASSED [ 1%]
tests/test_alerts_and_api.py::test_alert_filters PASSED [ 3%]
tests/test_alerts_and_api.py::test_alert_and_activity_detail PASSED [ 5%]
tests/test_alerts_and_api.py::test_activity_filters PASSED [ 7%]
tests/test_alerts_and_api.py::test_dashboard_numbers come from database PASSED [ 9%]
tests/test_alerts_and_api.py::test_audit_and_login_attempt_endpoints PASSED [ 11%]
tests/test_alerts_and_api.py::test_health_endpoint PASSED [ 13%]
tests/test_auth.py::test_manager_login_redirects to manager dashboard PASSED [ 15%]
tests/test_auth.py::test_employee_login_records activity and presence PASSED [ 17%]
tests/test_auth.py::test_invalid_password is logged with generic message PASSED [ 19%]
tests/test_auth.py::test_unknown_employee_id is logged without user PASSED [ 21%]
tests/test_auth.py::test_disabled_account cannot log in PASSED [ 23%]
tests/test_auth.py::test_lockout_after_repeated_failures PASSED [ 25%]
tests/test_auth.py::test_logout is post only and clears session PASSED [ 26%]
tests/test_auth.py::test_expired_session api returns 401 PASSED [ 28%]
tests/test_auth.py::test_forced_password_change for new accounts PASSED [ 30%]
tests/test_auth.py::test_security_headers present PASSED [ 32%]
tests/test_employees.py::test_create_employee PASSED [ 34%]
tests/test_employees.py::test_duplicate_employee_id and email PASSED [ 36%]
tests/test_employees.py::test_invalid_input rejected PASSED [ 38%]
tests/test_employees.py::test_disable_employee invalidates session PASSED [ 40%]
tests/test_employees.py::test_delete_endpoint soft disables PASSED [ 42%]
tests/test_employees.py::test_reset_password PASSED [ 44%]
tests/test_employees.py::test_manager_cannot_disable self PASSED [ 46%]
tests/test_employees.py::test_role_change raises privilege alert PASSED [ 48%]
tests/test_employees.py::test_employee_list search and filters PASSED [ 50%]
tests/test_rbac.py::test_employee blocked from manager page and it is detected PASSED [ 51%]
tests/test_rbac.py::test_employee blocked from manager api PASSED [ 53%]
tests/test_rbac.py::test_manager is redirected away from employee pages PASSED [ 55%]
tests/test_rbac.py::test_anonymous_users are sent to login PASSED [ 57%]
tests/test_rbac.py::test_employee api returns only own activity without risk PASSED [ 59%]
tests/test_rbac.py::test_record_permissions PASSED [ 61%]
tests/test_rbac.py::test_search only returns permitted records PASSED [ 63%]
tests/test_rbac.py::test_employee cannot create restricted records PASSED [ 65%]
tests/test_realtime.py::test_anonymous_socket rejected PASSED [ 67%]
tests/test_realtime.py::test_manager receives live activity and alert PASSED [ 69%]
tests/test_realtime.py::test_alert_update broadcast PASSED [ 71%]
tests/test_realtime.py::test_employee_socket gets no manager events PASSED [ 73%]
tests/test_realtime.py::test_heartbeat keeps presence PASSED [ 75%]
tests/test_realtime.py::test_disabled_account notified PASSED [ 76%]
tests/test_risk_engine.py::test_level thresholds PASSED [ 78%]
tests/test_risk_engine.py::test_normal activity adds no risk PASSED [ 80%]
tests/test_risk_engine.py::test_sensitive access is medium risk and explained PASSED [ 82%]
tests/test_risk_engine.py::test_unauthorized access reaches high with escalation alert PASSED [ 84%]
tests/test_risk_engine.py::test_repeated unauthorized is critical and capped PASSED [ 86%]
tests/test_risk_engine.py::test_repeated failed logins alert for known and unknown ids PASSED [ 88%]
tests/test_risk_engine.py::test_mass_export detected PASSED [ 90%]
tests/test_risk_engine.py::test_new_location login PASSED [ 92%]
tests/test_risk_engine.py::test_off_hours rule PASSED [ 94%]
tests/test_risk_engine.py::test_decay lowers scores after quiet period PASSED [ 96%]
tests/test_risk_engine.py::test_rules are configurable PASSED [ 98%]
tests/test_seed_extra.py::test_extra_employees are consistent and idempotent PASSED [100%]

===== 52 passed in 62.21s (0:01:02) =====

```

Figure 5.1: Output of the automated test suite (52 passed)

6. Results and Snapshots

6.1 Web application screenshots

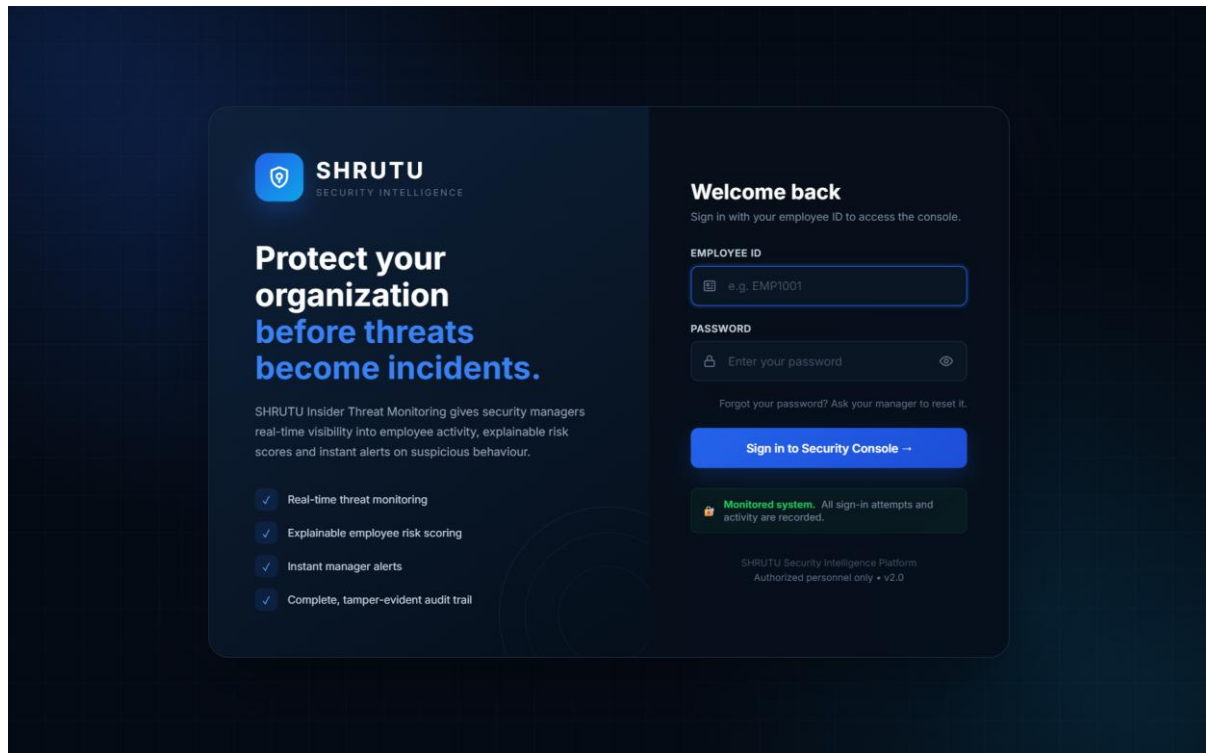


Figure 6.1: Login page

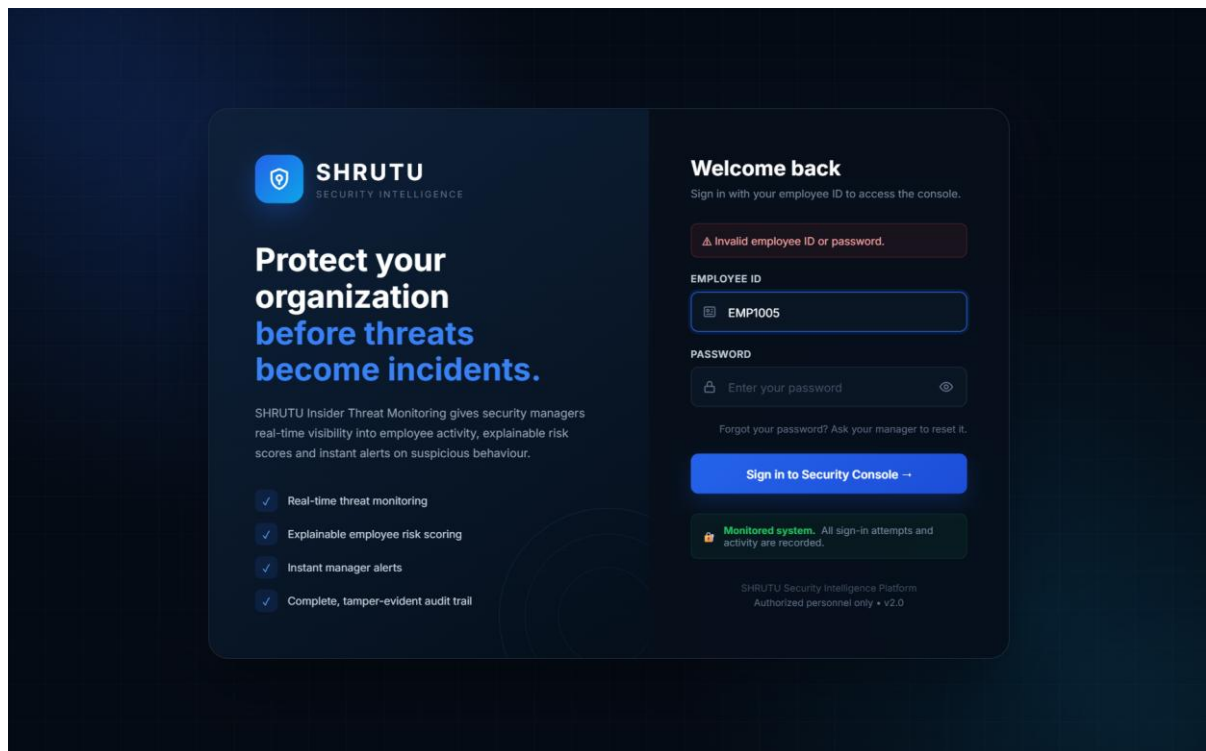


Figure 6.2: Invalid login — generic error message (attempt recorded)

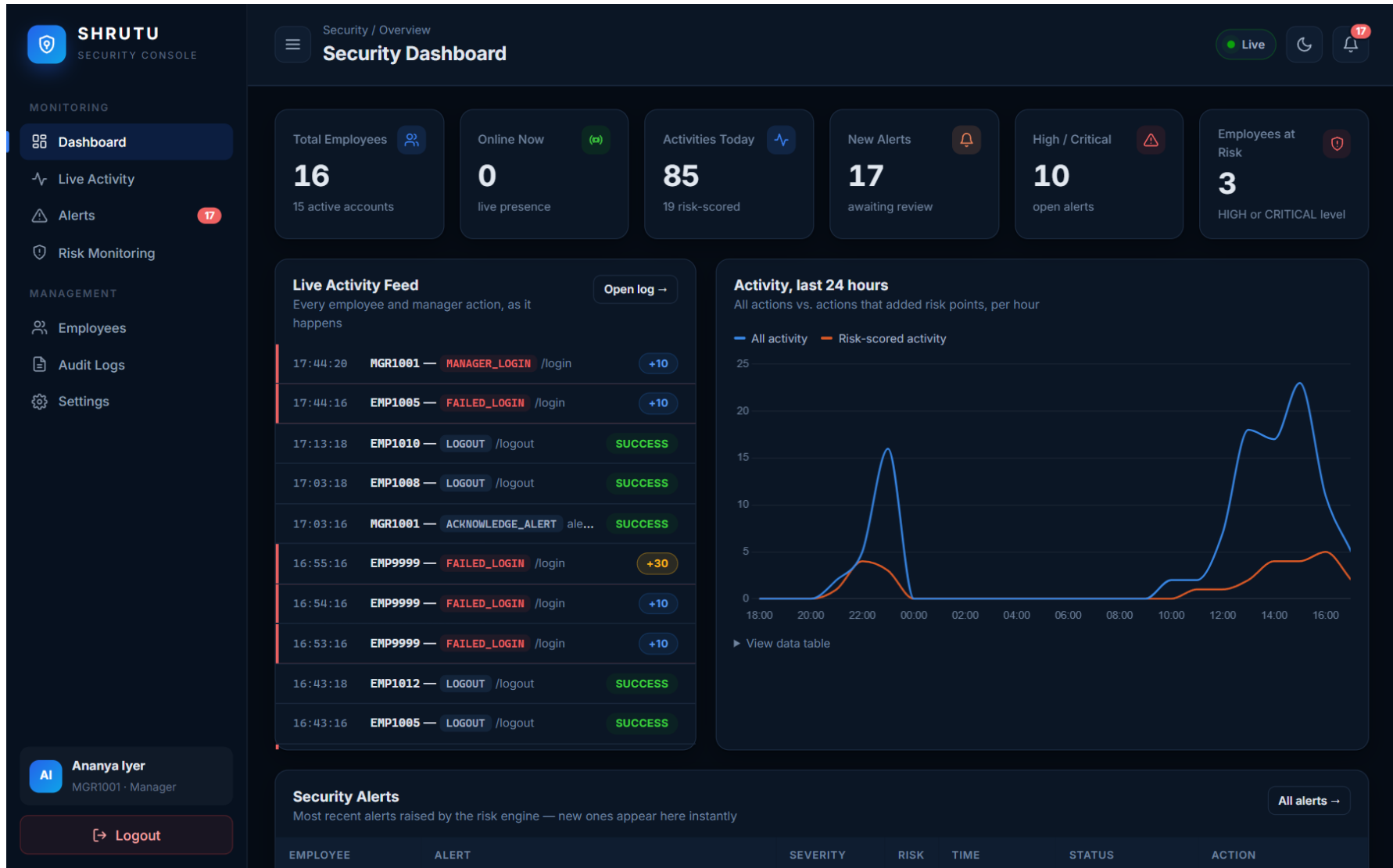


Figure 6.3: Manager Security Dashboard: KPIs, live activity feed and 24-hour activity chart

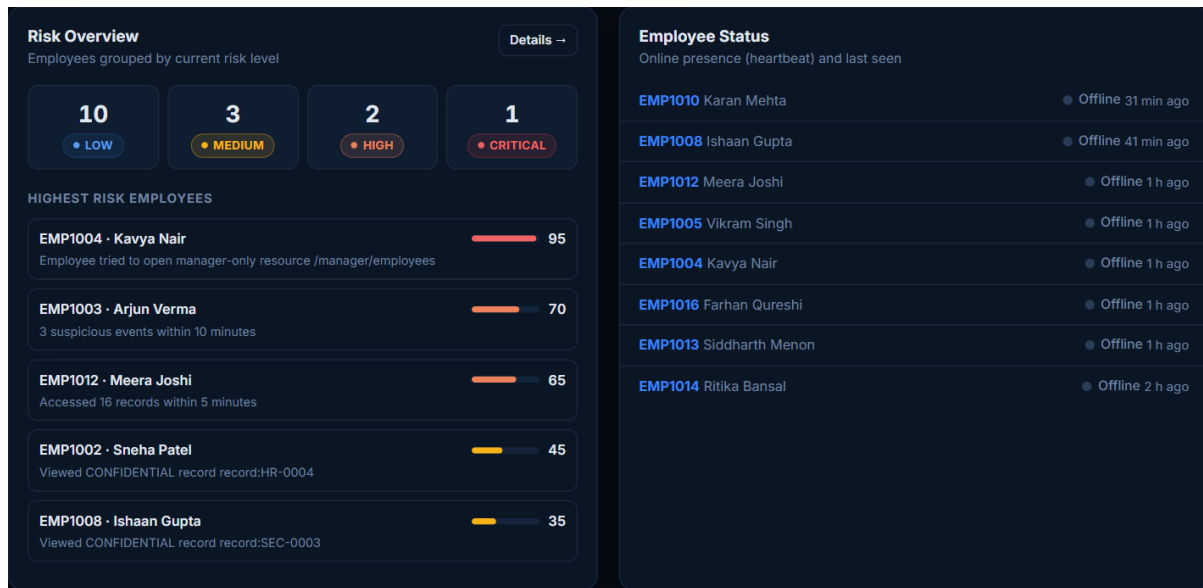


Figure 6.4: Dashboard: risk overview and employee online/offline status

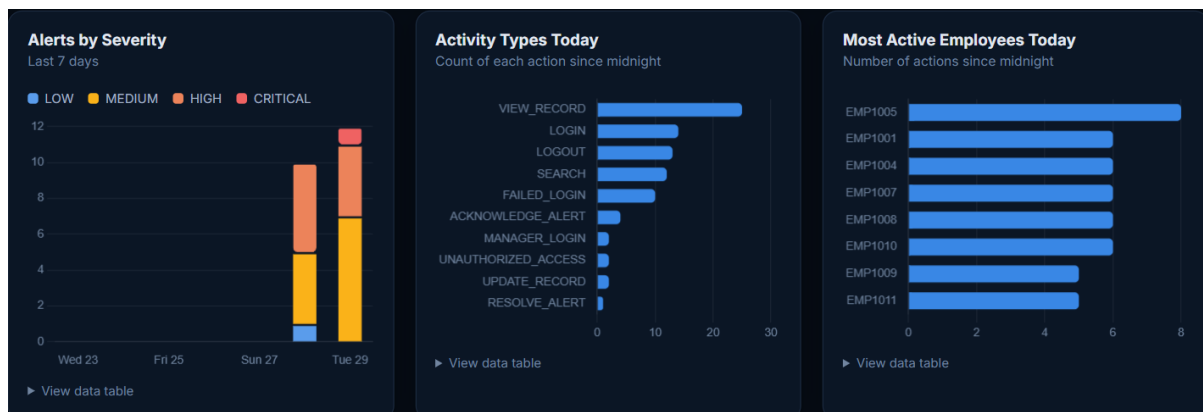


Figure 6.5: Dashboard charts: alerts by severity, activity types, most active employees

The screenshot displays the 'Employee Management' section of the SHRUTU Security Console. The interface includes a sidebar with navigation options: Dashboard, Live Activity, Alerts (17), Risk Monitoring, Employees (selected), Audit Logs, and Settings. The main content area shows a table of employees with columns for Employee ID, Department, Role, Account, Presence, Risk, Last Login, and Actions. A search bar and filters for Department, Account status, Role, Risk level, and Online only are present at the top of the table.

EMPLOYEE	DEPARTMENT	ROLE	ACCOUNT	PRESENCE	RISK	LAST LOGIN	ACTIONS
AI Ananya Iyer MGR1001 - ananya.iyer@shrutu.example	IT & Security	Security Manager	Manager	Active	Online	Today 17:44:20 127.0.0.1	View Reset
RD Rohan Das EMP1000 - rohan.das@shrutu.example	Finance	Accounts Assistant	Employee	Disabled	Offline	24 Sep 17:43:16 10.10.1.40	View Reset
RS Rahul Sharma EMP1001 - rahul.sharma@shrutu.example	Finance	Financial Analyst	Employee	Active	Offline	Today 12:43:16 10.10.1.21	View Reset
SP Sneha Patel EMP1002 - sneha.patel@shrutu.example	Human Resources	HR Executive	Employee	Active	Offline	Today 11:43:16 10.10.2.14	View Reset
AV Arjun Verma EMP1003 - arjun.verma@shrutu.example	Engineering	Software Engineer	Employee	Active	Offline	28 Sep 23:05:12 10.10.3.33	View Reset
KN Kavya Nair EMP1004 - kavya.nair@shrutu.example	Sales	Sales Associate	Employee	Active	Offline	Today 14:40:16 10.10.4.18	View Reset
VS Vikram Singh EMP1005 - vikram.singh@shrutu.example	Legal	Legal Counsel	Employee	Active	Offline	Today 15:25:16 10.10.5.9	View Reset

Figure 6.6: Employee management with search and filters

This screenshot shows the same 'Employee Management' interface as Figure 6.6, but with a confirmation message at the top right: 'Employee EMP1017 created. Temporary password: TempPass123 — share it securely; it is not shown again.' The table of employees remains the same, but the user interface elements like the sidebar and search filters are consistent with the previous figure.

Figure 6.7: Employee created — confirmation

SHRUTU
EMPLOYEE PORTAL

Account / Security
Change password

Please set a new password before continuing.

Set a new password to continue
Your account is using a temporary password set by a manager. Choose a personal password before continuing.

Temporary password

New password

At least 8 characters, with at least one letter and one number.

Confirm new password

Update password

Test Employee
EMP1017 - Finance

Logout

SHRUTU Security Intelligence © 2026 - Insider Threat Monitoring System

Figure 6.8: *New employee must set a new password at first login*

SHRUTU
EMPLOYEE PORTAL

Workspace / Home
Welcome, Rahul

RS **Rahul Sharma**
Employee ID: EMP1001 | Department: Finance | Role: Financial Analyst
Signed in | Session started 29 Sep 2026, 17:44 from 127.0.0.1

Search records **+ Add record**

Records available to you
16
Finance + company-wide

Records you created
0
all time

Your actions today
7
logged automatically

My Recent Activity
Your own actions in this system

TIME	ACTION	DETAILS	RESULT
29 Sep 17:44:35	LOGIN	Signed in	SUCCESS
29 Sep 14:43:16	LOGOUT	Signed out	SUCCESS
29 Sep 13:43:16	UPDATE_RECORD	Updated record "Vendor Payment Schedule — S...	SUCCESS
29 Sep 12:53:16	VIEW_RECORD	Viewed INTERNAL record "Invoice INV-88240 — ...	SUCCESS
29 Sep 12:50:16	VIEW_RECORD	Viewed INTERNAL record "Invoice INV-88213 — ...	SUCCESS
29 Sep 12:48:16	SEARCH	Searched "invoice" — 3 result(s)	SUCCESS
29 Sep 12:43:16	LOGIN	Signed in	SUCCESS
28 Sep 17:42:55	LOGOUT	Signed out	SUCCESS

Open a record by code
e.g. FIN-0003

RECORD CODE **Open**

RECENTLY UPDATED

- FIN-9982 Vendor Payment Schedule — September **INTERNAL**
- FIN-9996 Annual Budget Draft FY2027 **CONFIDENTIAL**
- FIN-9999 Tax Filing Checklist 2026 **INTERNAL**
- FIN-9998 Invoice INV-88240 — Contoso Retail **INTERNAL**
- FIN-9991 Q2 2026 Revenue Summary **INTERNAL**

Rahul Sharma
EMP1001 - Finance

Logout

Figure 6.9: *Employee dashboard (no risk information is shown to employees)*

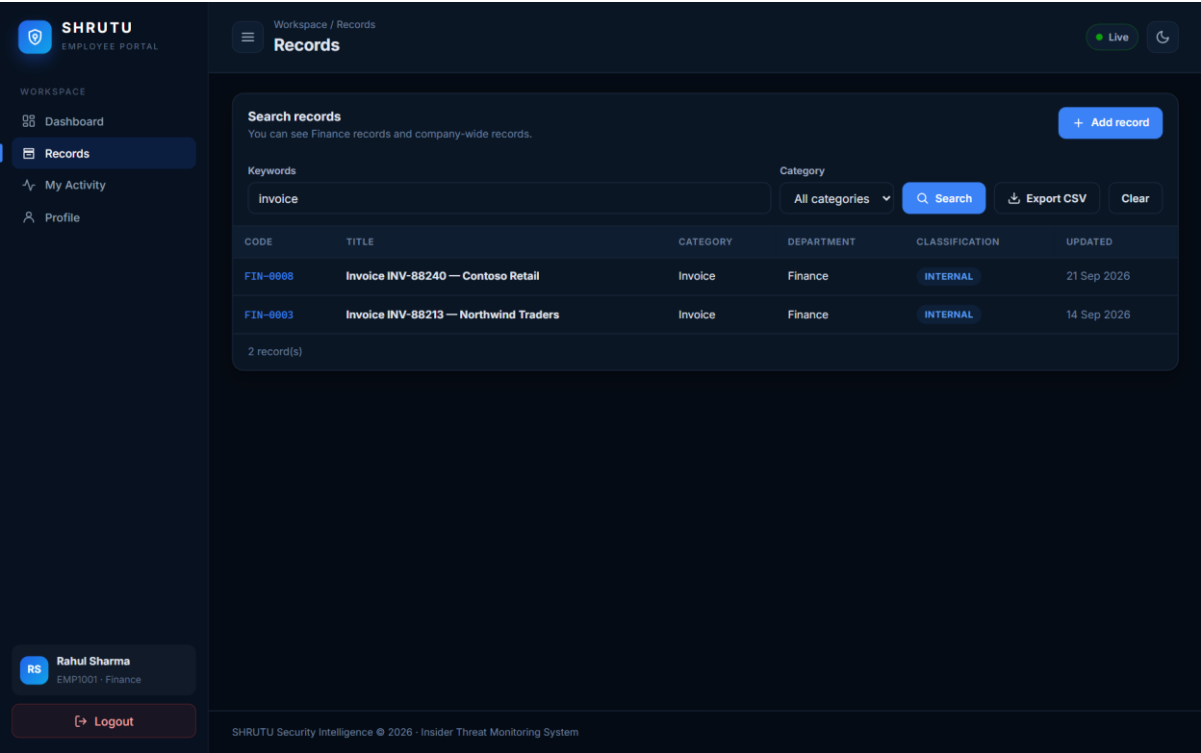


Figure 6.10: Employee record search

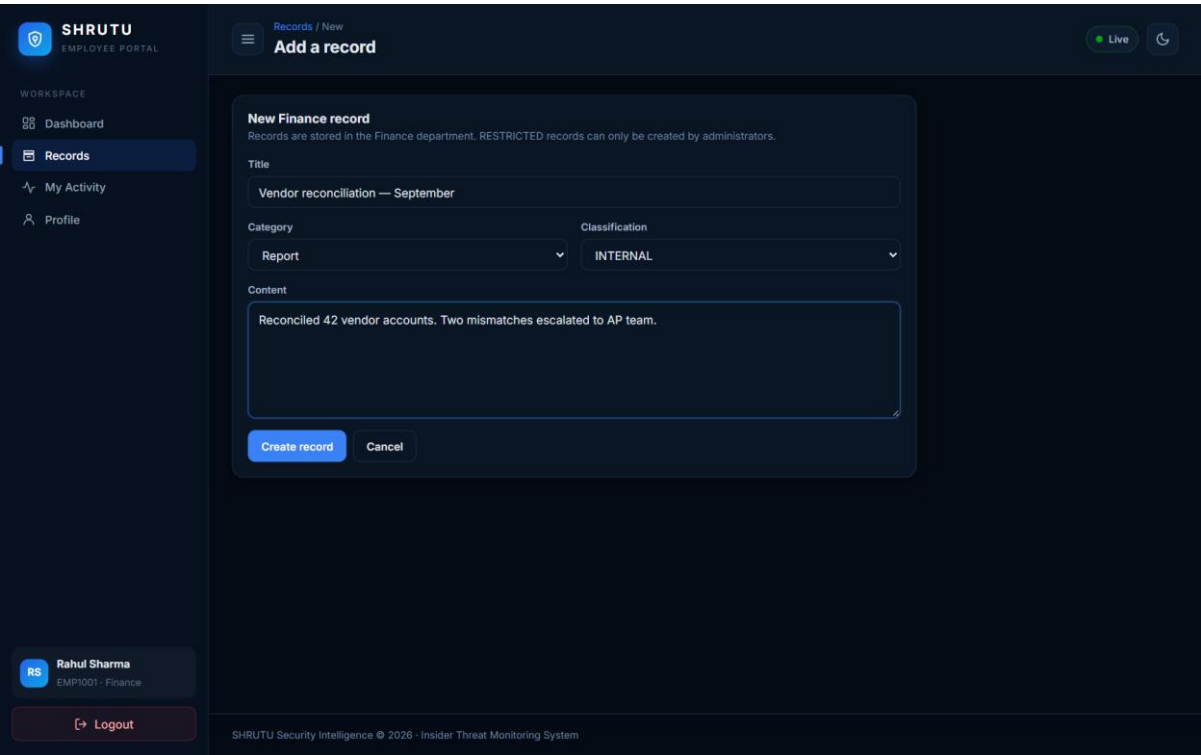


Figure 6.11: Add record form

The screenshot shows the SHRUTU Employee Portal interface. The left sidebar contains navigation links: Dashboard, Records (selected), My Activity, and Profile. The main content area displays a record titled "Vendor reconciliation — September" under the "Records / FIN-0011" section. A green banner at the top states "Record FIN-0011 created." Below this, the record details show "FIN-0011 · Vendor reconciliation — September" with a sub-header "Report · Finance · updated 29 Sep 2026, 17:44:44". The record content states "Reconciled 42 vendor accounts. Two mismatches escalated to AP team." and includes a timestamp "Created 29 Sep 2026, 17:44:44 by EMP1001 · last edited by EMP1001". The bottom left shows the user profile for Rahul Sharma (EMP1001 - Finance) and a Logout button. The footer indicates "SHRUTU Security Intelligence © 2026 · Insider Threat Monitoring System".

Figure 6.12: Record created

The screenshot displays the SHRUTU Security Console interface. The left sidebar includes navigation links: Dashboard (selected), Live Activity, Alerts (19), Risk Monitoring, Employees, Audit Logs, and Settings. The main content area is titled "Security Dashboard" and provides an overview of system status: 17 total employees (16 active accounts), 1 online now (live presence), 91 activities today (21 risk-scored), and 19 new alerts (awaiting review). A "Live Activity Feed" lists recent actions by employees, such as "VIEW_RECORD", "ADD_RECORD", "SEARCH", "LOGIN", "CREATE_EMPLOYEE", "MANAGER_LOGIN", "FAILED_LOGIN", and "LOGOUT", with associated risk scores. A "Security Alerts" section at the bottom shows the most recent alerts raised by the risk engine. On the right, two "NEW MEDIUM ALERT" pop-ups are visible, detailing specific security events like "Sensitive record accessed" and "Login from a new location". A line graph titled "Activity, last 24 hours" shows the volume of all activity and risk-scored activity over time.

Figure 6.13: Manager's live feed while the employee works

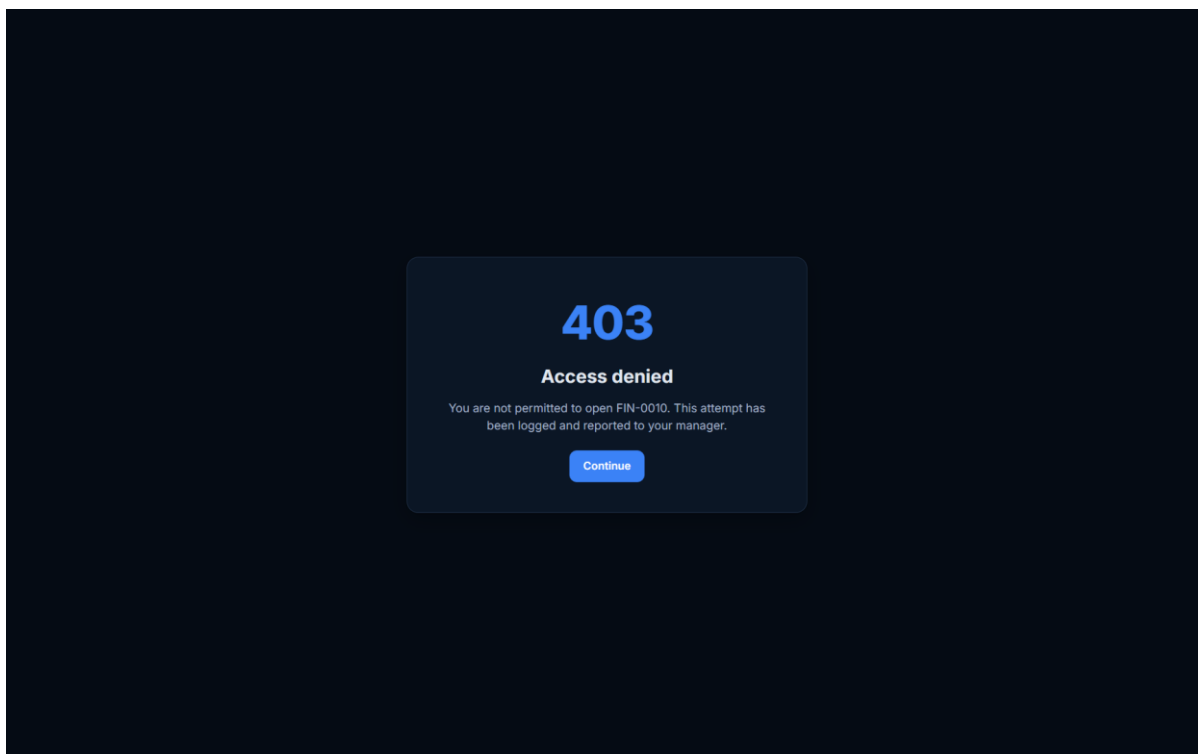


Figure 6.14: *Employee blocked from a RESTRICTED record (HTTP 403)*

SHRUTU SECURITY CONSOLE

MONITORING

- Dashboard
- Live Activity
- Alerts 24
- Risk Monitoring

MANAGEMENT

- Employees
- Audit Logs
- Settings

Security / Overview

Security Dashboard

Total Employees 17
16 active accounts

Online Now 1
live presence

Activities Today 94
24 risk-scored

New Alerts 24
awaiting review

Live Activity Feed Open log →

Every employee and manager action, as it happens

17:44:50	EMP1001	UNAUTHORIZED_ACCESS	/mana...	+65
17:44:49	EMP1001	UNAUTHORIZED_ACCESS	record...	+30
17:44:48	EMP1001	UNAUTHORIZED_ACCESS	record...	+55
17:44:44	EMP1001	VIEW_RECORD	record:FI...	SUCCESS
17:44:44	EMP1001	ADD_RECORD	record:FIN-...	SUCCESS
17:44:39	EMP1001	VIEW_RECORD	record:FIN-0004	+15
17:44:37	EMP1001	SEARCH	records	SUCCESS
17:44:35	EMP1001	LOGIN	/login	+10
17:44:29	MGR1001	CREATE_EMPLOYEE	empl...	SUCCESS
17:44:20	MGR1001	MANAGER_LOGIN	/login	+10

Activity, last 24 hours

All actions vs. actions that added risk points, per hour

— All activity — Risk-scored activity

View data table

Security Alerts

Most recent alerts raised by the risk engine — new ones appear here instantly

EMPLOYEE	ALERT	SEVERITY	RISK	TIME	STATUS	ACTION
EMP1001 - Rahul Sharma	Privilege escalation attempt	HIGH	CRITICAL	17:44:50	NEW CRITICAL-RISK ALERT (2)	View activity Acknowledge all
EMP1001 - Rahul Sharma	Repeated unauthorized access attempts	HIGH	CRITICAL	17:44:50	NEW CRITICAL-RISK ALERT (2)	View activity Acknowledge all
EMP1001 - Rahul Sharma	Unauthorized access attempt	HIGH	CRITICAL	17:44:48	NEW CRITICAL-RISK ALERT (3)	View activity Acknowledge all
EMP1001 - Rahul Sharma	Multiple suspicious events	HIGH	CRITICAL	17:44:48	NEW CRITICAL-RISK ALERT (3)	View activity Acknowledge all
EMP1001 - Rahul Sharma	Risk level escalated	HIGH	CRITICAL	17:44:48	NEW CRITICAL-RISK ALERT (3)	View activity Acknowledge all
EMP1001 - Rahul Sharma	Sensitive record accessed	LOW	LOW	17:44:39	NEW MEDIUM ALERT	View activity Acknowledge

NEW CRITICAL-RISK ALERT (2)

Employee: EMP1001 - Rahul Sharma

Alerts: HIGH Privilege escalation attempt CRITICAL Repeated unauthorized access attempts

Reason: 3 unauthorized access attempts within 15 minutes

Risk score: 100 CRITICAL

Time: 17:44:50

View activity Acknowledge all

NEW CRITICAL-RISK ALERT (3)

Employee: EMP1001 - Rahul Sharma

Alerts: HIGH Unauthorized access attempt HIGH Multiple suspicious events CRITICAL Risk level escalated

Reason: Risk level rose from LOW to CRITICAL (score 80). Latest: 3 suspicious events within 10 minutes

Risk score: 80 CRITICAL

Time: 17:44:48

View activity Acknowledge all

NEW MEDIUM ALERT

Employee: EMP1001 - Rahul Sharma

Alert: Sensitive record accessed

Reason: Viewed CONFIDENTIAL record record:FIN-0004

Risk score: 25 LOW

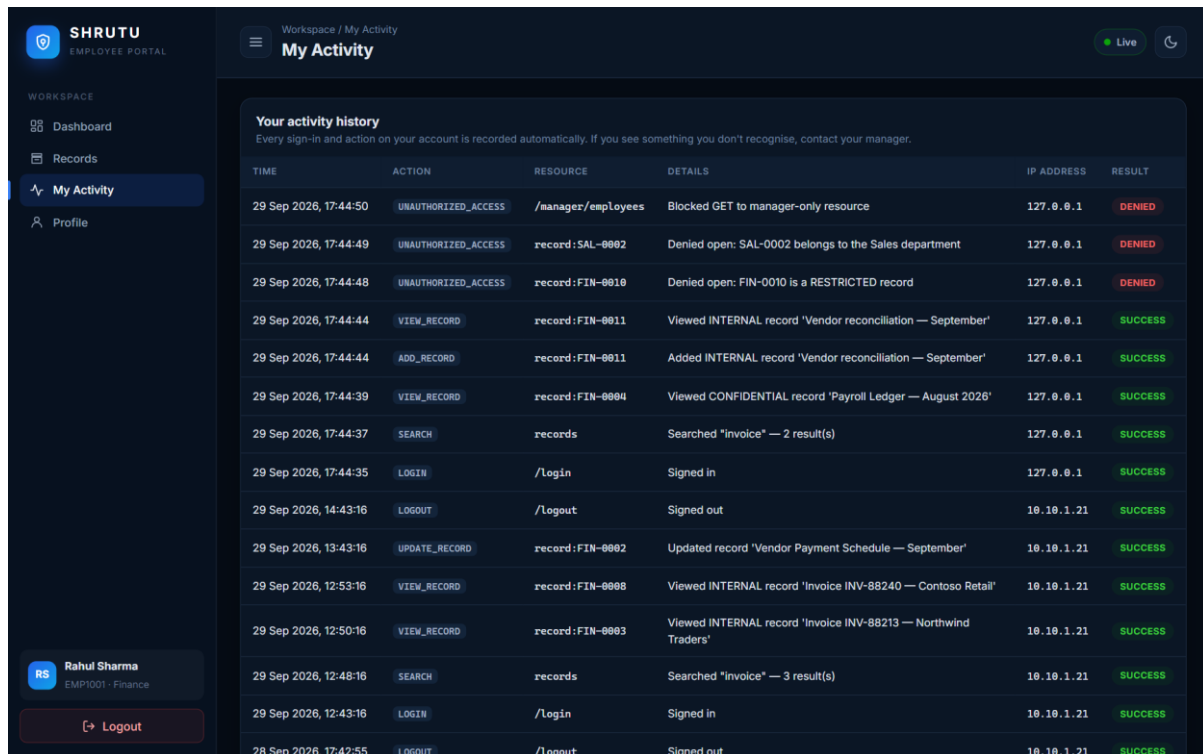
Time: 17:44:39

View activity Acknowledge

Ananya Iyer
MGR1001 - Manager

Logout

Figure 6.15: Real-time alert pop-ups on the manager's screen — no page refresh



SHRUTU
EMPLOYEE PORTAL

Workspace / My Activity

My Activity

WORKSPACE

- Dashboard
- Records
- My Activity**
- Profile

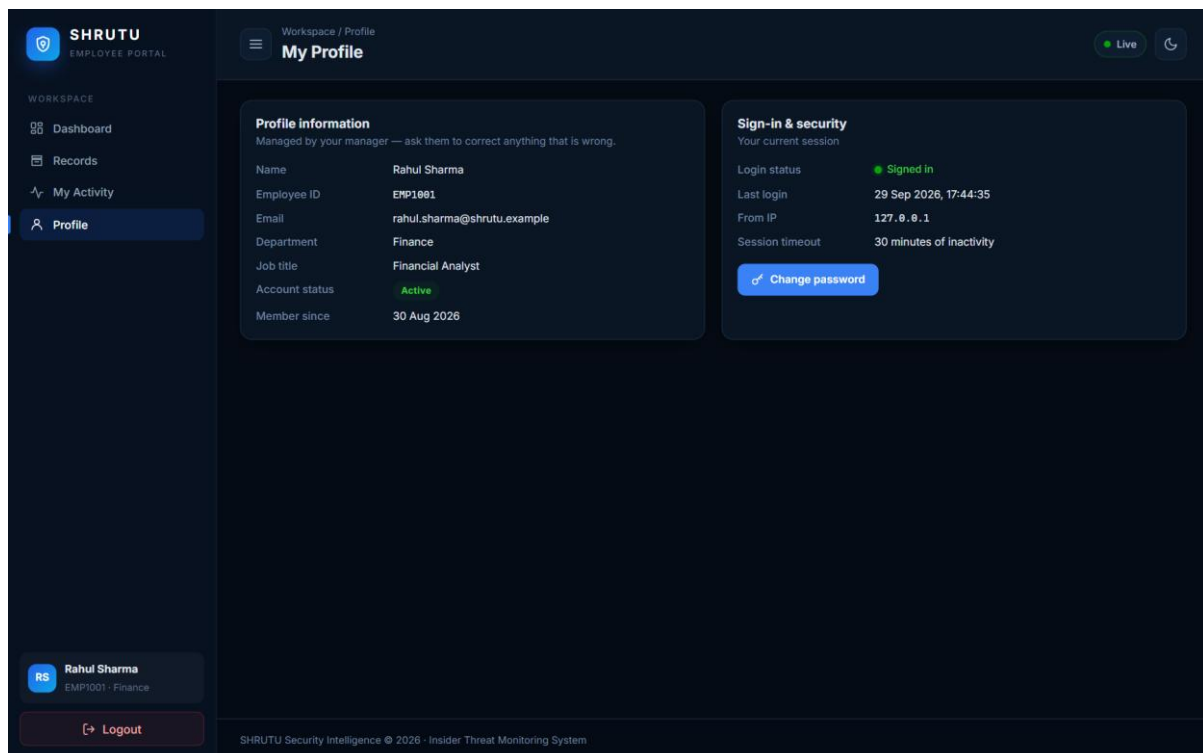
Your activity history
Every sign-in and action on your account is recorded automatically. If you see something you don't recognise, contact your manager.

TIME	ACTION	RESOURCE	DETAILS	IP ADDRESS	RESULT
29 Sep 2026, 17:44:50	UNAUTHORIZED_ACCESS	/manager/employees	Blocked GET to manager-only resource	127.0.0.1	DENIED
29 Sep 2026, 17:44:49	UNAUTHORIZED_ACCESS	record:SAL-0002	Denied open: SAL-0002 belongs to the Sales department	127.0.0.1	DENIED
29 Sep 2026, 17:44:48	UNAUTHORIZED_ACCESS	record:FIN-0010	Denied open: FIN-0010 is a RESTRICTED record	127.0.0.1	DENIED
29 Sep 2026, 17:44:44	VIEW_RECORD	record:FIN-0011	Viewed INTERNAL record 'Vendor reconciliation — September'	127.0.0.1	SUCCESS
29 Sep 2026, 17:44:44	ADD_RECORD	record:FIN-0011	Added INTERNAL record 'Vendor reconciliation — September'	127.0.0.1	SUCCESS
29 Sep 2026, 17:44:39	VIEW_RECORD	record:FIN-0004	Viewed CONFIDENTIAL record 'Payroll Ledger — August 2026'	127.0.0.1	SUCCESS
29 Sep 2026, 17:44:37	SEARCH	records	Searched "invoice" — 2 result(s)	127.0.0.1	SUCCESS
29 Sep 2026, 17:44:35	LOGIN	/Login	Signed in	127.0.0.1	SUCCESS
29 Sep 2026, 14:43:16	LOGOUT	/Logout	Signed out	10.10.1.21	SUCCESS
29 Sep 2026, 13:43:16	UPDATE_RECORD	record:FIN-0002	Updated record 'Vendor Payment Schedule — September'	10.10.1.21	SUCCESS
29 Sep 2026, 12:53:16	VIEW_RECORD	record:FIN-0008	Viewed INTERNAL record 'Invoice INV-88240 — Contoso Retail'	10.10.1.21	SUCCESS
29 Sep 2026, 12:50:16	VIEW_RECORD	record:FIN-0003	Viewed INTERNAL record 'Invoice INV-88213 — Northwind Traders'	10.10.1.21	SUCCESS
29 Sep 2026, 12:48:16	SEARCH	records	Searched "invoice" — 3 result(s)	10.10.1.21	SUCCESS
29 Sep 2026, 12:43:16	LOGIN	/Login	Signed in	10.10.1.21	SUCCESS
28 Sep 2026, 17:42:55	LOGOUT	/Logout	Signed out	10.10.1.21	SUCCESS

Rahul Sharma
EMP1001 - Finance

[Logout](#)

Figure 6.16: Employee: My Activity



SHRUTU
EMPLOYEE PORTAL

Workspace / Profile

My Profile

WORKSPACE

- Dashboard
- Records
- My Activity
- Profile**

Profile information
Managed by your manager — ask them to correct anything that is wrong.

Name	Rahul Sharma
Employee ID	EMP1001
Email	rahul.sharma@shrutu.example
Department	Finance
Job title	Financial Analyst
Account status	Active
Member since	30 Aug 2026

Sign-in & security
Your current session

Login status	Signed in
Last login	29 Sep 2026, 17:44:35
From IP	127.0.0.1
Session timeout	30 minutes of inactivity

[Change password](#)

Rahul Sharma
EMP1001 - Finance

[Logout](#)

SHRUTU Security Intelligence © 2026 - Insider Threat Monitoring System

Figure 6.17: Employee profile

SHRUTU SECURITY CONSOLE

Monitoring / Alerts **Security Alerts** Live 24

Alert Queue
Alerts are never deleted. Acknowledge to show you are investigating; resolve with a note to close.

Employee: ID or name | Status: Open (new + acknowledged) | Severity: Any | Alert type: Any | From: dd-mm-yyyy

To: dd-mm-yyyy

TIME	EMPLOYEE	ALERT	SEVERITY	RISK SCORE	STATUS	HANDLED BY	ACTION
Today 17:44:50	RS Rahul Sharma EMP1001 - Finance	Repeated unauthorized access attempts 3 unauthorized access attempts within 15 minutes	CRITICAL	100	NEW	—	View activity Acknowledge Resolve
Today 17:44:50	RS Rahul Sharma EMP1001 - Finance	Privilege escalation attempt Employee tried to open manager-only resource /manager/employees	HIGH	100	NEW	—	View activity Acknowledge Resolve
Today 17:44:48	RS Rahul Sharma EMP1001 - Finance	Risk level escalated Risk level rose from LOW to CRITICAL (score 80). Latest: 3 suspicious events within 10 minutes	CRITICAL	80	NEW	—	View activity Acknowledge Resolve
Today 17:44:48	RS Rahul Sharma EMP1001 - Finance	Multiple suspicious events 3 suspicious events within 10 minutes	HIGH	80	NEW	—	View activity Acknowledge Resolve

Ananya Iyer
MGR1001 - Manager
[Logout](#)

Figure 6.18: Alert queue

Alert details

Alert: **Repeated unauthorized access attempts #29**

Severity: **CRITICAL**

Status: **NEW**

Employee: EMP1001 - Rahul Sharma

Reason: 3 unauthorized access attempts within 15 minutes

Risk score: 100 at the time of the alert

Time: Today 17:44:50

Acknowledged: —

Resolved: —

Resolution note: —

TRIGGERING ACTIVITY

Action: **UNAUTHORIZED_ACCESS**

Resource: /manager/employees

Description: Blocked GET to manager-only resource

IP / agent: 127.0.0.1 Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0

Risk reason: +30 Attempted to access /manager/employees without permission; +10 Employee tried to open manager-only resource /manager/employees; +25 3 unauthorized access attempts within 15 minutes

[View activity](#) [Acknowledge](#) [Resolve](#)

Figure 6.19: Alert details with triggering activity

The screenshot displays the 'Activity details' modal in the SHRUTU system. The modal is dark-themed and contains the following information:

- Employee ID:** EMP1001
- Employee name:** Rahul Sharma
- Department:** Finance
- Action:** UNAUTHORIZED_ACCESS
- Resource:** /manager/employees
- Description:** Blocked GET to manager-only resource
- Timestamp:** Today 17:44:50
- IP address:** 127.0.0.1
- User agent:** Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
- Status:** DENIED
- Risk score:** +65 points added by this activity
- Employee risk now:** 100 (CRITICAL)
- RISK REASON:**
 - +30** Attempted to access /manager/employees without permission
Score 100 → 100 (capped at 100) · rule UNAUTHORIZED_ACCESS
 - +10** Employee tried to open manager-only resource /manager/employees
Score 100 → 100 (capped at 100) · rule PRIVILEGE_ESCALATION
 - +25** 3 unauthorized access attempts within 15 minutes
Score 100 → 100 (capped at 100) · rule REPEATED_UNAUTHORIZED
- ALERTS RAISED:**
 - HIGH** Privilege escalation attempt
Employee tried to open manager-only resource /manager/employees
NEW
Buttons: Acknowledge, Resolve
 - Denied unauthorized access attempts
Button: Acknowledge

Figure 6.20: Activity details with IP, user agent and point-by-point risk reason

Activity details

Alert #28 acknowledged.

Employee ID: EMP1001
 Employee name: Rahul Sharma
 Department: Finance
 Action: UNAUTHORIZED_ACCESS
 Resource: /manager/employees
 Description: Blocked GET to manager-only resource
 Timestamp: Today 17:44:50
 IP address: 127.0.0.1
 User agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
 Status: DENIED
 Risk score: +65 points added by this activity
 Employee risk now: 100 CRITICAL

RISK REASON

- +30 Attempted to access /manager/employees without permission
Score 100 → 100 (capped at 100) · rule UNAUTHORIZED_ACCESS
- +10 Employee tried to open manager-only resource /manager/employees
Score 100 → 100 (capped at 100) · rule PRIVILEGE_ESCALATION
- +25 3 unauthorized access attempts within 15 minutes
Score 100 → 100 (capped at 100) · rule REPEATED_UNAUTHORIZED

ALERTS RAISED

- Privilege escalation attempt**
Employee tried to open manager-only resource /manager/employees
HIGH ACKNOWLEDGED Resolve
- Repeated unauthorized access attempts**
3 unauthorized access attempts within 15
CRITICAL NEW Acknowledge

Figure 6.21: Alert acknowledged

SHRUTU SECURITY CONSOLE

Monitoring / Live Activity

Live Activity

Activity Log

Streams in real time. Click any row for the full record, IP address, user agent and risk explanation.

Live updates

Employee	Activity type	Result	Employee risk level	Alert severity	Alert status	
ID or name	All actions	Any	Any	Any	Any	
From: dd-mm-yyyy To: dd-mm-yyyy	Risk-scored only	Clear				
TIME	EMPLOYEE	ACTION	RESOURCE	DESCRIPTION	IP ADDRESS	STATUS
Today 17:45:04	MGR1001 Ananya Iyer - manager	ACKNOWLEDGE_ALERT	alert:28	Acknowledged HIGH alert 'Privilege escalation a...	127.0.0.1	SUCCESS
Today 17:44:50	EMP1001 Rahul Sharma	UNAUTHORIZED_ACCESS	/manager/employees	Blocked GET to manager-only resource	127.0.0.1	DENIED
Today 17:44:49	EMP1001 Rahul Sharma	UNAUTHORIZED_ACCESS	record:SAL-0002	Denied open: SAL-0002 belongs to the Sales de...	127.0.0.1	DENIED
Today 17:44:48	EMP1001 Rahul Sharma	UNAUTHORIZED_ACCESS	record:FIN-0010	Denied open: FIN-0010 is a RESTRICTED record	127.0.0.1	DENIED
Today 17:44:44	EMP1001 Rahul Sharma	VIEW_RECORD	record:FIN-0011	Viewed INTERNAL record 'Vendor reconciliation ...	127.0.0.1	SUCCESS
Today 17:44:44	EMP1001 Rahul Sharma	ADD_RECORD	record:FIN-0011	Added INTERNAL record 'Vendor reconciliation ...	127.0.0.1	SUCCESS

AI Ananya Iyer MGR1001 · Manager

Logout

Figure 6.22: Live Activity log

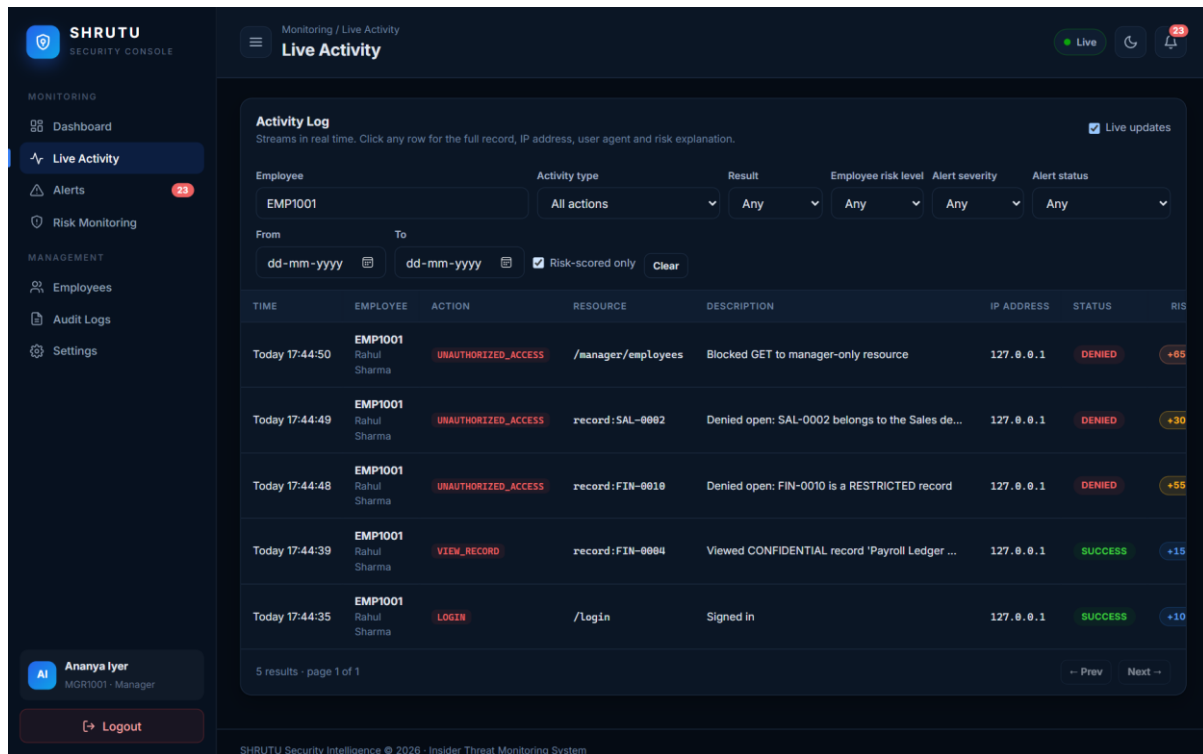


Figure 6.23: Activity log filtered by employee and risk-scored actions

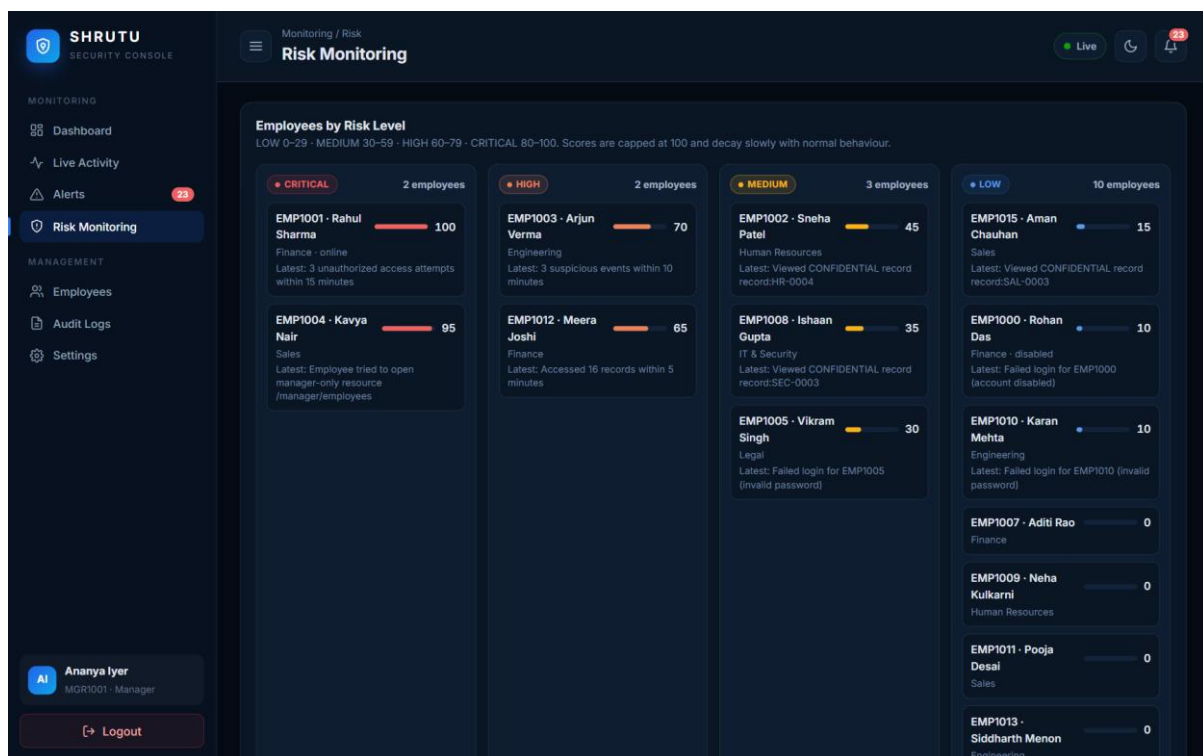


Figure 6.24: Risk Monitoring: employees grouped by level and explained score changes

SHRUTU SECURITY CONSOLE

Employees / EMP1001
Rahul Sharma

Live | 23

MONITORING

- Dashboard
- Live Activity
- Alerts (23)
- Risk Monitoring

MANAGEMENT

- Employees
- Audit Logs
- Settings

Employee Detail: Rahul Sharma (EMP1001)

Finance - Financial Analyst | rahul.sharma@shrutu.example | Employee | Active | Online

Last login: 29 Sep 2026, 17:44:35 from 127.0.0.1 | Created: 30 Aug 2026

Current risk: 100/100 (CRITICAL)

Why is the risk score 100?
Every point added by the rule engine, newest first. Scores decay by 2 points per hour of normal behaviour.

- +25 Risk increased by 25 because: 3 unauthorized access attempts within 15 minutes**
29 Sep 2026, 17:44:50 · score 100 → 100 · rule REPEATED_UNAUTHORIZED · [view activity](#)
- +10 Risk increased by 10 because: Employee tried to open manager-only resource /manager/employees**
29 Sep 2026, 17:44:50 · score 100 → 100 · rule PRIVILEGE_ESCALATION · [view activity](#)
- +30 Risk increased by 30 because: Attempted to access /manager/employees without permission**
29 Sep 2026, 17:44:50 · score 100 → 100 · rule UNAUTHORIZED_ACCESS · [view activity](#)
- +30 Risk increased by 30 because: Attempted to access record:SAL-0002 without permission**
29 Sep 2026, 17:44:49 · score 80 → 100 · rule UNAUTHORIZED_ACCESS · [view activity](#)
- +25 Risk increased by 25 because: 3 suspicious events within 10 minutes**
29 Sep 2026, 17:44:48 · score 55 → 80 · rule SUSPICIOUS_BURST · [view activity](#)
- +30 Risk increased by 30 because: Attempted to access record:FIN-0010 without permission**
29 Sep 2026, 17:44:48 · score 25 → 55 · rule UNAUTHORIZED_ACCESS · [view activity](#)
- +15 Risk increased by 15 because: Viewed CONFIDENTIAL record record:FIN-0004**
29 Sep 2026, 17:44:39 · score 10 → 25 · rule SENSITIVE_ACCESS · [view activity](#)
- +10 Risk increased by 10 because: Login from new IP address 127.0.0.1 (previously used: 10.10.1.21)**
29 Sep 2026, 17:44:35 · score 0 → 10 · rule NEW_LOCATION · [view activity](#)

Account
Edit details and access

Full name:

Email:

Department: Job title:

Role: Account status:

[Save changes](#) [Reset password](#)

ACTIONS, LAST 7 DAYS

ACTION	COUNT
VIEW_RECORD	5
LOGIN	3
SEARCH	3
UNAUTHORIZED_ACCESS	3

AI Ananya Iyer
MGR1001 · Manager

[Logout](#)

Figure 6.25: Employee detail: why the risk score is what it is

SHRUTU SECURITY CONSOLE

Management / Audit
Audit Logs

Live | 23

MONITORING

- Dashboard
- Live Activity
- Alerts (23)
- Risk Monitoring

MANAGEMENT

- Employees
- Audit Logs
- Settings

Audit Log of Manager Actions

Every action taken by managers is recorded, so the monitoring system itself stays accountable.

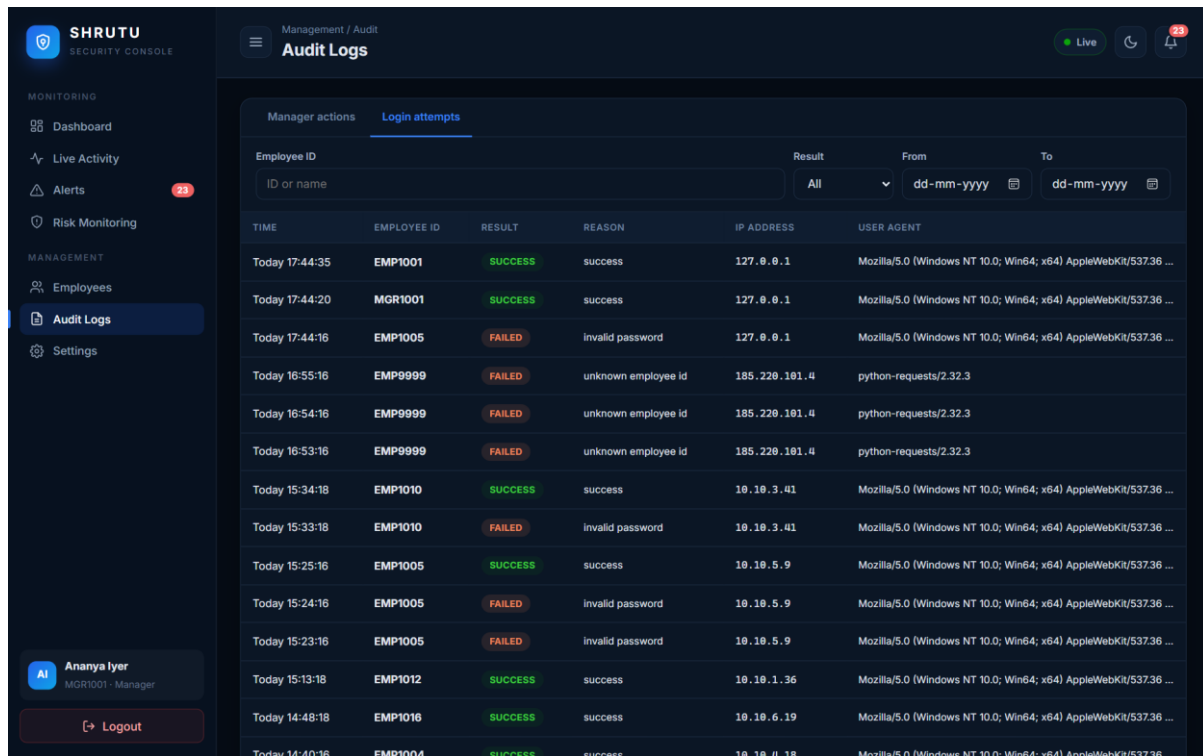
Manager: Action: From: To:

TIME	MANAGER	ACTION	TARGET	DETAILS	IP ADDRESS
Today 17:45:04	MGR1001 Ananya Iyer	ACKNOWLEDGE_ALERT	alert:28	Acknowledged HIGH alert 'Privilege escalation attempt' for...	127.0.0.1
Today 17:44:29	MGR1001 Ananya Iyer	CREATE_EMPLOYEE	employee:EMP1017	Created employee account EMP1017 (Test Employee, Fina...	127.0.0.1
Today 17:44:20	MGR1001 Ananya Iyer	MANAGER_LOGIN	/login	Signed in	127.0.0.1
Today 17:03:16	MGR1001 Ananya Iyer	ACKNOWLEDGE_ALERT	alert:14	Acknowledged HIGH alert 'Multiple failed login attempts' f...	10.10.0.5
Today 14:13:18	MGR1001 Ananya Iyer	ACKNOWLEDGE_ALERT	alert:21	Acknowledged MEDIUM alert 'Sensitive record accessed' ...	10.10.0.5
Today 12:43:16	MGR1001 Ananya Iyer	RESOLVE_ALERT	alert:2	Resolved MEDIUM alert 'Sensitive record accessed' for E...	10.10.0.5
Today 12:23:16	MGR1001 Ananya Iyer	ACKNOWLEDGE_ALERT	alert:3	Acknowledged MEDIUM alert 'Sensitive record accessed' ...	10.10.0.5
Today 10:53:16	MGR1001 Ananya Iyer	ACKNOWLEDGE_ALERT	alert:4	Acknowledged MEDIUM alert 'Access outside working hou...	10.10.0.5
Today 10:43:16	MGR1001 Ananya Iyer	MANAGER_LOGIN	/login	Signed in	10.10.0.5
26 Sep 17:43:16	MGR1001	DISABLE_EMPLOYEE	employee:EMP1000	Disabled account EMP1000 (Rohan Das) — resigned, last ...	10.10.0.5

AI Ananya Iyer
MGR1001 · Manager

[Logout](#)

Figure 6.26: Audit log of manager actions



Management / Audit

Audit Logs

Manager actions: [Login attempts](#)

Employee ID:

Result: [All](#)

From:

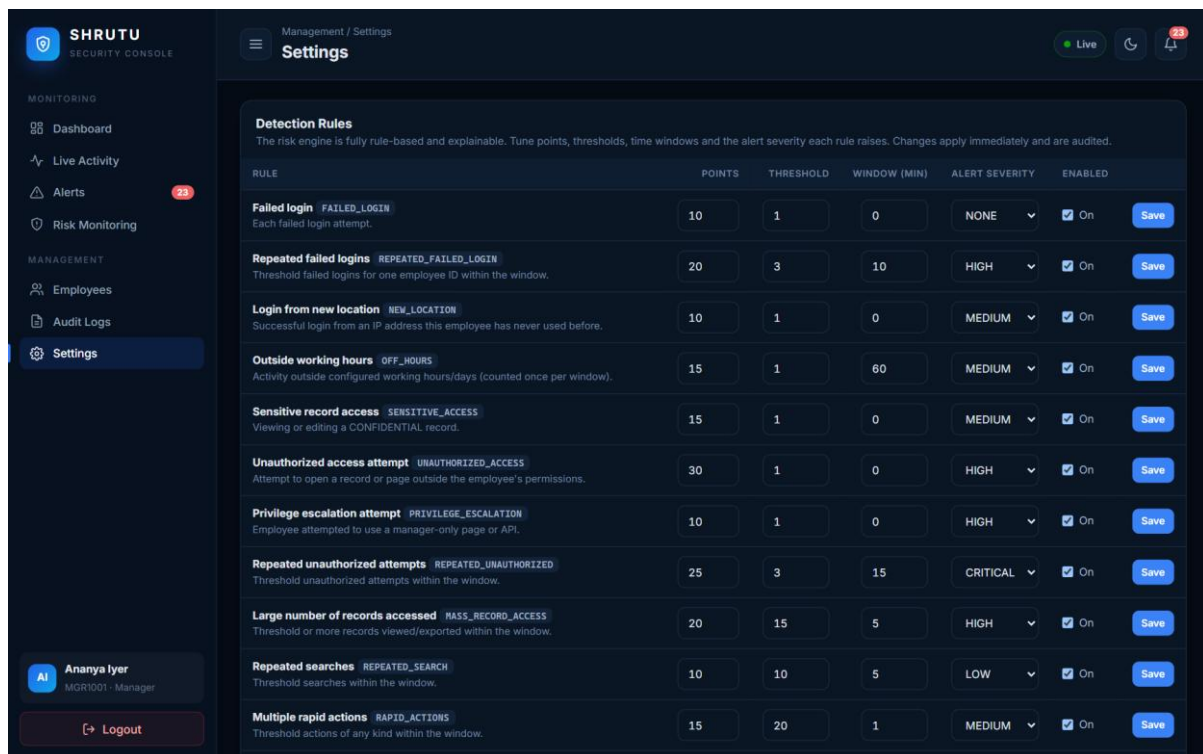
To:

TIME	EMPLOYEE ID	RESULT	REASON	IP ADDRESS	USER AGENT
Today 17:44:35	EMP1001	SUCCESS	success	127.0.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 17:44:20	MGR1001	SUCCESS	success	127.0.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 17:44:16	EMP1005	FAILED	invalid password	127.0.0.1	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 16:55:16	EMP9999	FAILED	unknown employee id	185.220.101.4	python-requests/2.32.3
Today 16:54:16	EMP9999	FAILED	unknown employee id	185.220.101.4	python-requests/2.32.3
Today 16:53:16	EMP9999	FAILED	unknown employee id	185.220.101.4	python-requests/2.32.3
Today 15:34:18	EMP1010	SUCCESS	success	10.10.3.41	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 15:33:18	EMP1010	FAILED	invalid password	10.10.3.41	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 15:25:16	EMP1005	SUCCESS	success	10.10.5.9	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 15:24:16	EMP1005	FAILED	invalid password	10.10.5.9	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 15:23:16	EMP1005	FAILED	invalid password	10.10.5.9	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 15:13:18	EMP1012	SUCCESS	success	10.10.1.36	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 14:48:18	EMP1016	SUCCESS	success	10.10.6.19	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...
Today 14:40:16	EMP1004	SUCCESS	success	10.10.0.18	Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 ...

Ananya Iyer
MGR1001 - Manager

[Logout](#)

Figure 6.27: Login attempts report



Management / Settings

Settings

Detection Rules

The risk engine is fully rule-based and explainable. Tune points, thresholds, time windows and the alert severity each rule raises. Changes apply immediately and are audited.

RULE	POINTS	THRESHOLD	WINDOW (MIN)	ALERT SEVERITY	ENABLED
Failed login <code>FAILED_LOGIN</code> Each failed login attempt.	10	1	0	NONE	☒ On Save
Repeated failed logins <code>REPEATED_FAILED_LOGIN</code> Threshold failed logins for one employee ID within the window.	20	3	10	HIGH	☒ On Save
Login from new location <code>NEW_LOCATION</code> Successful login from an IP address this employee has never used before.	10	1	0	MEDIUM	☒ On Save
Outside working hours <code>OFF_HOURS</code> Activity outside configured working hours/days (counted once per window).	15	1	60	MEDIUM	☒ On Save
Sensitive record access <code>SENSITIVE_ACCESS</code> Viewing or editing a CONFIDENTIAL record.	15	1	0	MEDIUM	☒ On Save
Unauthorized access attempt <code>UNAUTHORIZED_ACCESS</code> Attempt to open a record or page outside the employee's permissions.	30	1	0	HIGH	☒ On Save
Privilege escalation attempt <code>PRIVILEGE_ESCALATION</code> Employee attempted to use a manager-only page or API.	10	1	0	HIGH	☒ On Save
Repeated unauthorized attempts <code>REPEATED_UNAUTHORIZED</code> Threshold unauthorized attempts within the window.	25	3	15	CRITICAL	☒ On Save
Large number of records accessed <code>MASS_RECORD_ACCESS</code> Threshold or more records viewed/exported within the window.	20	15	5	HIGH	☒ On Save
Repeated searches <code>REPEATED_SEARCH</code> Threshold searches within the window.	10	10	5	LOW	☒ On Save
Multiple rapid actions <code>RAPID_ACTIONS</code> Threshold actions of any kind within the window.	15	20	1	MEDIUM	☒ On Save

Ananya Iyer
MGR1001 - Manager

[Logout](#)

Figure 6.28: Settings: configurable detection rules

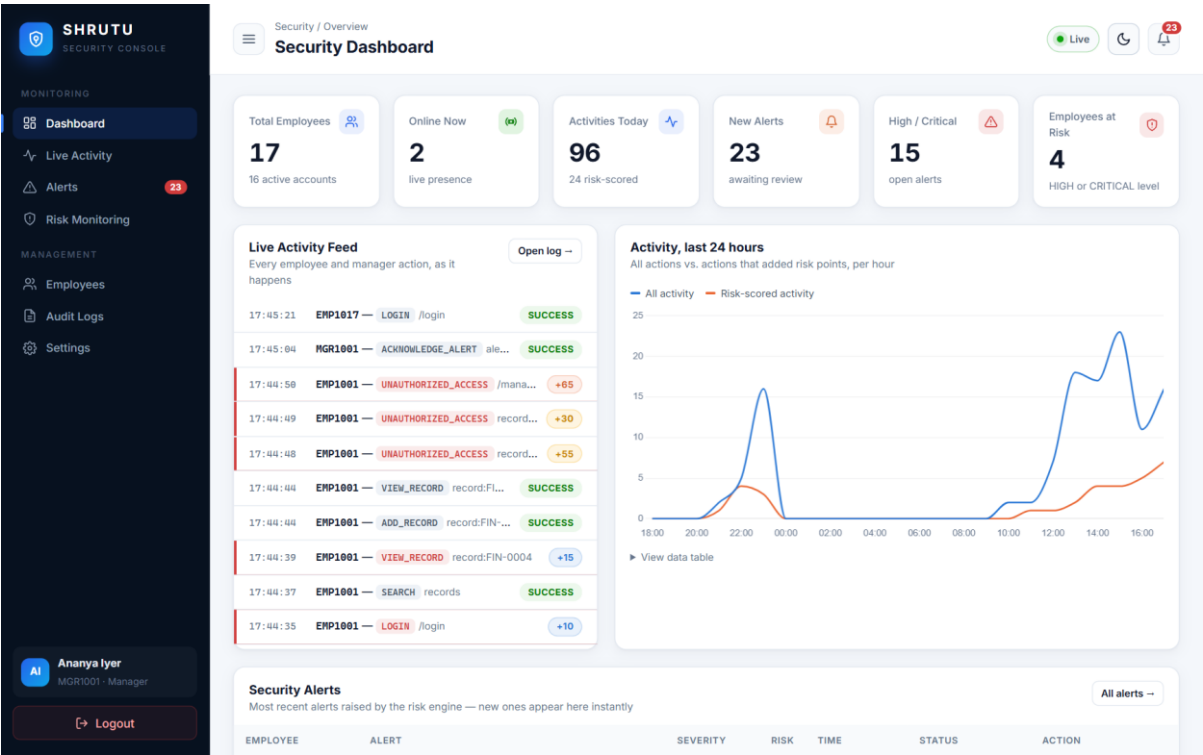


Figure 6.29: Dashboard in light theme

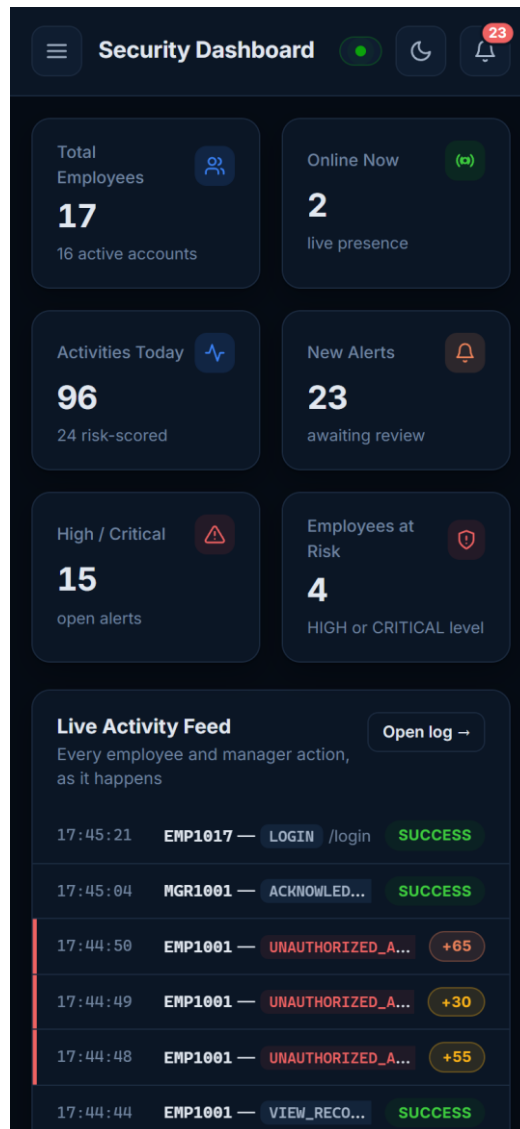


Figure 6.30: Mobile view (390 px wide)

6.2 Sample queries and outputs from the DBMS

The following queries were executed directly in the MySQL 8 command-line client on the project database (demo data). They demonstrate joins, aggregation, grouping with HAVING, sub-queries and the data dictionary.

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT u.employee_id, u.name, u.department, r.current_score AS score, r.risk_level
-> FROM users u JOIN risk_scores r ON r.user_id = u.id
-> WHERE u.role = 'employee'
-> ORDER BY r.current_score DESC
-> LIMIT 10;
```

employee_id	name	department	score	risk_level
EMP1001	Rahul Sharma	Finance	100	CRITICAL
EMP1004	Kavya Nair	Sales	95	CRITICAL
EMP1003	Arjun Verma	Engineering	70	HIGH
EMP1012	Meera Joshi	Finance	65	HIGH
EMP1002	Sneha Patel	Human Resources	45	MEDIUM
EMP1008	Ishaan Gupta	IT & Security	35	MEDIUM
EMP1005	Vikram Singh	Legal	30	MEDIUM
EMP1015	Aman Chauhan	Sales	15	LOW
EMP1000	Rohan Das	Finance	10	LOW
EMP1010	Karan Mehta	Engineering	10	LOW

10 rows in set

Figure 6.31: Employees ranked by current risk (JOIN users + risk_scores)

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT u.department, COUNT(*) AS employees, ROUND(AVG(r.current_score),1) AS avg_score,
-> SUM(r.risk_level IN ('HIGH','CRITICAL')) AS at_risk
-> FROM users u JOIN risk_scores r ON r.user_id = u.id
-> WHERE u.role = 'employee'
-> GROUP BY u.department
-> ORDER BY avg_score DESC;
```

department	employees	avg_score	at_risk
Sales	3	36.7	1
Finance	5	35.0	2
Engineering	3	26.7	1
Human Resources	2	22.5	0
IT & Security	2	17.5	0
Legal	2	15.0	0

6 rows in set

Figure 6.32: Department-wise risk summary (GROUP BY + aggregates)

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT action, COUNT(*) AS total, SUM(risk_score > 0) AS risk_scored
-> FROM activity_logs
-> WHERE timestamp >= CURDATE()
-> GROUP BY action
-> ORDER BY total DESC;
```

action	total	risk_scored
VIEW_RECORD	27	6
LOGIN	16	2
LOGOUT	13	0
SEARCH	13	0
FAILED_LOGIN	10	10
ACKNOWLEDGE_ALERT	5	0
UNAUTHORIZED_ACCESS	5	5
MANAGER_LOGIN	2	1
UPDATE_RECORD	2	0
ADD_RECORD	1	0
CREATE_EMPLOYEE	1	0
RESOLVE_ALERT	1	0

12 rows in set

Figure 6.33: Today's activity by action type (used by the dashboard chart)

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT a.id, a.employee_id, a.alert_type, a.severity, a.status, l.action, l.resource
-> FROM alerts a JOIN activity_logs l ON l.id = a.activity_id
-> WHERE a.status <> 'RESOLVED'
-> ORDER BY a.timestamp DESC
-> LIMIT 10;
```

id	employee_id	alert_type	severity	status	action	resource
28	EMP1001	PRIVILEGE_ESCALATION	HIGH	ACKNOWLEDGED	UNAUTHORIZED_ACCESS	/manager/employees
29	EMP1001	REPEATED_UNAUTHORIZED	CRITICAL	NEW	UNAUTHORIZED_ACCESS	/manager/employees
25	EMP1001	UNAUTHORIZED_ACCESS	HIGH	NEW	UNAUTHORIZED_ACCESS	record:FIN-0010
26	EMP1001	SUSPICIOUS_BURST	HIGH	NEW	UNAUTHORIZED_ACCESS	record:FIN-0010
27	EMP1001	RISK_LEVEL_ESCALATION	CRITICAL	NEW	UNAUTHORIZED_ACCESS	record:FIN-0010
24	EMP1001	SENSITIVE_ACCESS	MEDIUM	NEW	VIEW_RECORD	record:FIN-0004
23	EMP1001	NEW_LOCATION	MEDIUM	NEW	LOGIN	/login
22	MGR1001	NEW_LOCATION	MEDIUM	NEW	MANAGER_LOGIN	/login
14	EMP9999	REPEATED_FAILED_LOGIN	HIGH	ACKNOWLEDGED	FAILED_LOGIN	/login
11	EMP1004	UNAUTHORIZED_ACCESS	HIGH	NEW	UNAUTHORIZED_ACCESS	/manager/employees

10 rows in set

Figure 6.34: Open alerts with their triggering activity (JOIN alerts + activity_logs)

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT DATE_FORMAT(created_at,'%H:%i') AS time, rule_key, points, score_before, score_after
-> FROM risk_events
-> WHERE user_id = (SELECT id FROM users WHERE employee_id = 'EMP1004')
-> ORDER BY created_at, id;
```

time	rule_key	points	score_before	score_after
14:38	FAILED_LOGIN	10	0	10
15:43	SENSITIVE_ACCESS	15	10	25
16:13	UNAUTHORIZED_ACCESS	30	25	55
16:28	UNAUTHORIZED_ACCESS	30	55	85
16:28	PRIVILEGE_ESCALATION	10	85	95

5 rows in set

Figure 6.35: Why is EMP1004's score 95? (explainable risk events)

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> SELECT employee_id, COUNT(*) AS failures, GROUP_CONCAT(DISTINCT reason) AS reasons,
-> GROUP_CONCAT(DISTINCT ip_address) AS ips
-> FROM login_attempts
-> WHERE success = 0 AND timestamp >= NOW() - INTERVAL 1 DAY
-> GROUP BY employee_id
-> HAVING failures >= 1
-> ORDER BY failures DESC;
```

employee_id	failures	reasons	ips
EMP1005	3	INVALID_PASSWORD	10.10.5.9,127.0.0.1
EMP9999	3	UNKNOWN_EMPLOYEE_ID	185.220.101.4
EMP1000	1	ACCOUNT_DISABLED	49.36.120.7
EMP1004	1	INVALID_PASSWORD	10.10.4.18
EMP1008	1	INVALID_PASSWORD	10.10.6.12
EMP1010	1	INVALID_PASSWORD	10.10.3.41

6 rows in set

Figure 6.36: Failed login attempts in the last 24 hours (HAVING)

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> SELECT severity, status, COUNT(*) AS alerts
-> FROM alerts
-> GROUP BY severity, status
-> ORDER BY FIELD(severity, 'CRITICAL', 'HIGH', 'MEDIUM', 'LOW'), status;

+-----+-----+-----+
| severity | status | alerts |
+-----+-----+-----+
| CRITICAL | NEW    | 3      |
| HIGH     | NEW    | 10     |
| HIGH     | ACKNOWLEDGED | 2      |
| MEDIUM  | NEW    | 9      |
| MEDIUM  | ACKNOWLEDGED | 3      |
| MEDIUM  | RESOLVED | 1      |
| LOW      | NEW    | 1      |
+-----+-----+-----+

7 rows in set
```

Figure 6.37: Alert counts by severity and status

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> SELECT DATE_FORMAT(timestamp, '%d %b %H:%i') AS time, employee_id, action, resource
-> FROM activity_logs
-> WHERE actor_role = 'manager'
-> ORDER BY timestamp DESC
-> LIMIT 8;

+-----+-----+-----+-----+
| time | employee_id | action | resource |
+-----+-----+-----+-----+
| 29 Sep 17:45 | MGR1001 | ACKNOWLEDGE_ALERT | alert:28 |
| 29 Sep 17:44 | MGR1001 | CREATE_EMPLOYEE | employee:EMP1017 |
| 29 Sep 17:44 | MGR1001 | MANAGER_LOGIN | /login |
| 29 Sep 17:03 | MGR1001 | ACKNOWLEDGE_ALERT | alert:14 |
| 29 Sep 14:13 | MGR1001 | ACKNOWLEDGE_ALERT | alert:21 |
| 29 Sep 12:43 | MGR1001 | RESOLVE_ALERT | alert:2 |
| 29 Sep 12:23 | MGR1001 | ACKNOWLEDGE_ALERT | alert:3 |
| 29 Sep 10:53 | MGR1001 | ACKNOWLEDGE_ALERT | alert:4 |
+-----+-----+-----+-----+

8 rows in set
```

Figure 6.38: Manager audit trail (latest actions)

```
MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

mysql> SELECT department, SUM(sensitivity='PUBLIC') AS public_, SUM(sensitivity='INTERNAL') AS internal_,
-> SUM(sensitivity='CONFIDENTIAL') AS confidential_, SUM(sensitivity='RESTRICTED') AS restricted_
-> FROM records GROUP BY department ORDER BY department;

+-----+-----+-----+-----+-----+
| department | public_ | internal_ | confidential_ | restricted_ |
+-----+-----+-----+-----+-----+
| Engineering | 0 | 4 | 2 | 0 |
| Finance | 1 | 6 | 3 | 1 |
| General | 4 | 1 | 0 | 0 |
| Human Resources | 1 | 2 | 2 | 1 |
| IT & Security | 1 | 1 | 1 | 1 |
| Legal | 0 | 2 | 1 | 1 |
| Sales | 0 | 3 | 2 | 0 |
+-----+-----+-----+-----+-----+

7 rows in set
```

Figure 6.39: Records by department and classification

MySQL 8.0 Command Line Client — database: insider_threat_system (demo copy)

```
mysql> DESCRIBE activity_logs;
```

Field	Type	Null	Key	Default	Extra
id	bigint	NO	PRI	NULL	auto_increment
user_id	int	YES	MUL	NULL	
employee_id	varchar(20)	NO	MUL	NULL	
actor_role	varchar(20)	YES		NULL	
action	varchar(40)	NO	MUL	NULL	
resource	varchar(255)	YES		NULL	
description	varchar(500)	YES		NULL	
record_count	int	NO		0	
timestamp	datetime	NO	MUL	CURRENT_TIMESTAMP	DEFAULT_GENERATED
ip_address	varchar(45)	YES		NULL	
user_agent	varchar(255)	YES		NULL	
status	varchar(10)	NO		SUCCESS	
risk_score	int	NO		0	
risk_reason	varchar(1000)	YES		NULL	

14 rows in set

Figure 6.40: Structure of the *activity_logs* table

7. Conclusion

7.1 Summary of achievements

The project delivered a complete, working insider-threat monitoring system rather than a static prototype. A normalised MySQL schema of nine tables and thirteen foreign keys stores users, business records, activity, login attempts, risk scores with their explanations, configurable rules and alerts. Every employee and manager action flows through a single transactional pipeline that logs it, scores it with thirteen explainable rules, raises alerts and — only after the data is committed — pushes events to managers' browsers in real time. The system provides separate, secure interfaces for employees and managers, employee management, investigation tools, charts, search and filters, and a full audit trail. It passed 52 automated tests and an end-to-end demonstration, and it is deployed publicly with a cloud MySQL database.

7.2 Benefits

- Suspicious behaviour is detected automatically and reported within about a second instead of days later.
- Every alert and every risk point is explained in plain language, so managers can trust and act on the results.
- Server-side access control not only blocks misuse but turns every blocked attempt into evidence.
- The monitoring system is itself accountable: all manager actions are audited and alerts are never deleted.
- Strong data integrity through foreign keys and transactions; no partial or orphaned monitoring data.
- Zero licence and hosting cost, with thresholds tunable by non-programmers.

7.3 Limitations

- Monitors only actions inside this application.
- Rule-based detection needs thresholds tuned per organisation and cannot discover completely new patterns by itself.
- IP-based location check only; no device fingerprinting.
- Single server process for Socket.IO; free hosting tiers sleep when idle.

8. Future Enhancements

- **Machine-learning anomaly detection:** learn each employee's normal behaviour (time, volume, resources) and flag deviations, alongside the explainable rules.
- **Geo-IP and device fingerprinting** for real "unusual location / new device" detection.
- **Multi-channel escalation:** e-mail, SMS or Slack/Teams messages for CRITICAL alerts and an on-call rota.
- **Case management:** assign alerts to investigators, add comments and attachments, track SLAs.
- **Multi-factor authentication** for managers, and a separate read-only auditor role.
- **Integration with other systems** (e-mail gateway, file servers, VPN, cloud storage) through an ingestion API.
- **Advanced analytics:** department risk trends, weekly reports exported to PDF, peer-group comparison.
- **Mobile app** for managers with push notifications.
- **Scalability:** Redis message queue for multiple Socket.IO workers, table partitioning and archiving of old activity logs, Alembic schema migrations.

9. Bibliography / References

- [1] Silberschatz, A., Korth, H. F., & Sudarshan, S. Database System Concepts, 7th ed. McGraw-Hill, 2019.
- [2] Elmasri, R., & Navathe, S. B. Fundamentals of Database Systems, 7th ed. Pearson, 2016.
- [3] Oracle Corporation. MySQL 8.0 Reference Manual.
<https://dev.mysql.com/doc/refman/8.0/en/>
- [4] Pallets Projects. Flask Documentation (3.1). <https://flask.palletsprojects.com/>
- [5] SQLAlchemy Authors. SQLAlchemy 2.x Documentation.
<https://docs.sqlalchemy.org/>
- [6] Grinberg, M. Flask-SocketIO Documentation. <https://flask-socketio.readthedocs.io/>
- [7] Flask-Login and Flask-WTF documentation. <https://flask-login.readthedocs.io/> ;
<https://flask-wtf.readthedocs.io/>
- [8] Socket.IO Documentation (v4). <https://socket.io/docs/v4/>
- [9] Chart.js Documentation (v4). <https://www.chartjs.org/docs/latest/>
- [10] OWASP Foundation. OWASP Top 10 and Authentication / Session Management Cheat Sheets. <https://owasp.org/>
- [11] CERT Insider Threat Center, Carnegie Mellon University. Common Sense Guide to Mitigating Insider Threats, 7th ed. SEI, 2022.
- [12] Cybersecurity and Infrastructure Security Agency (CISA). Insider Threat Mitigation Guide. 2020.
- [13] Render Documentation. <https://render.com/docs> ; Aiven for MySQL Documentation.
<https://aiven.io/docs/products/mysql>
- [14] pytest Documentation. <https://docs.pytest.org/>

10. Appendix

Appendix A — Complete database creation script (schema.sql)

```
-- =====
-- SHRUTU Insider Threat Monitoring System – MySQL 8 schema
--
-- Run:  mysql -u root -p < database/schema.sql
-- (or)  python init_db.py          (reads credentials from .env)
--
-- WARNING: re-running this file drops and recreates the tables below
-- (only in the insider_threat_system database).
-- =====

CREATE DATABASE IF NOT EXISTS insider_threat_system
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE insider_threat_system;

SET FOREIGN_KEY_CHECKS = 0;
DROP TABLE IF EXISTS risk_events;
DROP TABLE IF EXISTS alerts;
DROP TABLE IF EXISTS activity_logs;
DROP TABLE IF EXISTS login_attempts;
DROP TABLE IF EXISTS risk_scores;
DROP TABLE IF EXISTS risk_rules;
DROP TABLE IF EXISTS records;
DROP TABLE IF EXISTS users;
DROP TABLE IF EXISTS departments;
SET FOREIGN_KEY_CHECKS = 1;

-----
-- Departments (lookup)
-----
CREATE TABLE departments (
  id      INT AUTO_INCREMENT PRIMARY KEY,
  name    VARCHAR(60) NOT NULL UNIQUE
) ENGINE=InnoDB;

-----
-- Users: employees and managers. Passwords are Werkzeug (scrypt) hashes.
-----
CREATE TABLE users (
  id                INT AUTO_INCREMENT PRIMARY KEY,
  employee_id       VARCHAR(20) NOT NULL UNIQUE,
  name              VARCHAR(100) NOT NULL,
  email             VARCHAR(120) NOT NULL UNIQUE,
  password_hash     VARCHAR(255) NOT NULL,
  department        VARCHAR(60) NOT NULL,
  job_title         VARCHAR(80) NULL,
  role              ENUM('employee','manager') NOT NULL DEFAULT 'employee',
  status            ENUM('active','disabled') NOT NULL DEFAULT 'active',
  must_change_password TINYINT(1) NOT NULL DEFAULT 1,
  is_online         TINYINT(1) NOT NULL DEFAULT 0,
  last_login        DATETIME NULL,
  last_login_ip     VARCHAR(45) NULL,
  last_seen         DATETIME NULL,
  created_by        INT NULL,
  created_at        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at        DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX ix_users_employee_id (employee_id),
  INDEX ix_users_role_status (role, status),
  CONSTRAINT fk_users_department FOREIGN KEY (department) REFERENCES departments(name) ON UPDATE CASCADE,
  CONSTRAINT fk_users_created_by FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Business records employees work with (the monitored resource)
-----
CREATE TABLE records (
  id            INT AUTO_INCREMENT PRIMARY KEY,
  record_code   VARCHAR(20) NOT NULL UNIQUE,
  title         VARCHAR(150) NOT NULL,
  category      VARCHAR(30) NOT NULL DEFAULT 'Other',
  department    VARCHAR(60) NOT NULL,
```

```

    sensitivity ENUM('PUBLIC','INTERNAL','CONFIDENTIAL','RESTRICTED') NOT NULL DEFAULT 'INTERNAL',
    content TEXT NOT NULL,
    created_by INT NULL,
    updated_by INT NULL,
    created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
    INDEX ix_records_department (department),
    CONSTRAINT fk_records_created_by FOREIGN KEY (created_by) REFERENCES users(id) ON DELETE SET NULL,
    CONSTRAINT fk_records_updated_by FOREIGN KEY (updated_by) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Activity logs: every meaningful action (employees AND managers)
-----
CREATE TABLE activity_logs (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    employee_id VARCHAR(20) NOT NULL,
    actor_role VARCHAR(20) NULL,
    action VARCHAR(40) NOT NULL,
    resource VARCHAR(255) NULL,
    description VARCHAR(500) NULL,
    record_count INT NOT NULL DEFAULT 0,
    timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    ip_address VARCHAR(45) NULL,
    user_agent VARCHAR(255) NULL,
    status VARCHAR(10) NOT NULL DEFAULT 'SUCCESS',
    risk_score INT NOT NULL DEFAULT 0,
    risk_reason VARCHAR(1000) NULL,
    INDEX ix_activity_user_time (user_id, timestamp),
    INDEX ix_activity_employee_time (employee_id, timestamp),
    INDEX ix_activity_action (action),
    INDEX ix_activity_timestamp (timestamp),
    CONSTRAINT fk_activity_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Alerts: never deleted, kept as an audit trail
-----
CREATE TABLE alerts (
    id BIGINT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NULL,
    employee_id VARCHAR(20) NOT NULL,
    activity_id BIGINT NULL,
    alert_type VARCHAR(40) NOT NULL,
    severity ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL,
    message VARCHAR(500) NOT NULL,
    risk_score INT NOT NULL DEFAULT 0,
    timestamp DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    status ENUM('NEW','ACKNOWLEDGED','RESOLVED') NOT NULL DEFAULT 'NEW',
    acknowledged_by INT NULL,
    acknowledged_at DATETIME NULL,
    resolved_by INT NULL,
    resolved_at DATETIME NULL,
    resolution_note VARCHAR(500) NULL,
    INDEX ix_alerts_user (user_id),
    INDEX ix_alerts_employee (employee_id),
    INDEX ix_alerts_status (status),
    INDEX ix_alerts_timestamp (timestamp),
    CONSTRAINT fk_alerts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET
NULL,
    CONSTRAINT fk_alerts_activity FOREIGN KEY (activity_id) REFERENCES activity_logs(id) ON DELETE SET
NULL,
    CONSTRAINT fk_alerts_ack_by FOREIGN KEY (acknowledged_by) REFERENCES users(id) ON DELETE SET
NULL,
    CONSTRAINT fk_alerts_res_by FOREIGN KEY (resolved_by) REFERENCES users(id) ON DELETE SET
NULL
) ENGINE=InnoDB;

-----
-- Current risk score per user
-----
CREATE TABLE risk_scores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    user_id INT NOT NULL UNIQUE,
    current_score INT NOT NULL DEFAULT 0,
    risk_level ENUM('LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'LOW',

```

```

    last_updated      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
    last_increase_at  DATETIME NULL,
    CONSTRAINT fk_risk_scores_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
) ENGINE=InnoDB;

-----
-- Risk events: WHY a score changed (explainability)
-----
CREATE TABLE risk_events (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  user_id     INT          NOT NULL,
  activity_id BIGINT       NULL,
  rule_key    VARCHAR(40)  NOT NULL,
  points      INT          NOT NULL,
  reason      VARCHAR(500) NOT NULL,
  score_before INT         NOT NULL,
  score_after  INT         NOT NULL,
  created_at   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  INDEX ix_risk_events_user_time (user_id, created_at),
  INDEX ix_risk_events_created (created_at),
  CONSTRAINT fk_risk_events_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE
CASCADE,
  CONSTRAINT fk_risk_events_activity FOREIGN KEY (activity_id) REFERENCES activity_logs(id) ON DELETE SET
NULL
) ENGINE=InnoDB;

-----
-- Login attempts (successful and failed)
-----
CREATE TABLE login_attempts (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  employee_id VARCHAR(20) NOT NULL,
  user_id     INT          NULL,
  ip_address  VARCHAR(45)  NULL,
  user_agent  VARCHAR(255) NULL,
  timestamp   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP,
  success     TINYINT(1)  NOT NULL,
  reason      VARCHAR(60)  NULL,
  INDEX ix_login_attempts_employee_time (employee_id, timestamp),
  INDEX ix_login_attempts_ip_time (ip_address, timestamp),
  INDEX ix_login_attempts_timestamp (timestamp),
  CONSTRAINT fk_login_attempts_user FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE SET NULL
) ENGINE=InnoDB;

-----
-- Configurable detection rules (editable in Manager > Settings)
-----
CREATE TABLE risk_rules (
  id          INT AUTO_INCREMENT PRIMARY KEY,
  rule_key    VARCHAR(40) NOT NULL UNIQUE,
  name        VARCHAR(100) NOT NULL,
  description  VARCHAR(255) NULL,
  points      INT          NOT NULL DEFAULT 0,
  threshold   INT          NOT NULL DEFAULT 1,
  window_minutes INT      NOT NULL DEFAULT 0,
  severity    ENUM('NONE','LOW','MEDIUM','HIGH','CRITICAL') NOT NULL DEFAULT 'NONE',
  enabled     TINYINT(1)  NOT NULL DEFAULT 1,
  updated_at   DATETIME    NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP
) ENGINE=InnoDB;

-----
-- Reference data required by the application
-----
INSERT INTO departments (name) VALUES
  ('Finance'), ('Human Resources'), ('Engineering'),
  ('IT & Security'), ('Sales'), ('Legal');

INSERT INTO risk_rules (rule_key, name, description, points, threshold, window_minutes, severity) VALUES
  ('FAILED_LOGIN', 'Failed login', 'Each failed login attempt.',
10, 1, 0, 'NONE'),
  ('REPEATED_FAILED_LOGIN', 'Repeated failed logins', 'Threshold failed logins for one employee ID
within the window.', 20, 3, 10, 'HIGH'),
  ('NEW_LOCATION', 'Login from new location', 'Successful login from an IP address this
employee has never used before.', 10, 1, 0, 'MEDIUM'),
  ('OFF_HOURS', 'Outside working hours', 'Activity outside configured working hours/days
(counted once per window).', 15, 1, 60, 'MEDIUM'),

```

```

('SENSITIVE_ACCESS',      'Sensitive record access',      'Viewing or editing a CONFIDENTIAL record.',
15, 1, 0, 'MEDIUM'),
('UNAUTHORIZED_ACCESS',   'Unauthorized access attempt',   'Attempt to open a record or page outside the
employee's permissions.',      30, 1, 0, 'HIGH'),
('PRIVILEGE_ESCALATION',   'Privilege escalation attempt',   'Employee attempted to use a manager-only page
or API.',                      10, 1, 0, 'HIGH'),
('REPEATED_UNAUTHORIZED', 'Repeated unauthorized attempts', 'Threshold unauthorized attempts within the
window.',                      25, 3, 15, 'CRITICAL'),
('MASS_RECORD_ACCESS',     'Large number of records accessed', 'Threshold or more records viewed/exported
within the window.',          20, 15, 5, 'HIGH'),
('REPEATED_SEARCH',       'Repeated searches',              'Threshold searches within the window.',
10, 10, 5, 'LOW'),
('RAPID_ACTIONS',         'Multiple rapid actions',         'Threshold actions of any kind within the
window.',                     15, 20, 1, 'MEDIUM'),
('UNUSUAL_FREQUENCY',     'Unusual access frequency',       'Actions in the window exceed threshold x the
employee's normal hourly rate (7-day baseline).', 15, 3, 60, 'MEDIUM'),
('SUSPICIOUS_BURST',      'Multiple suspicious events',     'Threshold risk-scored events (excluding failed
logins) within the window.',   25, 3, 10, 'HIGH');

```

Appendix B — Source code of key modules

The complete source code ($\approx 7,000$ lines) is available in the project repository. The listings below cover the database models and the core monitoring logic.

B.1 models/user.py

```

from flask import current_app
from flask_login import UserMixin
from werkzeug.security import check_password_hash, generate_password_hash

from extensions import db
from utils.helpers import iso, now

ROLES = ("employee", "manager")
ACCOUNT_STATUSES = ("active", "disabled")

class Department(db.Model):
    __tablename__ = "departments"

    id = db.Column(db.Integer, primary_key=True)
    name = db.Column(db.String(60), unique=True, nullable=False)

class User(UserMixin, db.Model):
    __tablename__ = "users"

    id = db.Column(db.Integer, primary_key=True)
    employee_id = db.Column(db.String(20), unique=True, nullable=False, index=True)
    name = db.Column(db.String(100), nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)
    password_hash = db.Column(db.String(255), nullable=False)
    department = db.Column(db.String(60), db.ForeignKey("departments.name"), nullable=False)
    job_title = db.Column(db.String(80))
    role = db.Column(db.Enum(*ROLES, name="user_role"), nullable=False, default="employee")
    status = db.Column(db.Enum(*ACCOUNT_STATUSES, name="account_status"), nullable=False, default="active")
    must_change_password = db.Column(db.Boolean, nullable=False, default=True)
    is_online = db.Column(db.Boolean, nullable=False, default=False)
    last_login = db.Column(db.DateTime)
    last_login_ip = db.Column(db.String(45))
    last_seen = db.Column(db.DateTime)
    created_by = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    created_at = db.Column(db.DateTime, nullable=False, default=now)
    updated_at = db.Column(db.DateTime, nullable=False, default=now, onupdate=now)

    risk = db.relationship("RiskScore", uselist=False, back_populates="user", lazy="joined")

# ----- passwords

```

```

def set_password(self, raw_password):
    self.password_hash = generate_password_hash(raw_password)

def check_password(self, raw_password):
    return check_password_hash(self.password_hash, raw_password)

# ----- helpers
@property
def is_manager(self):
    return self.role == "manager"

@property
def is_active(self): # used by Flask-Login: disabled accounts cannot hold a session
    return self.status == "active"

@property
def online(self):
    if not self.is_online or not self.last_seen:
        return False
    timeout = current_app.config["PRESENCE_TIMEOUT_SECONDS"]
    return (now() - self.last_seen).total_seconds() <= timeout

@property
def initials(self):
    parts = [p for p in self.name.split() if p]
    return "".join(p[0] for p in parts[:2]).upper() or "?"

def to_public_dict(self):
    """Safe for the employee themself (no risk data, never the password hash)."""
    return {
        "employee_id": self.employee_id,
        "name": self.name,
        "email": self.email,
        "department": self.department,
        "job_title": self.job_title,
        "role": self.role,
        "status": self.status,
        "last_login": iso(self.last_login),
        "created_at": iso(self.created_at),
    }

def to_manager_dict(self):
    """Full monitoring view for managers (still never the password hash)."""
    data = self.to_public_dict()
    data.update({
        "id": self.id,
        "online": self.online,
        "last_seen": iso(self.last_seen),
        "last_login_ip": self.last_login_ip,
        "must_change_password": self.must_change_password,
        "updated_at": iso(self.updated_at),
        "risk_score": self.risk.current_score if self.risk else 0,
        "risk_level": self.risk.risk_level if self.risk else "LOW",
    })
    return data

class LoginAttempt(db.Model):
    __tablename__ = "login_attempts"

    id = db.Column(db.BigInteger().with_variant(db.Integer, "sqlite"), primary_key=True)
    employee_id = db.Column(db.String(20), nullable=False, index=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    ip_address = db.Column(db.String(45))
    user_agent = db.Column(db.String(255))
    timestamp = db.Column(db.DateTime, nullable=False, default=now, index=True)
    success = db.Column(db.Boolean, nullable=False)
    reason = db.Column(db.String(60))

    def to_dict(self):
        return {
            "id": self.id,
            "employee_id": self.employee_id,

```

```

        "ip_address": self.ip_address,
        "user_agent": self.user_agent,
        "timestamp": iso(self.timestamp),
        "success": self.success,
        "reason": self.reason,
    }

```

B.2 models/activity.py

```

from extensions import db
from utils.helpers import iso, now

BigId = db.BigInteger().with_variant(db.Integer, "sqlite")

# Actions performed by employees
EMPLOYEE_ACTIONS = (
    "LOGIN", "LOGOUT", "FAILED_LOGIN", "SEARCH", "VIEW_RECORD", "ADD_RECORD",
    "UPDATE_RECORD", "EXPORT_DATA", "UNAUTHORIZED_ACCESS", "PASSWORD_CHANGE",
)
# Actions performed by managers (audit trail)
MANAGER_ACTIONS = (
    "MANAGER_LOGIN", "CREATE_EMPLOYEE", "UPDATE_EMPLOYEE", "DISABLE_EMPLOYEE",
    "ENABLE_EMPLOYEE", "RESET_PASSWORD", "ACKNOWLEDGE_ALERT", "RESOLVE_ALERT",
    "UPDATE_RULE",
)
ACTIVITY_STATUSES = ("SUCCESS", "FAILED", "DENIED")

class ActivityLog(db.Model):
    __tablename__ = "activity_logs"

    id = db.Column(BigId, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"), index=True)
    employee_id = db.Column(db.String(20), nullable=False, index=True)
    actor_role = db.Column(db.String(20)) # employee / manager / NULL for unknown IDs
    action = db.Column(db.String(40), nullable=False, index=True)
    resource = db.Column(db.String(255))
    description = db.Column(db.String(500))
    record_count = db.Column(db.Integer, nullable=False, default=0)
    timestamp = db.Column(db.DateTime, nullable=False, default=now, index=True)
    ip_address = db.Column(db.String(45))
    user_agent = db.Column(db.String(255))
    status = db.Column(db.String(10), nullable=False, default="SUCCESS")
    risk_score = db.Column(db.Integer, nullable=False, default=0)
    risk_reason = db.Column(db.String(1000))

    user = db.relationship("User", lazy="joined")

    def to_feed_dict(self):
        """Compact form pushed over Socket.IO to the manager live feed."""
        return {
            "id": self.id,
            "timestamp": iso(self.timestamp),
            "employee_id": self.employee_id,
            "name": self.user.name if self.user else None,
            "actor_role": self.actor_role,
            "action": self.action,
            "resource": self.resource,
            "status": self.status,
            "risk_score": self.risk_score,
        }

    def to_manager_dict(self):
        data = self.to_feed_dict()
        data.update({
            "description": self.description,
            "record_count": self.record_count,
            "ip_address": self.ip_address,
            "user_agent": self.user_agent,
            "risk_reason": self.risk_reason,
            "department": self.user.department if self.user else None,
        })
        return data

```

```

def to_employee_dict(self):
    """What an employee may see about their own actions: no risk information."""
    return {
        "timestamp": iso(self.timestamp),
        "action": self.action,
        "resource": self.resource,
        "description": self.description,
        "status": self.status,
        "ip_address": self.ip_address,
    }

```

B.3 models/alert.py

```

from extensions import db
from utils.helpers import iso, now

SEVERITIES = ("LOW", "MEDIUM", "HIGH", "CRITICAL")
ALERT_STATUSES = ("NEW", "ACKNOWLEDGED", "RESOLVED")

ALERT_TITLES = {
    "REPEATED_FAILED_LOGIN": "Multiple failed login attempts",
    "NEW_LOCATION": "Login from a new location",
    "OFF_HOURS": "Access outside working hours",
    "SENSITIVE_ACCESS": "Sensitive record accessed",
    "UNAUTHORIZED_ACCESS": "Unauthorized access attempt",
    "PRIVILEGE_ESCALATION": "Privilege escalation attempt",
    "REPEATED_UNAUTHORIZED": "Repeated unauthorized access attempts",
    "MASS_RECORD_ACCESS": "Large number of records accessed",
    "REPEATED_SEARCH": "Repeated searches",
    "RAPID_ACTIONS": "Multiple rapid actions",
    "UNUSUAL_FREQUENCY": "Unusual access frequency",
    "SUSPICIOUS_BURST": "Multiple suspicious events",
    "RISK_LEVEL_ESCALATION": "Risk level escalated",
    "PRIVILEGE_CHANGE": "Account privileges changed",
}

class Alert(db.Model):
    __tablename__ = "alerts"

    id = db.Column(db.BigInteger().with_variant(db.Integer, "sqlite"), primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"), index=True)
    employee_id = db.Column(db.String(20), nullable=False, index=True)
    activity_id = db.Column(db.BigInteger, db.ForeignKey("activity_logs.id", ondelete="SET NULL"))
    alert_type = db.Column(db.String(40), nullable=False)
    severity = db.Column(db.Enum(*SEVERITIES, name="alert_severity"), nullable=False)
    message = db.Column(db.String(500), nullable=False)
    risk_score = db.Column(db.Integer, nullable=False, default=0)
    timestamp = db.Column(db.DateTime, nullable=False, default=now, index=True)
    status = db.Column(db.Enum(*ALERT_STATUSES, name="alert_status"), nullable=False, default="NEW",
index=True)
    acknowledged_by = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    acknowledged_at = db.Column(db.DateTime)
    resolved_by = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    resolved_at = db.Column(db.DateTime)
    resolution_note = db.Column(db.String(500))

    user = db.relationship("User", foreign_keys=[user_id], lazy="joined")
    acknowledger = db.relationship("User", foreign_keys=[acknowledged_by])
    resolver = db.relationship("User", foreign_keys=[resolved_by])
    activity = db.relationship("ActivityLog")

    @property
    def title(self):
        return ALERT_TITLES.get(self.alert_type, self.alert_type.replace("_", " ").title())

    def to_dict(self):
        return {
            "id": self.id,
            "employee_id": self.employee_id,
            "name": self.user.name if self.user else None,

```

```

        "department": self.user.department if self.user else None,
        "activity_id": self.activity_id,
        "alert_type": self.alert_type,
        "title": self.title,
        "severity": self.severity,
        "message": self.message,
        "risk_score": self.risk_score,
        "risk_level": self.user.risk.risk_level if self.user and self.user.risk else None,
        "timestamp": iso(self.timestamp),
        "status": self.status,
        "acknowledged_by": self.acknowledger.employee_id if self.acknowledger else None,
        "acknowledged_at": iso(self.acknowledged_at),
        "resolved_by": self.resolver.employee_id if self.resolver else None,
        "resolved_at": iso(self.resolved_at),
        "resolution_note": self.resolution_note,
    }
}

```

B.4 models/risk.py

```

from extensions import db
from utils.helpers import iso, now

RISK_LEVELS = ("LOW", "MEDIUM", "HIGH", "CRITICAL")
RULE_SEVERITIES = ("NONE", "LOW", "MEDIUM", "HIGH", "CRITICAL")

class RiskScore(db.Model):
    """Current cumulative risk of one user (one row per user)."""
    __tablename__ = "risk_scores"

    id = db.Column(db.Integer, primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), unique=True,
        nullable=False)
    current_score = db.Column(db.Integer, nullable=False, default=0)
    risk_level = db.Column(db.Enum(*RISK_LEVELS, name="risk_level"), nullable=False, default="LOW")
    last_updated = db.Column(db.DateTime, nullable=False, default=now)
    last_increase_at = db.Column(db.DateTime)

    user = db.relationship("User", back_populates="risk")

class RiskEvent(db.Model):
    """Explains every change to a risk score ("why did it go up?")."""
    __tablename__ = "risk_events"

    id = db.Column(db.BigInteger().with_variant(db.Integer, "sqlite"), primary_key=True)
    user_id = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="CASCADE"), nullable=False,
        index=True)
    activity_id = db.Column(db.BigInteger, db.ForeignKey("activity_logs.id", ondelete="SET NULL"))
    rule_key = db.Column(db.String(40), nullable=False)
    points = db.Column(db.Integer, nullable=False)
    reason = db.Column(db.String(500), nullable=False)
    score_before = db.Column(db.Integer, nullable=False)
    score_after = db.Column(db.Integer, nullable=False)
    created_at = db.Column(db.DateTime, nullable=False, default=now, index=True)

    user = db.relationship("User")

    def to_dict(self):
        return {
            "id": self.id,
            "employee_id": self.user.employee_id if self.user else None,
            "name": self.user.name if self.user else None,
            "activity_id": self.activity_id,
            "rule_key": self.rule_key,
            "points": self.points,
            "reason": self.reason,
            "score_before": self.score_before,
            "score_after": self.score_after,
            "created_at": iso(self.created_at),
        }
}

```

```

class RiskRule(db.Model):
    """Configurable detection rule: managers can tune points/thresholds in Settings."""
    __tablename__ = "risk_rules"

    id = db.Column(db.Integer, primary_key=True)
    rule_key = db.Column(db.String(40), unique=True, nullable=False)
    name = db.Column(db.String(100), nullable=False)
    description = db.Column(db.String(255))
    points = db.Column(db.Integer, nullable=False, default=0)
    threshold = db.Column(db.Integer, nullable=False, default=1)
    window_minutes = db.Column(db.Integer, nullable=False, default=0)
    severity = db.Column(db.Enum(*RULE_SEVERITIES, name="rule_severity"), nullable=False, default="NONE")
    enabled = db.Column(db.Boolean, nullable=False, default=True)
    updated_at = db.Column(db.DateTime, nullable=False, default=now, onupdate=now)

    def to_dict(self):
        return {
            "id": self.id,
            "rule_key": self.rule_key,
            "name": self.name,
            "description": self.description,
            "points": self.points,
            "threshold": self.threshold,
            "window_minutes": self.window_minutes,
            "severity": self.severity,
            "enabled": self.enabled,
            "updated_at": iso(self.updated_at),
        }

```

B.5 models/record.py

```

"""
Business records: the "normal work" employees do inside the application.
Access to them is what the monitoring system watches.
"""

from extensions import db
from utils.helpers import iso, now

SENSITIVITY_LEVELS = ("PUBLIC", "INTERNAL", "CONFIDENTIAL", "RESTRICTED")
RECORD_CATEGORIES = ("Report", "Contract", "Customer", "Invoice", "Policy", "Personnel", "Technical",
"Other")
SHARED_DEPARTMENT = "General" # company-wide records

class Record(db.Model):
    __tablename__ = "records"

    id = db.Column(db.Integer, primary_key=True)
    record_code = db.Column(db.String(20), unique=True, nullable=False)
    title = db.Column(db.String(150), nullable=False)
    category = db.Column(db.String(30), nullable=False, default="Other")
    department = db.Column(db.String(60), nullable=False, index=True)
    sensitivity = db.Column(db.Enum(*SENSITIVITY_LEVELS, name="record_sensitivity"), nullable=False,
default="INTERNAL")
    content = db.Column(db.Text, nullable=False)
    created_by = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    updated_by = db.Column(db.Integer, db.ForeignKey("users.id", ondelete="SET NULL"))
    created_at = db.Column(db.DateTime, nullable=False, default=now)
    updated_at = db.Column(db.DateTime, nullable=False, default=now, onupdate=now)

    creator = db.relationship("User", foreign_keys=[created_by])
    updater = db.relationship("User", foreign_keys=[updated_by])

    @property
    def is_sensitive(self):
        return self.sensitivity == "CONFIDENTIAL"

    def to_dict(self):
        return {

```

```

        "id": self.id,
        "record_code": self.record_code,
        "title": self.title,
        "category": self.category,
        "department": self.department,
        "sensitivity": self.sensitivity,
        "content": self.content,
        "created_at": iso(self.created_at),
        "updated_at": iso(self.updated_at),
    }

```

B.6 services/activity_logger.py

```

"""
Centralised activity logging – the single entry point of the monitoring pipeline.

Employee action -> route -> log_activity()
1. insert activity_logs row
2. risk engine evaluates it
3. risk_scores / risk_events updated
4. alerts created if rules with a severity fired
5. COMMIT (all of the above in one transaction)
6. Socket.IO push to managers (only after the data is safely stored)
"""
import logging

from flask import current_app
from sqlalchemy.exc import SQLAlchemyError

from extensions import db
from models import ActivityLog
from services import alert_service, notification_service, risk_engine
from utils.helpers import client_ip, client_user_agent, now

log = logging.getLogger(__name__)

def log_activity(user, action, resource=None, description=None, risk_score=0, *,
                status="SUCCESS", employee_id=None, record_count=0, context=None):
    """
    Record `action` performed by `user` (None for unknown login IDs) and run it
    through the risk pipeline. `risk_score` is an optional base score added on
    top of whatever the rules decide. Returns the stored ActivityLog.
    """
    emp_id = user.employee_id if user is not None else (employee_id or "UNKNOWN")
    activity = ActivityLog(
        user_id=user.id if user is not None else None,
        employee_id=emp_id[:20],
        actor_role=user.role if user is not None else None,
        action=action,
        resource=(resource or "")[:255] or None,
        description=(description or "")[:500] or None,
        record_count=record_count or 0,
        timestamp=now(),
        ip_address=client_ip(),
        user_agent=client_user_agent(),
        status=status,
    )
    alerts = []
    try:
        db.session.add(activity)
        db.session.flush() # gives the activity an id so rules/alerts can reference it

        triggers = risk_engine.evaluate(activity, user, context)
        if risk_score:
            triggers.insert(0, risk_engine.Trigger("BASE_SCORE", int(risk_score),
                                                    f"Base score for {action}", "NONE"))
        cap = current_app.config["RISK_SCORE_CAP"]
        activity.risk_score = min(cap, sum(t.points for t in triggers))
        activity.risk_reason = "; ".join(f"{t.points} {t.reason}" for t in triggers)[:1000] or None

        level_before = level_after = score = None
        if user is not None and activity.risk_score > 0:

```

```

        level_before, level_after, score = risk_engine.apply_score(user, activity, triggers)

        alerts = alert_service.generate_alerts(user, activity, triggers, level_before, level_after, score)
        db.session.commit()
    except SQLAlchemyError:
        db.session.rollback()
        log.exception("Risk pipeline failed for %s %s – storing the bare activity", emp_id, action)
        alerts = []
        activity = _store_bare(user, emp_id, action, resource, description, status, record_count)
        if activity is None:
            return None

    notification_service.emit_activity(activity)
    for alert in alerts:
        notification_service.emit_alert(alert)
    return activity

def _store_bare(user, emp_id, action, resource, description, status, record_count):
    """Fallback so an activity is never silently lost if scoring fails."""
    try:
        activity = ActivityLog(user_id=user.id if user is not None else None, employee_id=emp_id[:20],
                               actor_role=user.role if user is not None else None, action=action,
                               resource=(resource or "")[:255] or None,
                               description=(description or "")[:500] or None,
                               record_count=record_count or 0, timestamp=now(), ip_address=client_ip(),
                               user_agent=client_user_agent(), status=status)

        db.session.add(activity)
        db.session.commit()
        return activity
    except SQLAlchemyError:
        db.session.rollback()
        log.exception("Could not store activity %s for %s", action, emp_id)
        return None

```

B.7 services/risk_engine.py

```

"""
Explainable, rule-based risk engine.

Every activity is evaluated against the enabled rules in the `risk_rules`
table. Each rule that fires produces a Trigger with the number of points it
adds and a human readable reason. Points are added to the employee's
cumulative score (capped at 100) and every change is written to
`risk_events`, so a manager can always see *why* a score went up.

Rule thresholds, windows, points and alert severities are data, not code:
managers can tune them from Manager > Settings.
"""
from dataclasses import dataclass
from datetime import timedelta

from flask import current_app, g

from extensions import db
from models import ActivityLog, Alert, RiskEvent, RiskRule, RiskScore
from utils.helpers import now, risk_level_for

LEVEL_ORDER = {"LOW": 0, "MEDIUM": 1, "HIGH": 2, "CRITICAL": 3}

# rule_key, name, description, points, threshold, window_minutes, severity
# (kept in sync with the INSERT in database/schema.sql)
DEFAULT_RULES = [
    ("FAILED_LOGIN", "Failed login", "Each failed login attempt.", 10, 1, 0, "NONE"),
    ("REPEATED_FAILED_LOGIN", "Repeated failed logins", "Threshold failed logins for one employee ID within
the window.", 20, 3, 10, "HIGH"),
    ("NEW_LOCATION", "Login from new location", "Successful login from an IP address this employee has never
used before.", 10, 1, 0, "MEDIUM"),
    ("OFF_HOURS", "Outside working hours", "Activity outside configured working hours/days (counted once per
window).", 15, 1, 60, "MEDIUM"),
    ("SENSITIVE_ACCESS", "Sensitive record access", "Viewing or editing a CONFIDENTIAL record.", 15, 1, 0,
"MEDIUM"),

```

```

    ("UNAUTHORIZED_ACCESS", "Unauthorized access attempt", "Attempt to open a record or page outside the
employee's permissions.", 30, 1, 0, "HIGH"),
    ("PRIVILEGE_ESCALATION", "Privilege escalation attempt", "Employee attempted to use a manager-only page
or API.", 10, 1, 0, "HIGH"),
    ("REPEATED_UNAUTHORIZED", "Repeated unauthorized attempts", "Threshold unauthorized attempts within the
window.", 25, 3, 15, "CRITICAL"),
    ("MASS_RECORD_ACCESS", "Large number of records accessed", "Threshold or more records viewed/exported
within the window.", 20, 15, 5, "HIGH"),
    ("REPEATED_SEARCH", "Repeated searches", "Threshold searches within the window.", 10, 10, 5, "LOW"),
    ("RAPID_ACTIONS", "Multiple rapid actions", "Threshold actions of any kind within the window.", 15, 20,
1, "MEDIUM"),
    ("UNUSUAL_FREQUENCY", "Unusual access frequency", "Actions in the window exceed threshold x the
employee's normal hourly rate (7-day baseline).", 15, 3, 60, "MEDIUM"),
    ("SUSPICIOUS_BURST", "Multiple suspicious events", "Threshold risk-scored events (excluding failed
logins) within the window.", 25, 3, 10, "HIGH"),
]

DATA_ACTIONS = {"SEARCH", "VIEW_RECORD", "ADD_RECORD", "UPDATE_RECORD", "EXPORT_DATA"}
RECORD_ACCESS_ACTIONS = ("VIEW_RECORD", "EXPORT_DATA")
LOGIN_RULES = {"FAILED_LOGIN", "REPEATED_FAILED_LOGIN", "NEW_LOCATION"}
MANAGER_PATH_PREFIXES = ("/manager", "/api/manager")
DAY_NAMES = ["Monday", "Tuesday", "Wednesday", "Thursday", "Friday", "Saturday", "Sunday"]

@dataclass
class Trigger:
    rule_key: str
    points: int
    reason: str
    severity: str # NONE / LOW / MEDIUM / HIGH / CRITICAL

# ----- rule store

def ensure_default_rules():
    """Insert any default rule that is missing (never overwrites tuned values)."""
    existing = {key for (key,) in db.session.query(RiskRule.rule_key).all()}
    added = False
    for key, name, desc, points, threshold, window, severity in DEFAULT_RULES:
        if key not in existing:
            db.session.add(RiskRule(rule_key=key, name=name, description=desc, points=points,
                                   threshold=threshold, window_minutes=window, severity=severity))
            added = True
    if added:
        db.session.commit()

def active_rules():
    """Enabled rules keyed by rule_key, cached for the current request."""
    if "risk_rules" not in g:
        g.risk_rules = {r.rule_key: r for r in RiskRule.query.filter_by(enabled=True).all()}
    return g.risk_rules

# ----- query helpers

def _count_actions(employee_id, since, actions=None, extra=None):
    q = ActivityLog.query.filter(ActivityLog.employee_id == employee_id, ActivityLog.timestamp >= since)
    if actions:
        q = q.filter(ActivityLog.action.in_(actions))
    if extra is not None:
        q = q.filter(extra)
    return q.count()

def _fired_recently(user, employee_id, rule_key, since):
    """Aggregate rules fire once per window, then stay quiet until it passes."""
    if user is not None:
        return db.session.query(RiskEvent.id).filter(
            RiskEvent.user_id == user.id, RiskEvent.rule_key == rule_key,
            RiskEvent.created_at >= since).first() is not None
    return db.session.query(Alert.id).filter(

```

```

        Alert.employee_id == employee_id, Alert.alert_type == rule_key,
        Alert.timestamp >= since).first() is not None

def _is_off_hours(ts):
    cfg = current_app.config
    return ts.weekday() not in cfg["WORK_DAYS"] or not (cfg["WORK_HOURS_START"] <= ts.hour <
    cfg["WORK_HOURS_END"])

# ----- rules
# Each rule: (activity, user, context, rule) -> Trigger | None

def _failed_login(a, user, ctx, rule):
    if a.action != "FAILED_LOGIN":
        return None
    why = ctx.get("failure_reason", "invalid credentials")
    return Trigger(rule.rule_key, rule.points, f"Failed login for {a.employee_id} ({why})", rule.severity)

def _repeated_failed_login(a, user, ctx, rule):
    if a.action != "FAILED_LOGIN":
        return None
    since = a.timestamp - timedelta(minutes=rule.window_minutes)
    failures = _count_actions(a.employee_id, since, ["FAILED_LOGIN"])
    if failures < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"{failures} failed login attempts for {a.employee_id} within {rule.window_minutes}
minutes",
        rule.severity)

def _new_location(a, user, ctx, rule):
    if a.action not in ("LOGIN", "MANAGER_LOGIN") or user is None or not a.ip_address:
        return None
    since = a.timestamp - timedelta(days=90)
    previous = db.session.query(ActivityLog.ip_address).filter(
        ActivityLog.user_id == user.id, ActivityLog.action.in_(["LOGIN", "MANAGER_LOGIN"]),
        ActivityLog.timestamp >= since, ActivityLog.id != a.id).distinct().all()
    known = {ip for (ip,) in previous if ip}
    if not known or a.ip_address in known: # first ever login is not "unusual"
        return None
    shown = ", ".join(sorted(known)[:3])
    return Trigger(rule.rule_key, rule.points,
        f"Login from new IP address {a.ip_address} (previously used: {shown})", rule.severity)

def _off_hours(a, user, ctx, rule):
    if a.action not in DATA_ACTIONS | {"LOGIN"} or not _is_off_hours(a.timestamp):
        return None
    since = a.timestamp - timedelta(minutes=max(rule.window_minutes, 1))
    if _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    cfg = current_app.config
    return Trigger(rule.rule_key, rule.points,
        f"{a.action.replace('_', ' ').title()} at {a.timestamp:%H:%M} on
{DAY_NAMES[a.timestamp.weekday()]}"
        f"is outside working hours ({cfg['WORK_HOURS_START']:02d}:00-
{cfg['WORK_HOURS_END']:02d}:00, Mon-Fri)",
        rule.severity)

def _sensitive_access(a, user, ctx, rule):
    if a.action not in ("VIEW_RECORD", "UPDATE_RECORD") or ctx.get("sensitivity") != "CONFIDENTIAL":
        return None
    verb = "Viewed" if a.action == "VIEW_RECORD" else "Modified"
    return Trigger(rule.rule_key, rule.points, f"{verb} CONFIDENTIAL record {a.resource}", rule.severity)

```

```

def _unauthorized(a, user, ctx, rule):
    if a.action != "UNAUTHORIZED_ACCESS":
        return None
    return Trigger(rule.rule_key, rule.points,
        f"Attempted to access {a.resource} without permission", rule.severity)

def _privilege_escalation(a, user, ctx, rule):
    if a.action != "UNAUTHORIZED_ACCESS" or not (a.resource or "").startswith(MANAGER_PATH_PREFIXES):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"Employee tried to open manager-only resource {a.resource}", rule.severity)

def _repeated_unauthorized(a, user, ctx, rule):
    if a.action != "UNAUTHORIZED_ACCESS":
        return None
    since = a.timestamp - timedelta(minutes=rule.window_minutes)
    attempts = _count_actions(a.employee_id, since, ["UNAUTHORIZED_ACCESS"])
    if attempts < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"{attempts} unauthorized access attempts within {rule.window_minutes} minutes",
rule.severity)

def _mass_record_access(a, user, ctx, rule):
    if a.action not in RECORD_ACCESS_ACTIONS:
        return None
    since = a.timestamp - timedelta(minutes=rule.window_minutes)
    total = db.session.query(db.func.coalesce(db.func.sum(ActivityLog.record_count), 0)).filter(
        ActivityLog.employee_id == a.employee_id, ActivityLog.timestamp >= since,
        ActivityLog.action.in_(RECORD_ACCESS_ACTIONS), ActivityLog.status == "SUCCESS").scalar()
    if total < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"Accessed {int(total)} records within {rule.window_minutes} minutes", rule.severity)

def _repeated_search(a, user, ctx, rule):
    if a.action != "SEARCH":
        return None
    since = a.timestamp - timedelta(minutes=rule.window_minutes)
    searches = _count_actions(a.employee_id, since, ["SEARCH"])
    if searches < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"{searches} searches within {rule.window_minutes} minutes", rule.severity)

def _rapid_actions(a, user, ctx, rule):
    since = a.timestamp - timedelta(minutes=max(rule.window_minutes, 1))
    actions = _count_actions(a.employee_id, since)
    if actions < rule.threshold or _fired_recently(user, a.employee_id, rule.rule_key, since):
        return None
    return Trigger(rule.rule_key, rule.points,
        f"{actions} actions within {max(rule.window_minutes, 1)} minute(s)", rule.severity)

def _unusual_frequency(a, user, ctx, rule):
    window = max(rule.window_minutes, 1)
    since = a.timestamp - timedelta(minutes=window)
    recent = _count_actions(a.employee_id, since)
    if recent < current_app.config["UNUSUAL_FREQUENCY_MIN_ACTIONS"]:
        return None
    baseline_rows = db.session.query(ActivityLog.timestamp).filter(
        ActivityLog.employee_id == a.employee_id,
        ActivityLog.timestamp >= a.timestamp - timedelta(days=7),
        ActivityLog.timestamp < since).all()
    if not baseline_rows:
        return None # no history yet, nothing to compare against
    active_hours = {ts.replace(minute=0, second=0) for (ts,) in baseline_rows}

```

```

normal_rate = len(baseline_rows) / len(active_hours) * (window / 60)
if recent <= rule.threshold * normal_rate or _fired_recently(user, a.employee_id, rule.rule_key, since):
    return None
return Trigger(rule.rule_key, rule.points,
               f"{recent} actions in the last {window} minutes vs a normal rate of ~{normal_rate:.0f}",
               rule.severity)

RULES = [
    ("FAILED_LOGIN", _failed_login),
    ("REPEATED_FAILED_LOGIN", _repeated_failed_login),
    ("NEW_LOCATION", _new_location),
    ("OFF_HOURS", _off_hours),
    ("SENSITIVE_ACCESS", _sensitive_access),
    ("UNAUTHORIZED_ACCESS", _unauthorized),
    ("PRIVILEGE_ESCALATION", _privilege_escalation),
    ("REPEATED_UNAUTHORIZED", _repeated_unauthorized),
    ("MASS_RECORD_ACCESS", _mass_record_access),
    ("REPEATED_SEARCH", _repeated_search),
    ("RAPID_ACTIONS", _rapid_actions),
    ("UNUSUAL_FREQUENCY", _unusual_frequency),
]

# ----- evaluation

def evaluate(activity, user, context=None):
    """Return the list of Triggers fired by this (already flushed) activity."""
    ctx = context or {}
    rules = active_rules()
    is_employee = user is not None and user.role == "employee"
    triggers = []
    for key, check in RULES:
        rule = rules.get(key)
        if rule is None:
            continue
        if not is_employee and key not in LOGIN_RULES:
            continue # managers / unknown IDs are only evaluated for login behaviour
        trig = check(activity, user, ctx, rule)
        if trig:
            triggers.append(trig)

    # Failed logins escalate through their own rule, so typos never count as a "burst".
    burst = rules.get("SUSPICIOUS_BURST")
    if (burst and is_employee and activity.action != "FAILED_LOGIN"
        and any(t.points > 0 for t in triggers)):
        since = activity.timestamp - timedelta(minutes=burst.window_minutes)
        earlier = _count_actions(activity.employee_id, since,
                                extra=db.and_(ActivityLog.risk_score > 0, ActivityLog.id != activity.id,
                                                ActivityLog.action != "FAILED_LOGIN"))
        if earlier + 1 >= burst.threshold and not _fired_recently(user, activity.employee_id, burst.rule_key,
since):
            triggers.append(Trigger(burst.rule_key, burst.points,
                                   f"{earlier + 1} suspicious events within {burst.window_minutes} minutes",
                                   burst.severity))

    return triggers

def get_or_create_score(user):
    if user.risk is None:
        user.risk = RiskScore(user_id=user.id, current_score=0, risk_level="LOW", last_updated=now())
        db.session.add(user.risk)
    return user.risk

def apply_score(user, activity, triggers):
    """Add trigger points to the user's score and record why. Returns (level_before, level_after, score)."""
    cap = current_app.config["RISK_SCORE_CAP"]
    rs = get_or_create_score(user)
    level_before = rs.risk_level
    score = rs.current_score
    for t in triggers:
        if t.points <= 0:

```

```

        continue
    after = min(cap, score + t.points)
    db.session.add(RiskEvent(user_id=user.id, activity_id=activity.id, rule_key=t.rule_key,
                             points=t.points, reason=t.reason, score_before=score, score_after=after,
                             created_at=activity.timestamp))

    score = after
    if score != rs.current_score:
        rs.current_score = score
        rs.risk_level = risk_level_for(score)
        rs.last_updated = activity.timestamp
        rs.last_increase_at = activity.timestamp
    return level_before, rs.risk_level, score

def apply_decay():
    """Slowly lower scores of employees with no recent risky behaviour."""
    cfg = current_app.config
    per_hour = cfg["RISK_DECAY_PER_HOUR"]
    if per_hour <= 0:
        return 0
    current = now()
    grace = current - timedelta(minutes=cfg["RISK_DECAY_GRACE_MINUTES"])
    changed = 0
    for rs in RiskScore.query.filter(RiskScore.current_score > 0).all():
        if rs.last_increase_at and rs.last_increase_at > grace:
            continue
        hours = (current - rs.last_updated).total_seconds() / 3600
        decrease = int(hours * per_hour)
        if decrease < 1:
            continue
        rs.current_score = max(0, rs.current_score - decrease)
        rs.risk_level = risk_level_for(rs.current_score)
        rs.last_updated = rs.last_updated + timedelta(hours=decrease / per_hour)
        changed += 1
    if changed:
        db.session.commit()
    return changed

```

B.8 services/alert_service.py

```

"""
Alert generation and lifecycle (NEW -> ACKNOWLEDGED -> RESOLVED).
Alerts are never deleted: they form part of the audit trail.
"""
from datetime import timedelta

from flask import current_app

from extensions import db
from models import Alert
from services.risk_engine import LEVEL_ORDER
from utils.helpers import now

def _in_cooldown(employee_id, alert_type, at):
    since = at - timedelta(minutes=current_app.config["ALERT_COOLDOWN_MINUTES"])
    return db.session.query(Alert.id).filter(
        Alert.employee_id == employee_id, Alert.alert_type == alert_type,
        Alert.status != "RESOLVED", Alert.timestamp >= since).first() is not None

def generate_alerts(user, activity, triggers, level_before=None, level_after=None, score=None):
    """Create alerts for triggered rules that carry a severity. Returns new Alert objects (not committed)."""
    employee_id = activity.employee_id
    risk_score = score if score is not None else (user.risk.current_score if user and user.risk else
    activity.risk_score)
    created = []
    for t in triggers:
        if t.severity == "NONE" or _in_cooldown(employee_id, t.rule_key, activity.timestamp):
            continue
        alert = Alert(user_id=user.id if user else None, employee_id=employee_id, activity_id=activity.id,
                      alert_type=t.rule_key, severity=t.severity, message=t.reason[:500],
                      risk_score=risk_score, timestamp=activity.timestamp, status="NEW")

```

```

        db.session.add(alert)
        created.append(alert)

    escalated = (user is not None and level_before and level_after
                 and LEVEL_ORDER[level_after] > LEVEL_ORDER[level_before]
                 and level_after in ("HIGH", "CRITICAL"))
    if escalated:
        latest = triggers[-1].reason if triggers else activity.action
        alert = Alert(user_id=user.id, employee_id=employee_id, activity_id=activity.id,
                     alert_type="RISK_LEVEL_ESCALATION", severity=level_after,
                     message=f"Risk level rose from {level_before} to {level_after} (score {risk_score}). "
                             f"Latest: {latest}":500],
                     risk_score=risk_score, timestamp=activity.timestamp, status="NEW")
        db.session.add(alert)
        created.append(alert)
    return created

def raise_alert(user, alert_type, severity, message, activity=None):
    """Create a single alert outside the rule engine (e.g. privilege changes)."""
    alert = Alert(user_id=user.id, employee_id=user.employee_id,
                  activity_id=activity.id if activity else None, alert_type=alert_type,
                  severity=severity, message=message[:500],
                  risk_score=user.risk.current_score if user.risk else 0,
                  timestamp=now(), status="NEW")
    db.session.add(alert)
    return alert

class AlertStateError(ValueError):
    pass

def acknowledge(alert, manager):
    if alert.status != "NEW":
        raise AlertStateError(f"Alert is already {alert.status.lower()}.")
    alert.status = "ACKNOWLEDGED"
    alert.acknowledged_by = manager.id
    alert.acknowledged_at = now()

def resolve(alert, manager, note=None):
    if alert.status == "RESOLVED":
        raise AlertStateError("Alert is already resolved.")
    if alert.status == "NEW": # resolving implies it was seen
        alert.acknowledged_by = manager.id
        alert.acknowledged_at = now()
    alert.status = "RESOLVED"
    alert.resolved_by = manager.id
    alert.resolved_at = now()
    alert.resolution_note = (note or "").strip()[:500] or None

```

B.9 utils/security.py

```

"""
Role-based access control. Authorization is always enforced here on the
server; hiding buttons in the UI is only cosmetic.
"""
from functools import wraps

from flask import abort, jsonify, redirect, request, url_for
from flask_login import current_user

from extensions import login_manager
from utils.helpers import wants_json

def forbidden(message="You do not have permission to access this resource."):
    if wants_json():
        response = jsonify({"error": "forbidden", "message": message})

```

```

        response.status_code = 403
        return response
    abort(403)

def login_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not current_user.is_authenticated:
            return login_manager.unauthorized()
        return view(*args, **kwargs)
    return wrapped

def manager_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not current_user.is_authenticated:
            return login_manager.unauthorized()
        if not current_user.is_manager:
            # an employee probing manager URLs is itself a monitored event
            from services.activity_logger import log_activity
            log_activity(current_user, "UNAUTHORIZED_ACCESS", resource=request.path,
                        description=f"Blocked {request.method} to manager-only resource",
                        status="DENIED")
            return forbidden()
        return view(*args, **kwargs)
    return wrapped

def employee_required(view):
    @wraps(view)
    def wrapped(*args, **kwargs):
        if not current_user.is_authenticated:
            return login_manager.unauthorized()
        if current_user.is_manager:
            if wants_json():
                return forbidden("This endpoint is for employee accounts.")
            return redirect(url_for("manager.dashboard"))
        return view(*args, **kwargs)
    return wrapped

def home_url_for(user):
    return url_for("manager.dashboard") if user.is_manager else url_for("employee.dashboard")

def is_safe_next(target):
    return bool(target) and target.startswith("/") and not target.startswith("//") and "\\" not in target

```

B.10 routes/auth.py

```

"""Authentication: login (with lockout), logout, password change."""
from datetime import timedelta

from flask import (Blueprint, current_app, flash, redirect, render_template, request,
                  session, url_for)
from flask_login import current_user, login_user, logout_user

from extensions import db
from models import LoginAttempt, User
from services.activity_logger import log_activity
from services.notification_service import emit_presence
from utils.helpers import client_ip, client_user_agent, now
from utils.security import home_url_for, is_safe_next, login_required
from utils.validators import normalize_employee_id, validate_password

auth_bp = Blueprint("auth", __name__)

```

```

GENERIC_LOGIN_ERROR = "Invalid employee ID or password."

def _recent_failures(**filters):
    since = now() - timedelta(minutes=current_app.config["LOGIN_LOCKOUT_MINUTES"])
    return LoginAttempt.query.filter_by(success=False, **filters).filter(LoginAttempt.timestamp >=
since).count()

def _is_locked_out(employee_id, ip):
    cfg = current_app.config
    return (_recent_failures(employee_id=employee_id) >= cfg["LOGIN_MAX_FAILURES"]
        or _recent_failures(ip_address=ip) >= cfg["LOGIN_IP_MAX_FAILURES"])

def _record_attempt(employee_id, user, success, reason):
    db.session.add(LoginAttempt(employee_id=employee_id[:20] or "UNKNOWN", user_id=user.id if user else None,
        ip_address=client_ip(), user_agent=client_user_agent(),
        timestamp=now(), success=success, reason=reason))

def _fail(employee_id, user, reason, description, message, status_code=401):
    """Store the failed attempt + FAILED_LOGIN activity (committed together by log_activity)."""
    _record_attempt(employee_id, user, False, reason)
    log_activity(user, "FAILED_LOGIN", resource="/login", description=description, status="FAILED",
        employee_id=employee_id or "UNKNOWN", context={"failure_reason": reason.lower().replace("_",
" ")})
    return render_template("login.html", error=message, employee_id=employee_id), status_code

@auth_bp.route("/")
def login_page():
    if current_user.is_authenticated:
        return redirect(home_url_for(current_user))
    return render_template("login.html")

@auth_bp.route("/login", methods=["POST"])
def login():
    employee_id = normalize_employee_id(request.form.get("employee_id"))
    password = request.form.get("password") or ""
    if not employee_id or not password:
        return render_template("login.html", error="Enter your employee ID and password.",
            employee_id=employee_id), 400

    ip = client_ip()
    user = User.query.filter_by(employee_id=employee_id).first()

    if _is_locked_out(employee_id, ip):
        minutes = current_app.config["LOGIN_LOCKOUT_MINUTES"]
        return _fail(employee_id, user, "LOCKED_OUT", "Login blocked: too many failed attempts",
            f"Too many failed attempts. Try again in {minutes} minutes.", 429)

    if user is None:
        return _fail(employee_id, None, "UNKNOWN_EMPLOYEE_ID", "Login with unknown employee ID",
            GENERIC_LOGIN_ERROR)
    if not user.check_password(password):
        return _fail(employee_id, user, "INVALID_PASSWORD", "Incorrect password", GENERIC_LOGIN_ERROR)
    if user.status != "active":
        return _fail(employee_id, user, "ACCOUNT_DISABLED", "Login attempt on a disabled account",
            "This account is disabled. Please contact your manager.", 403)

    # ---- success
    session.clear() # prevent session fixation
    login_user(user)
    session.permanent = True
    user.last_login = now()
    user.last_login_ip = ip
    user.last_seen = now()
    user.is_online = True

```

```

_record_attempt(employee_id, user, True, "SUCCESS")
action = "MANAGER_LOGIN" if user.is_manager else "LOGIN"
log_activity(user, action, resource="/login", description="Signed in")
emit_presence(user)

if user.must_change_password:
    flash("Please set a new password before continuing.", "info")
    return redirect(url_for("auth.change_password"))
target = request.args.get("next")
return redirect(target if is_safe_next(target) else home_url_for(user))

@auth_bp.route("/logout", methods=["POST"])
@login_required
def logout():
    user = current_user._get_current_object()
    log_activity(user, "LOGOUT", resource="/logout", description="Signed out")
    user.is_online = False
    db.session.commit()
    emit_presence(user)
    logout_user()
    session.clear()
    flash("You have been signed out securely.", "success")
    return redirect(url_for("auth.login_page"))

@auth_bp.route("/change-password", methods=["GET", "POST"])
@login_required
def change_password():
    forced = current_user.must_change_password
    if request.method == "POST":
        current_pw = request.form.get("current_password") or ""
        new_pw = request.form.get("new_password") or ""
        confirm = request.form.get("confirm_password") or ""
        error = None
        if not current_user.check_password(current_pw):
            error = "Your current password is incorrect."
        elif new_pw != confirm:
            error = "The new passwords do not match."
        elif new_pw == current_pw:
            error = "The new password must be different from the current one."
        else:
            error = validate_password(new_pw, current_app.config["PASSWORD_MIN_LENGTH"])
        if error:
            flash(error, "error")
            return render_template("change_password.html", forced=forced), 400

        current_user.set_password(new_pw)
        current_user.must_change_password = False
        log_activity(current_user._get_current_object(), "PASSWORD_CHANGE", resource="/change-password",
                    description="Changed own password" + (" (first login)" if forced else ""))
        flash("Password updated successfully.", "success")
        return redirect(home_url_for(current_user))
    return render_template("change_password.html", forced=forced)

```

B.11 templates/login.html (interface)

```

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>SHRUTU | Sign in</title>
    <link rel="icon" href="{{ url_for('static', filename='images/logo.svg') }}" type="image/svg+xml">
    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link href="https://fonts.googleapis.com/css2?family=Inter:wght@400;500;600;700&display=swap"
rel="stylesheet">
    <link rel="stylesheet" href="{{ url_for('static', filename='css/login.css') }}">
</head>
<body>
<div class="glow glow-one"></div>
<div class="glow glow-two"></div>

```

```

<div class="login-container">
  <section class="brand-section">
    <div class="brand">
      <div class="brand-icon">{% include "partials/logo_mark.html" %}</div>
      <div>
        <div class="brand-name">SHRUTU</div>
        <div class="brand-subtitle">SECURITY INTELLIGENCE</div>
      </div>
    </div>
    <h1 class="welcome-title">Protect your organization<br><span>before threats become
incidents.</span></h1>
    <p class="description">
      SHRUTU Insider Threat Monitoring gives security managers real-time visibility into
      employee activity, explainable risk scores and instant alerts on suspicious behaviour.
    </p>
    <div class="features">
      <div class="feature"><div class="feature-icon">✓</div>Real-time threat monitoring</div>
      <div class="feature"><div class="feature-icon">✓</div>Explainable employee risk scoring</div>
      <div class="feature"><div class="feature-icon">✓</div>Instant manager alerts</div>
      <div class="feature"><div class="feature-icon">✓</div>Complete, tamper-evident audit trail</div>
    </div>
    <div class="security-orbit"></div>
  </section>

  <section class="login-section">
    <div class="login-heading">
      <h2>Welcome back</h2>
      <p>Sign in with your employee ID to access the console.</p>
    </div>

    {% with messages = get_flashed_messages(with_categories=true) %}
      {% for category, message in messages %}
        <div class="flash {{ category }}" role="status">{{ message }}</div>
      {% endfor %}
    {% endwith %}
    {% if error %}
      <div class="error-message" role="alert">⚠ {{ error }}</div>
    {% endif %}

    <form method="POST" action="{% url_for('auth.login', next=request.args.get('next')) %}"
autocomplete="on" novalidate>
      <input type="hidden" name="csrf_token" value="{% csrf_token() %}">
      <div class="form-group">
        <label for="employee_id">EMPLOYEE ID</label>
        <div class="input-wrapper">
          <span class="input-icon">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-
linecap="round" stroke-linejoin="round"><rect x="3" y="4" width="18" height="16" rx="2"/><circle cx="9"
cy="10" r="2"/><path d="M15 8h2M15 12h2M7 16h10"/></svg>
          </span>
          <input type="text" id="employee_id" name="employee_id" class="uppercase"
placeholder="e.g. EMP1001"
value="{% employee_id or '' %}" maxlength="20" required autocomplete="username"
autofocus>
        </div>
      </div>
      <div class="form-group">
        <label for="password">PASSWORD</label>
        <div class="input-wrapper">
          <span class="input-icon">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-
linecap="round" stroke-linejoin="round"><rect x="4" y="11" width="16" height="10" rx="2"/><path d="M8 11V7a4
4 0 0 1 8 0v4"/></svg>
          </span>
          <input type="password" id="password" name="password" placeholder="Enter your password"
required autocomplete="current-password" maxlength="128">
          <button type="button" class="password-toggle" id="password-toggle" aria-label="Show
password">
            <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-
linecap="round" stroke-linejoin="round"><path d="M2 12s3.5-7 10-7 10 7-3.5 7-10 7S2 12 2 12z"/><circle
cx="12" cy="12" r="3"/></svg>
          </button>
        </div>
      </div>
      <div class="login-options">Forgot your password? Ask your manager to reset it.</div>
      <button type="submit" class="login-button">Sign in to Security Console →</button>
    </form>

```

```

        <div class="security-message">
            <span>🔒</span>
            <span><strong>Monitored system.</strong>&nbsp;</span> All sign-in attempts and activity are
recorded.</span>
        </div>
        <div class="footer">
            SHRUTU Security Intelligence Platform<br>
            <span>Authorized personnel only • v2.0</span>
        </div>
    </section>
</div>
<script src="{{ url_for('static', filename='js/login.js') }}"></script>
</body>
</html>

```

Appendix C — User manual

C.1 Running the project locally

```

python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env          (set MYSQL_PASSWORD and SECRET_KEY)
python init_db.py               (creates the database and demo data)
python app.py                   (open http://127.0.0.1:5000)
python -m pytest -q             (runs the 52 automated tests)

```

C.2 Demo accounts (local installation)

Table 10.1: Demo accounts

Role	Employee ID	Password	Notes
Manager	MGR1001	Manager@123	Ananya Iyer, IT & Security
Employee	EMP1001 – EMP1005, EMP1007 – EMP1016	Employee@123	Different seeded risk levels (LOW → CRITICAL)
Employee	EMP1000	Employee@123	Disabled account — login is refused

On the public deployment the demo passwords are replaced by private values set in the hosting environment.

C.3 Manager workflow

- Dashboard — watch KPIs, live feed and alerts.
- Alerts — click a row to investigate; Acknowledge while investigating; Resolve with a note to close.
- Employees — create accounts, edit, disable/enable, reset passwords.
- Risk Monitoring — see who is at which level and why.
- Audit Logs — review manager actions and all login attempts.
- Settings — tune the detection rules.

C.4 Employee workflow

- Log in with the employee ID; set a new password on first login.
- Records — search, open by code, add, edit and export records of your department.
- My Activity — review your own history; Profile — change password.

Appendix D — REST API endpoints

Table 10.2: *REST API endpoints*

Method	Endpoint	Role	Purpose
POST	/login, /logout	public / any	Authentication
GET	/api/health	public	Database status
GET	/api/manager/dashboard, /stats, /presence	manager	KPIs, charts, feed, presence
GET / POST	/api/manager/employees	manager	List (filters) / create employee
GET / PUT / DELETE	/api/manager/employees/<id>	manager	Details / update / disable
PUT	/api/manager/employees/<id>/status	manager	Enable or disable
POST	/api/manager/employees/<id>/reset-password	manager	Reset password
GET	/api/manager/activities[/<id>]	manager	Activity log (filters) / detail
GET	/api/manager/alerts[/<id>]	manager	Alerts (filters) / detail
PUT	/api/manager/alerts/<id>/acknowledge /resolve	manager	Handle alert
GET	/api/manager/risk, /audit, /login-attempts	manager	Risk overview, audit, attempts
GET / PUT	/api/manager/rules[/<id>]	manager	Detection rules
GET	/api/employee/activity, /profile	employee	Own data only